MASTER DSA - 3

Version 1.0

Contents:

I. RECURSION
   A. Basic working
      1. Pow(x, n)
      2. Subsets
      3. Climbing Stairs
      4. COMBINATION SUM
      5. PALINDROME PARTITION
      6. KTH PERMUTATIONS
      7. PRINT ALL PERMUTATIONS OF STRING
      8. M COLORING PROBLEM
      9. K-th Symbol in Grammar
      10. Find the Winner of the Circular Game
   B. BackTracking
      1. Queens problem
      2. Sudoku Solver
      3. Combinations
      4. Permutations I, II

II. DYNAMIC PROGRAMMING:
   A. Basic questions
      1. Stock Buy And Sell
      2. Maximum Product Subarray in an Array
   B. 1d array variations
      1. Two City Scheduling
      2. Coin change
      3. Longest Increasing Subsequence
      4. Number of Longest Increasing Subsequence
      5. Find the maximum sum in an array, such that you cannot add alternate elements(A) + house robber problem
   C. 2d Arrays variations
      1. Split Array Largest Sum
      2. Longest Common Subsequence
      3. Longest Palindromic Subsequence
      4. Maximum sum increasing subsequence
      5. Matrix Chain Multiplication
      6. Palindrome Partitioning (MCM Variation)
      7. Maximum sum path in the matrix
      8. Subset Sum
      9. Target Sum
      10. Minimum Swaps To Make Sequences Increasing
      11. Best Team With No Conflicts

12. **Jump Game I, II, III, IV, V**

D. **Others**

1. **Egg Dropping**

III. **GRIDS**

1. **Grid Unique Paths**
2. **Rotten oranges**
3. **No Of Island**

IV. **GRAPHS:**

A. **DFS & BFS**

1. **DFS & BFS**
2. **Clone a graph**
3. **Detect a cycle of Undirected using Bfs & Dfs**
4. **Detect a cycle of Directed graph using Bfs & Dfs**
5. **Bipartite check using Bfs & Dfs**

B. **SHORTEST PATH ALGOS (DIJKSTRA, BELLMAN FORD & FLOYD WARSHELL)**

1. **Calculate shortest distance using all 3 algos**
2.

C. **MIN SPANNING TREE (MST) (KRUSKAL'S & PRIMS)**

1. **Get MST using prims and kruskals**
2.

D. **TOPOSORT ALGO**

1. **KHAN'S ALGO (Bfs)**
2. **KOSARAJUS (Dfs)**

E. **UNION FIND/DISJOINT SET**

I. RECURSION:

1. Pow(x, n)

Implement pow(x, n), which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: x = 2.00000, n = 10 **Output:** 1024.00000

Example 2:

Input: x = 2.10000, n = 3 **Output:** 9.26100

Example 3:

Input: x = 2.00000, n = -2 **Output:** 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Approach - 1 recursive

```
class Solution {
    public double myPow(double x, int n) {
        if (n == 0) return 1;
        if (n < 0){
            return 1/x * myPow(1/x, -(n + 1));
        }
        return ((n&1) == 0) ? myPow(x*x, n/2) : x*myPow(x*x, n/2);
    }
}
```

Approach - 2 iterative.

```
class Solution {
    public double myPow(double x, int n) {
        if (n == 0 || x == 1) return 1;
        if (x == -1) return ((n & 1) == 0) ? 1: -1;
        if (n == Integer.MIN_VALUE) return 0;
        if (n < 0) {
            n = -n;
            x = 1/x;
        }
        double ans = 1;
```

```
        while(n > 0){
            if ((n&1) == 1){
                ans *= x;
            }
            x *= x;
            n >>= 1; //dividing by 2
        }
        return ans;
    }}
```

2. Subsets

Problem Statement: Given an array print all the sum of the subset generated from it, in the increasing order.

Example 1:

Input: N = 3, arr[] = {5,2,1} **Output:** 0,1,2,3,5,6,7,8

Explanation: We have to find all the subset's sum and print them.in this case the generated subsets are [[], [1], [2], [2,1], [5], [5,1], [5,2]. [5,2,1],so the sums we get will be 0,1,2,3,5,6,7,8

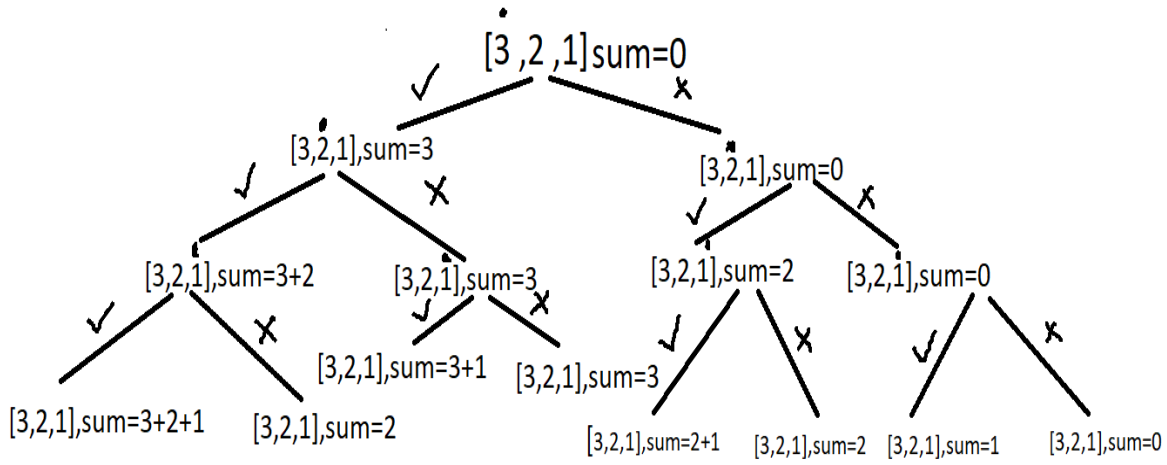
Input: N=3,arr[]= {3,1,2} **Output:** 0,1,2,3,3,4,5,6

Explanation: We have to find all the subset's sum and print them.in this case the generated subsets are [[], [1], [2], [2,1], [3], [3,1], [3,2]. [3,2,1],so the sums we get will be 0,1,2,3,3,4,5,6

Solution 1: Using recursion

Intuition: The main idea is that on every index you have two options either to select the element to add it to your subset(pick) or not select the element at that index and move to the next index(non-pick).

Approach: Traverse through the array and for each index solve for two arrays, one where you pick the element,i.e add the element to the sum or don't pick and move to the next element, recursively, until the base condition. Here when you reach the end of the array is the base condition.



```
    static ArrayList < Integer > subsetSums(ArrayList < Integer > arr, int N) {
        ArrayList < Integer > sumSubset = new ArrayList < > ();
        func(0, 0, arr, N, sumSubset);
        Collections.sort(sumSubset);
        return sumSubset;
    }

 static void func(int ind, int sum,
             ArrayList<Integer> arr, int N, ArrayList<Integer> sumSubset) {
        if (ind == N) {
            sumSubset.add(sum);
            return;
        }
        // pick the element
        func(ind + 1, sum + arr.get(ind), arr, N, sumSubset);
        // Do-not pick the element
        func(ind + 1, sum, arr, N, sumSubset);
    }
```

Time Complexity: O(2^n)+O(2^n log(2^n)). Each index has two ways. You can either pick it up or not pick it. So for n index time complexity for O(2^n) and for sorting it will take (2^n log(2^n)).
Space Complexity: O(2^n) for storing subset sums, since 2^n subsets can be generated for an array of size n.

VARIATION

Given an integer array `nums` that may contain duplicates, return *all possible subsets (the power set)*.

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1: `Input: nums = [1,2,2] Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]`

Example 2: `Input: nums = [0] Output: [[],[0]]`

`METHOD - 1 RECURSION`

```
public class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> lis = new ArrayList<>();
        bt(nums, 0, lis, new ArrayList<>());
        return lis;
    }

    private void bt(int[] nums, int ind,
                    List<List<Integer>> lis, List<Integer> cur) {
        lis.add(new ArrayList<>(cur));
```

```
        for(int i = ind; i < nums.length; i++) {
            if(i > ind && nums[i] == nums[i-1]) continue;   // avoid duplicates
            cur.add(nums[i]);
            bt(nums, i + 1, lis, cur);
            cur.remove(cur.size() - 1);
    }}}
```

METHOD - 2

Approach:

At every index, we make a decision whether to pick or not pick the element at that index. This will help us in generating all possible combinations but does not take care of the duplicates. Hence we will use a set to store all the combinations that will discard the duplicates.

```
    public static List < String > subsetsWithDup(int[] nums) {
        List < String > ans = new ArrayList < > ();
        HashSet < String > res = new HashSet < > ();
        List < Integer > ds = new ArrayList < > ();
        fun(nums, 0, ds, res);
        for (String it: res) {
            ans.add(it);
        }
        return ans;
    }

public static void fun(int[] nums, int index, List<Integer> ds, HashSet<String> res){
        if (index == nums.length) {
            Collections.sort(ds);
            res.add(ds.toString());
            return;
        }
        ds.add(nums[index]);
        fun(nums, index + 1, ds, res);
        ds.remove(ds.size() - 1);
        fun(nums, index + 1, ds, res);
    }
```

Solution 3: Optimal

Approach:

In the previous method, we were taking extra time to store the unique combination with the help of a set. To make the solution efficient we will have to decide on a method that will consider only the unique combinations without the help of additional data structure.

Lets understand with an example where arr = [1,2,2].

Initially start with an empty data structure. In the first recursion, call make a subset of size one, in the next recursion call a subset of size 2, and so on. But first, in order to make a subset of size one what options do we have?

We can pick up elements from either the first index or the second index or the third index. However, if we have already picked up two from the second index, picking up two from the third index will make another duplicate subset of size one. Since we are trying to avoid duplicate subsets we can

avoid picking up from the third index. This should give us an intuition that whenever there are duplicate elements in the array we pick up only the first occurrence.
The next recursion calls will continue from the point the previous one ended.
Let's summarize:-
Sort the input array.Make a recursive function that takes the input array ,the current subset,the current index and a list of list/ vector of vectors to contain the answer. Try to make a subset of size n during the nth recursion call and consider elements from every index while generating the combinations.
Only pick up elements that are appearing for the first time during a recursion call to avoid duplicates.
Once an element is picked up, move to the next index.The recursion will terminate when the end of array is reached.While returning backtrack by removing the last element that was inserted.

```
public static void findSubsets(int ind, int[] nums, List<Integer> ds,
List<List<Integer>> ansList) {
        ansList.add(new ArrayList<>(ds));
        for(int i = ind; i < nums.length; i++) {
            if(i!=ind && nums[i] == nums[i-1]) continue;
            ds.add(nums[i]);
            findSubsets(i+1, nums, ds, ansList);
            ds.remove(ds.size() - 1);
        }
    }
    public static List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> ansList = new ArrayList<>();
        findSubsets(0, nums, new ArrayList<>(), ansList);
        return ansList;
    }
```

METHOD - 4 USING BIT MASK

```
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);
        int N = nums.length;
        List<List<Integer>> list = new ArrayList();
        for (int i = 0; i < (1 << N); i++){
            List<Integer> lis = getList(i, nums);
            if (!list.contains(lis))
                list.add(lis);
        }
        return list;
    }
    private List<Integer> getList(int mask, int[] nums){
        List<Integer> list = new ArrayList();
        for (int i = 0; i < nums.length; i++){
            int m = ((mask >> i) & 1);
            if (m == 1)
                list.add(nums[i]);
        }
```

```
        return list;}}
```

3. Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top.Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1: `Input: n = 2 Output: 2`
`Explanation: There are two ways to climb to the top. 1. 1 step + 1 step 2. 2 steps`
Example 2: `Input: n = 3 Output: 3`
`Explanation: There are three ways to climb to the top.`
`1. 1 step + 1 step + 1 step`
`2. 1 step + 2 steps`
`3. 2 steps + 1 step`

Method - 1 Recursion

```
class Solution {
    public int climbStairs(int n) {
        if (n < 3) return n;
        return climbStairs(n - 1) + climbStairs( n - 2);
    }
}
```

Method - 2 dp

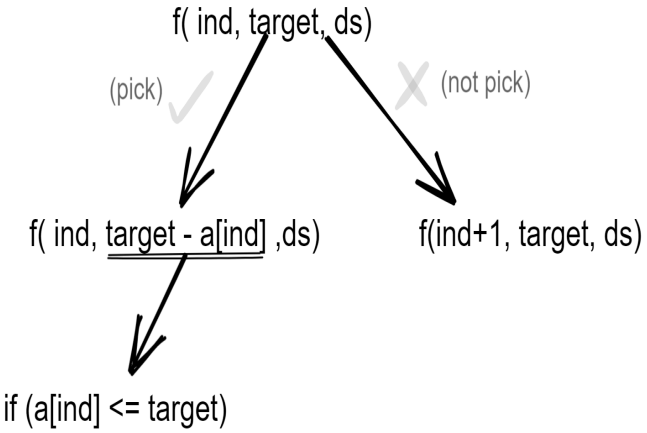
```
class Solution {
    int[] dp = new int[46];
    public int climbStairs(int n) {
        if (n == 2 || n == 1 || n == 0) return n;
        if (dp[n] != 0) return dp[n];
        dp[n] = climbStairs(n - 1) + climbStairs(n - 2);
        return dp[n];
    }}
```

4. COMBINATION SUM

Problem Statement: Given an array of distinct integers and a target, you have to return *the list of all unique combinations where the chosen numbers sum to* target. You may return the combinations in any order.
The same number may be chosen from the given array an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.
It is guaranteed that the number of unique combinations that sum up to target is less than 150 combinations for the given input.
Example 1:
Input: array = [2,3,6,7], target = 7 **Output**: [[2,2,3],[7]]
Explanation: 2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7. These are the only two combinations.
Example 2:
Input: array = [2], target = 1 **Output**: []
Method - 1 Recursion

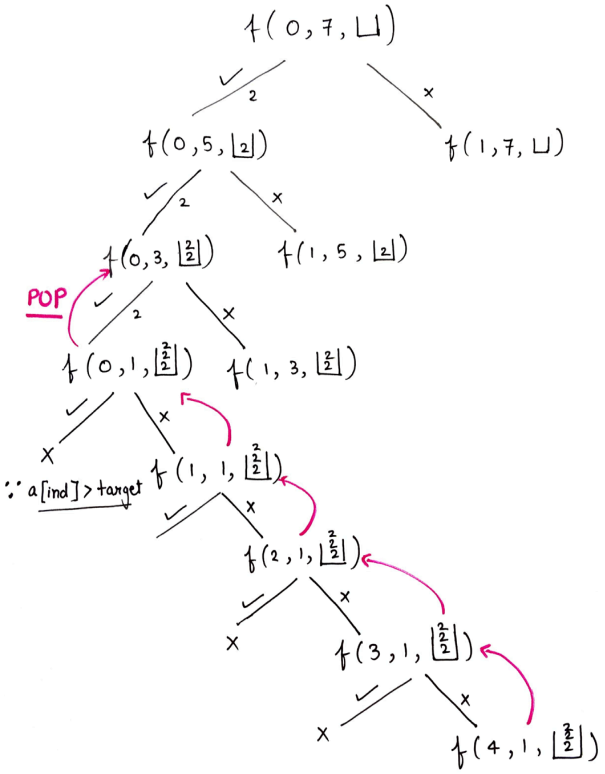
Intuition: For questions like printing combinations or subsequences, the first thing that should strike your mind is recursion.
How to think recursively? Whenever the problem is related to picking up elements from an array to form a combination, start thinking about the "pick and non-pick" approach.

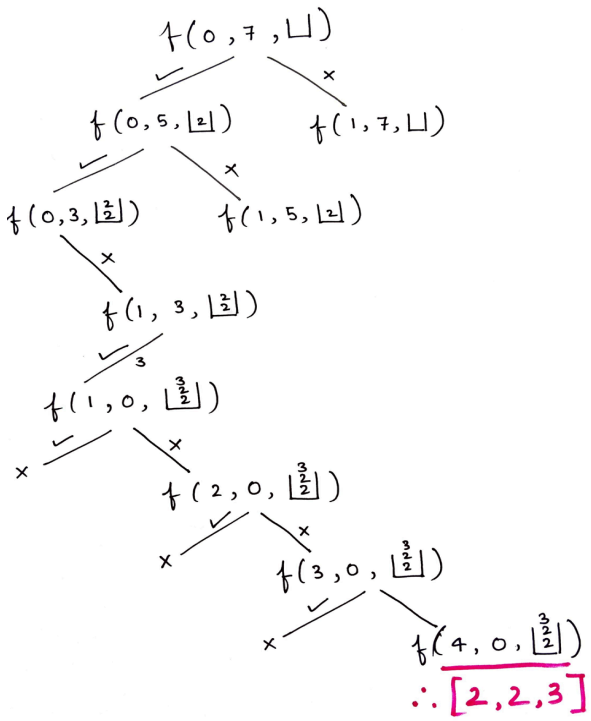
Approach: Defining recursive function:



Initially, the index will be 0, target as given and the data structure(vector or list) will be empty
Now there are 2 options viz to pick or not pick the current index element.
If you pick the element, again come back at the same index as multiple occurrences of the same element is possible so the target reduces to target – arr[index] (where target -arr[index]>=0)and also insert the current element into the data structure. If you decide not to pick the current element, move on to the next index and the target value stays as it is. Also, the current element is not inserted into the data structure.
While backtracking makes sure to pop the last element as shown in the recursion tree below.
Keep on repeating this process while index < size of the array for a particular recursion call.
You can also stop the recursion when the target value is 0, but here a generalized version without adding too many conditions is considered.
Using this approach, we can get all the combinations.
Base condition If index== size of array and target == 0 include the combination in our answer

Diagrammatic representation for Example 1:
Case 1:





Case 2:

```
class Solution {
    private void findCombinations(int ind, int[] arr, int target, List < List <
Integer >> ans, List < Integer > ds) {
        if (ind == arr.length) {
            if (target == 0) {
                ans.add(new ArrayList < > (ds));
            }
            return;
        }

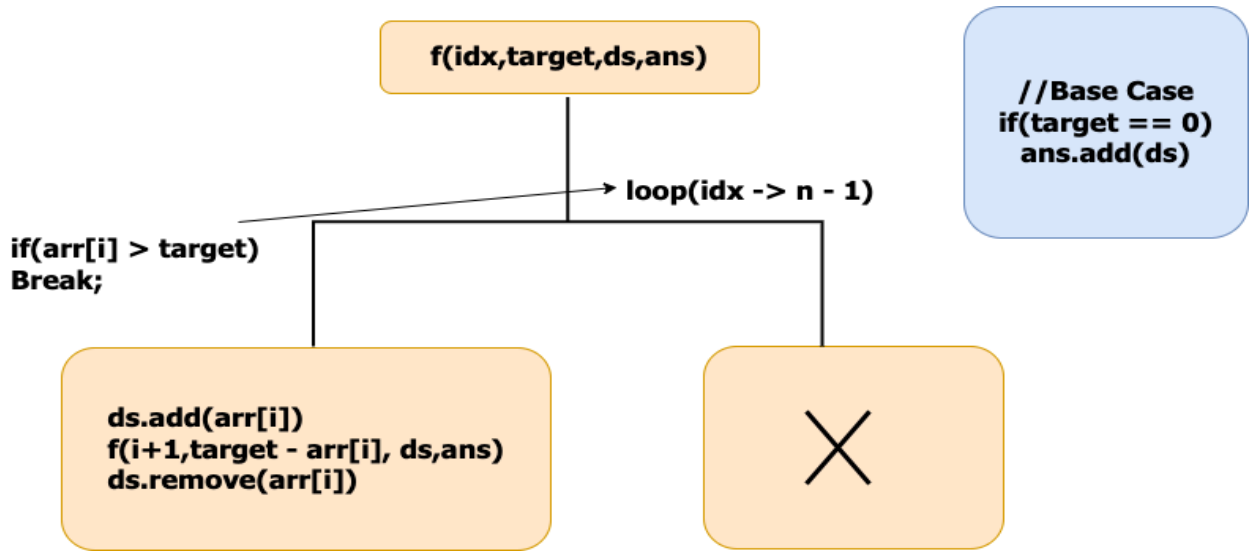
        if (arr[ind] <= target) {
            ds.add(arr[ind]);
            findCombinations(ind, arr, target - arr[ind], ans, ds);
            ds.remove(ds.size() - 1);
        }
        findCombinations(ind + 1, arr, target, ans, ds);
    }
    public List < List < Integer >> combinationSum(int[] candidates, int target) {
        List < List < Integer >> ans = new ArrayList < > ();
        findCombinations(0, candidates, target, ans, new ArrayList < > ());
        return ans;
    }
}
```

Time Complexity: O(2^t * k) where t is the target, k is the average length
Reason: Assume if you were not allowed to pick a single element multiple times, every element will have a couple of options: pick or not pick which is 2^n different recursion calls, also assuming that the average length of every combination generated is k. (to put length k data structure into another data structure)
Why not (2^n) but (2^t) (where n is the size of an array)?
Assume that there is 1 and the target you want to reach is 10 so 10 times you can "pick or not pick" an element.
Space Complexity: O(k*x), k is the average length and x is the no. of combinations

VARIATIONS IN PROBLEM

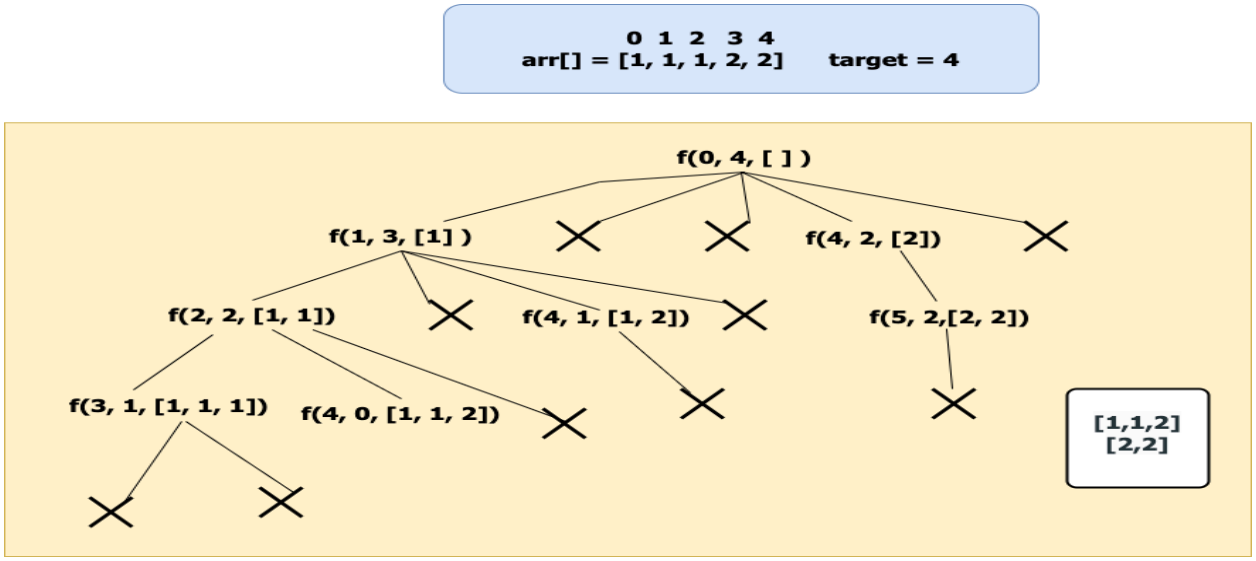
In this article we will solve the most asked interview question "Combination Sum II – Find all unique combinations".

Problem Statement: Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination.Note: The solution set must not contain duplicate combinations.
Example 1: Input: candidates = [10,1,2,7,6,1,5], target = 8
Output: [[1,1,6], [1,2,5], [1,7], [2,6]]
Explanation: These are the unique combinations whose sum is equal to target.
Example 2: Input: candidates = [2,5,2,1,2], target = 5 **Output**: [[1,2,2],[5]]
Explanation: These are the unique combinations whose sum is equal to target.
Solution 1: Using extra space and time complexity
Approach: Defining the Recursive Function:



Before starting the recursive call make sure to sort the elements because the ans should contain the combinations in sorted order and should not be repeated.
Initially, We start with the index 0, At index 0 we have n – 1 way to pick the first element of our subsequence. Check if the current index value can be added to our ds. If yes add it to the ds and move the index by 1. while moving the index skip the consecutive repeated elements because they will form duplicate sequences.

Reduce the target by arr[i],call the recursive call for f(idx + 1,target – 1,ds,ans) after the call make sure to pop the element from the ds.(By seeing the example recursive You will understand).
if(arr[i] > target) then terminate the recursive call because there is no use to check as the array is sorted in the next recursive call the index will be moving by 1 all the elements to its right will be in increasing order.**Base Condition:** Whenever the target value is zero add the ds to the ans return.Representation of Recursive call for the example given below:



If we observe the recursive call for f(2,2,[1,1]) when it is returning the ds doesn't include 1 so make sure to remove it from ds after the call.

```
class TUF {
    static void findCombinations(int ind, int[] arr, int target, List < List <
Integer >> ans, List < Integer > ds) {
        if (target == 0) {
            ans.add(new ArrayList < > (ds));
            return;
        }
        for (int i = ind; i < arr.length; i++) {
            if (i > ind && arr[i] == arr[i - 1]) continue;
            if (arr[i] > target) break;
            ds.add(arr[i]);
            findCombinations(i + 1, arr, target - arr[i], ans, ds);
            ds.remove(ds.size() - 1);
        }
    }
    public static List < List < Integer >> combinationSum2(int[] candidates, int
target) {
        List < List < Integer >> ans = new ArrayList < > ();
        Arrays.sort(candidates);
        findCombinations(0, candidates, target, ans, new ArrayList < > ());
        return ans;
    }
```

Variation in problem - II

`Combination Sum IV` - Given an array of distinct integers `nums` and a target integer `target`, return *the number of possible combinations that add up to* `target`. The test cases are generated so that the answer can fit in a 32-bit integer.

Example 1: `Input: nums = [1,2,3], target = 4 Output: 7`
`Explanation: The possible combination ways are:`
`(1, 1, 1, 1) (1, 1, 2) (1, 2, 1) (1, 3) (2, 1, 1) (2, 2) (3, 1)`
`Note that different sequences are counted as different combinations.`
Example 2: `Input: nums = [9], target = 3 Output: 0`

There is a mis**understanding** that people think it is not a `combination` problem, but rather a problem about `permutations`.
I'd say, this problem is about neither of those.
There are 3 major combinatorial objects/functions:

1. Combination: `C(n, k)` 2. Permutation: `P(n)` 3. Allocation: `A(n, k)`

Each one of them has a corresponding tree that represents all of the objects that are distinguished from each other.
We could retrieve each particular object by traversing a tree going from the root to a leaf.
I don't know whether it would be helpful for anyone, but I gained understanding by pondering at an image of an Allocation Tree for the sample input: `nums = [1,2,3], target = 4`.
Each node represents the sum of the edges on a path from the particular node that we're looking at to a root node.
Each edge represents a choice that we made to take a number from `nums[]` array. Because of that each node has `len(nums)` children.



Here is a better quality image: https://imgur.com/1c4wOhH
Or you could insert the code from here https://disk.yandex.ru/d/vRVVTVreaxJLHQ to here http://www.webgraphviz.com/ and see the graph in its full glory.
The depth of the tree is determined by the smallest number in the `num[]` array. On each level of a tree the sums increase from left to right.
The leftmost leaf contains the smallest sum that we could make by choosing 4 numbers `(nums[0], nums[0], nums[0], nums[0])`.
The rightmost leaf contains the largest sum that we could make by choosing 4 numbers `(nums[2], nums[2], nums[2], nums[2])`.
Our tree shows us all of the possible sums that we could make by choosing 4 numbers or less from array `nums[]`.

If we have to choose exactly 4 numbers from the nums[] array, then we have to look at the leafs of the tree as they represent all of the possible sums that could be attained by choosing exactly 4 numbers. We could notice that there is only one leaf with sum 4 (the leftmost leaf).
If we have to choose exactly 3 numbers from the nums[] array. Then we have to look at the 3'rd level of the tree (the root is at level 0).
The 3'rd level of the tree contains 3 nodes with sum 4 and it shows the paths that lead us to these sums: (1, 1, 2), (1, 2, 1), (2, 1, 1). All of the other nodes on 3'rd level have a sum which is larger than 4 (except for node (1, 1, 1)).
Each new level in this tree grows by a factor of 3 (by a factor of len(nums) to be precise).
That is, the number of nodes on level i is 3 x the number of nodes on level i - 1, or mathematically: num_nodes(level) = 3 * num_nodes(level - 1). If we expand this recursion we see num_nodes(level) = 3 * num_nodes(level - 1) = 3 * 3 * num_nodes(level - 2) = 3 * 3 * 3 * num_nodes(level - 3) = 3 ^ {tree height}.
That is the fastest growing tree among all combinatorial objects: combinations, permutations, allocations. Permutation tree grows slower than allocation tree, because with each new level of the tree the number of child nodes is chopped off by 1, so instead of the following progression: 3 * 3 * 3 we get 3 * 2 * 1.
Here is
Intuition for dynamic programming solution comes from the following drawing:



There are 4 ways to come to node with sum 4 from parent node with sum 3:

1. (1 + 1 + 1) + 1 = 3 + 1 = 4.
2. (1 + 2) + 1 = 3 + 1 = 4.
3. (2 + 1) + 1 = 3 + 1 = 4.
4. (3) + 1 = 3 + 1 = 4.

There are 2 ways to come to node with sum 4 from parent node with sum 2:

1. (1 + 1) + 2 = 2 + 2 = 4.
2. (2) + 2 = 2 + 2 = 4.

There is 1 way to come to node with sum 4 from parent node with sum 1:

1. (1) + 3 = 1 + 3 = 4.

So, if we have already stored in cnt[] array the number of nodes with sum 1 in cnt[1] = 1, the number of nodes with sum 2 in cnt[2] = 2, the number of nodes with sum 3 in cnt[3] = 4, then we can calculate the number of nodes with sum 4 by reusing cnt[1], cnt[2], cnt[3]:

```
cnt[4] = cnt[1] + cnt[2] + cnt[3] = 1 + 2 + 4 = 7
```

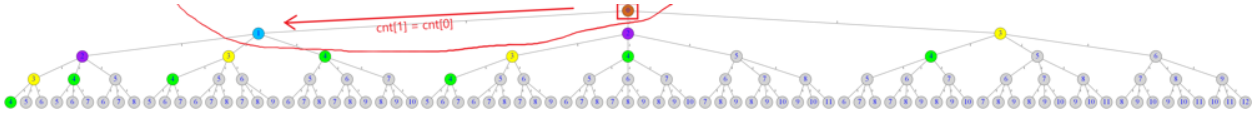
By the way, this picture also gives us an intuition why `cnt[0] = 1` and it is not `cnt[0] = 0`. The reason is that we have a single node with sum `0` - the root of the tree.

Now let's look at how dynamic programming computation looks like on a tree from one level to the next.

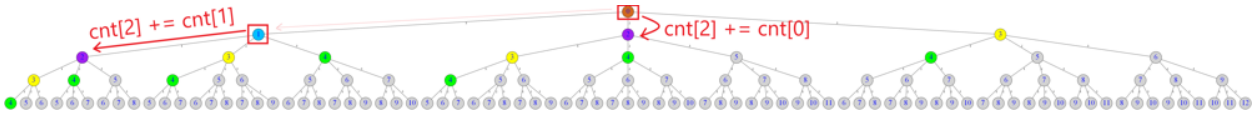
The node which is already computed will be surrounded by a red square.

At the start `cnt[0]` is already computed and is equal to `1`, so it is surrounded by the red square from the very beginning.

The next stage is to compute `cnt[1]` from `cnt[0]`. There is only one node with `cnt[0]`, and it is the only parent node which could be accessed from `cnt[1]`, so we essentially copy `cnt[1] = cnt[0] = 1`:



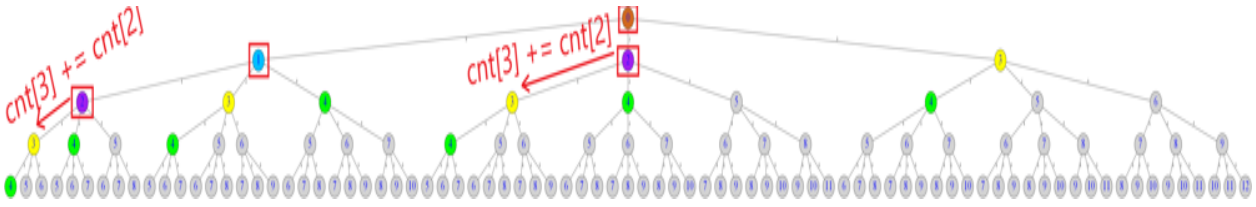
After we have processed `cnt[0]` and `cnt[1]` now we have all the necessary ingredients to compute `cnt[2]`.



Computation of `cnt[2]` is more interesting, because we need values from 2 different nodes to compute it:

```
cnt[2] = 0;
cnt[2] += cnt[1]; // The number of nodes with sum `1`
cnt[2] += cnt[0]; // The number of nodes with sum `0`
```

Now the real secret sauce is revealed when we look at the computation of `cnt[3]`. We continue reusing previous `cnt[0], cnt[1], cnt[2]` for computation of `cnt[3]`, but this time we will notice something special.



This image doesn't contain all the arrows to show the computation, because the image becomes messy. It only shows the most essential part needed for understanding: `cnt[3] += cnt[2];`.

The whole computation of cnt[3] looks like this:

```
cnt[3] = 0;
cnt[3] += cnt[0];  // Just adds `1`.
cnt[3] += cnt[1];  // Also simply adds `1`.
cnt[3] += cnt[2]; // Adds `2`! This is key to understanding how and why dynamic programming works!
```

As we can see from an image when we add cnt[3] += cnt[2] we do something special. There are two different nodes with sum 2 and we are adding both of these nodes simultaneously with a single operation! That is two different combinations (allocations!) that give sum 2: (1, 1) and (2) are already stored in cnt[2]. By doing the operation cnt[3] += cnt[2] we can imagine like we extend both of these two combinations: {(1, 1), (2)} by adding 1 to the end of each of them. So cnt[3] += cnt[2] is like the transformation of {(1, 1), (2)} to {(1, 1, 1), (2, 1)}!!!!

So we can look at computation of cnt[3] = cnt[0] + cnt[1] + cnt[2] like extention of paths by adding 1 in the end:

```
cnt[3] = 0;
cnt[3] += cnt[0];  // Single path is extended by appending `3` in the end: from (0) to (0, 3)
cnt[3] += cnt[1];  // Single path is extended by appending `2` in the end: from (0, 1) to (0, 1, 2).
cnt[3] += cnt[2]; // Two paths are extended by appending `1` in the end: from (0, 1, 1) to (0, 1, 1, 1) and from (0, 2) to (0, 2, 1)
```

Probably you have noticed that I have used 0 in the beginning of all the paths (0), (0, 1), (0, 1, 1), (0, 1, 1, 1). This way we can really see that we form each path by traversing from root node (which is 0) and it's easier to see how we transform these paths and compare them with each other:

(0) + (3) ⟶ (0, 3) // cnt[3 - 3] (0, 1) + (2) ⟶ (0, 1, 2) // cnt[3 - 2]

(0, 1, 1) + (1) ⟶ (0, 1, 1, 1) // cnt[3 - 1] (0, 2) + (1) ⟶ (0, 2, 1) // cnt[3 - 1]

Coin change vs this problem

First thing we need to understand is what is permutation and combination ?

In permutations - (1,1,2) ,(1,2,1),(2,,1,1) are treated as different and counted, whereas in combinations all these are considered same. The question is combination sum 4 but we need to print the permutation count

Now coming to the coin change 2 and this problem, In coin change 2 we are using combinations and combination sum 4 we are using permutations coin change 2 solution

Brute force soln.

```
class Solution {
    public int combinationSum4(int[] nums, int target) {
        return solve(nums, target);
    }
    private int solve(int[] nums, int target){
        if (0 > target){  return 0;  }
        if (target == 0){ return 1; }
        int cnt = 0;
        for (int n: nums){
```

```
            cnt += solve(nums, target - n);
        }
        return cnt;
    }}
```

Dp solution

```
class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1];
        Arrays.fill(dp, -1);
        return solve(nums, target, dp);
    }
    private int solve(int[] nums, int target, int[] dp){
        if (target < 0){  return 0;  }
        if (target == 0){   return 1;  }
        if (dp[target] != -1){  return dp[target];  }
        int cnt = 0;
        for (int n: nums){
            cnt += solve(nums, target - n, dp);
        }
        dp[target] = cnt;
        return cnt;
    }}
```

Top down approach

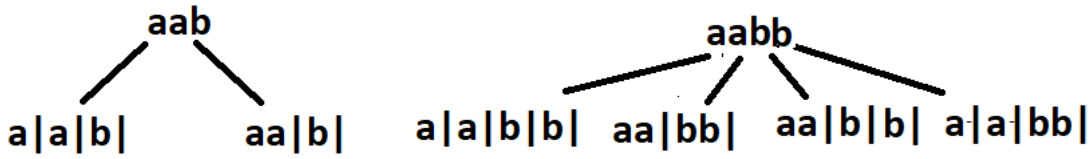
```
class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1];
        dp[0] = 1;
        for (int i = 1; i < dp.length; i++){
            for (int n: nums){
                if (i - n >= 0){
                   dp[i] += dp[i - n];
                }
            }
        }
        return dp[target];
    }}
```

5. PALINDROME PARTITION

Problem Statement: You are given a string s, partition it in such a way that every substring is a palindrome. Return all such palindromic partitions of s.

Example 1:

Input: s = “aab” **Output**: [["a","a","b"], ["aa","b"]] **Explanation**: The first answer is generated by making three partitions. The second answer is generated by making two partitions.

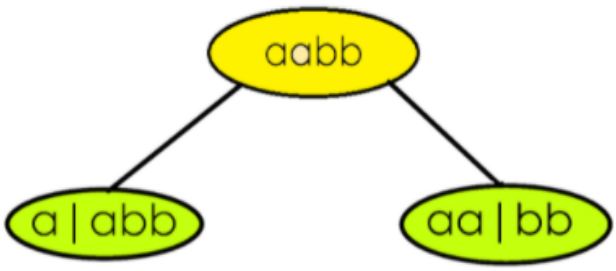


Example 2:

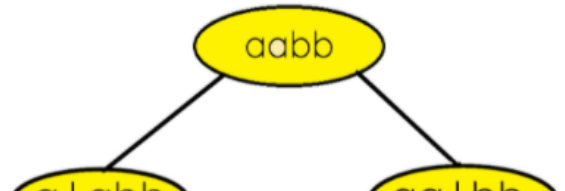
Input: s = “aabb”

Output: [[“a”,”a”,”b”,”b”], [“aa”,”bb”], [“a”,”a”,”bb”], [“aa”,”b”,”b”]] **Explanation**: See Figure

Approach: The initial idea will be to make partitions to generate substring and check if the substring generated out of the partition will be a palindrome. Partitioning means we would end up generating every substring and checking for palindrome at every step. Since this is a repetitive task being done again and again, at this point we should think of recursion. The recursion continues until the entire string is exhausted. After partitioning, every palindromic substring is inserted in a data structure When the base case has reached the list of palindromes generated during that recursion call is inserted in a vector of vectors/list of list.
We have already discussed the initial thought process and the basic outline of the solution. The approach will get clearer with an example.
Say s = “aabb” and assume indexes of string characters to be 0-based. For a better understanding, we have divided recursion into some steps.
STEP 1: We consider substrings starting from the 0th index.[0,0] is a palindrome, so partition right after the 0th index.[0,1] is another palindrome, make a partition after 1st index. Beyond this point, other substrings starting from index 0 are “aab” and “aabb”. These are not palindromes, hence no more. partitions are possible. The strings remaining on the right side of the partition are used as input to make recursive calls.

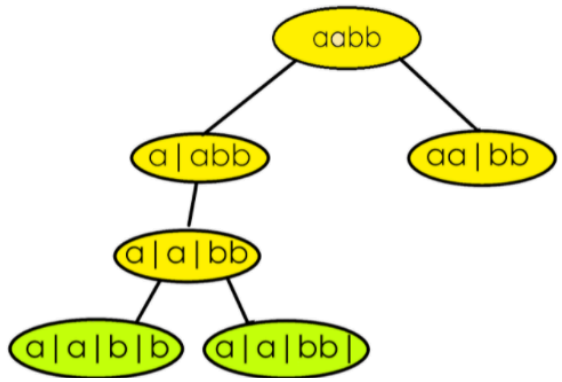


STEP 2: Consider the recursive call on the left(*refer to image)* where “abb” is the input.[1,1] is a palindrome, make a partition after it.[1,2] and [1,3] are not palindromes.



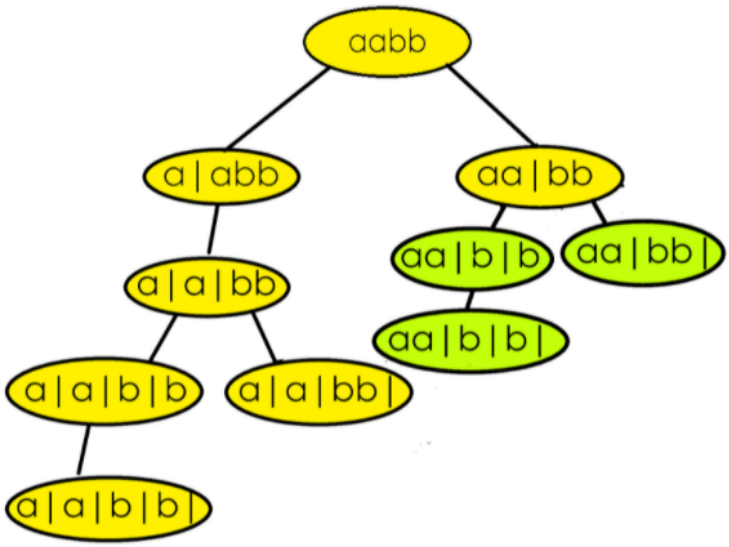
STEP 3: Here “bb” is the input.[2,2] as well as [2,3] are palindromes. Make one partition after the 2nd index and one after the 3rd index The entire string is exhausted after the 3rd index, so the right recursion ends here. Palindromes generated from the right recursion are inserted in our answer.

Our answer at this point :[[“a”,” a”,” bb”]]



The left recursion will continue with “b” as its input.[3,3] is a palindrome so one last partition for the left recursion is made after the 3rd index. Insert the palindromes.

ans = [[“a”,”a”,”bb”], [“a”,”a”,”b”,”b”]]



STEP 4: After the list of palindromic substrings are returned from the left recursive call, continue the same process for the call on the right that was left to recur. The right recursion is having “bb” as input, something we have already encountered in step 3. Hence we will repeat the same task which was done in step 3 onwards.

Final answer : [[“a”,”a”,”bb”], [“a”,”a”,”b”,”b”] ,[“aa”,”b”,”b”], [“aa”,”bb”]]

```
class TUF {
    public static List < List < String >> partition(String s) {
        List < List < String >> res = new ArrayList < > ();
        List < String > path = new ArrayList < > ();
        func(0, s, path, res);
        return res;
    }

    static void func(int index, String s, List < String > path, List < List < String
>> res) {
        if (index == s.length()) {
            res.add(new ArrayList < > (path));
            return;
        }
```

```
        for (int i = index; i < s.length(); ++i) {
            if (isPalindrome(s, index, i)) {
                path.add(s.substring(index, i + 1));
                func(i + 1, s, path, res);
                path.remove(path.size() - 1);
            }
        }
    }

    static boolean isPalindrome(String s, int start, int end) {
        while (start <= end) {
            if (s.charAt(start++) != s.charAt(end--))
                return false;
        }
        return true;
    }
```

6. KTH PERMUTATIONS

Given N and K, where N is the sequence of numbers from 1 to N([1,2,3….. N]) find the Kth permutation sequence.
For N = 3 the 3! Permutation sequences in order would look like this:-
Note: 1<=K<=N! Hence for a given input its Kth permutation always exists

K = 1	"123"
K = 2	"132"
K = 3	"213"
K = 4	"231"
K = 5	"312"
K = 6	"321"

Example 1:

Input: N = 3, K = 3 **Output**: "213"

Explanation: The sequence has 3! permutations as illustrated in the figure above. K = 3 corresponds to the third sequence.

Example 2:

Input: N = 3, K = 5 **Result**: "312"

Explanation: The sequence has 3! permutations as illustrated in the figure above. K = 5 corresponds to the fifth sequence.

Solution 1: Brute Force Solution

Approach: The extreme naive solution is to generate all the possible permutations of the given sequence. This is achieved using recursion and every permutation generated is stored in some other data structure (here we have used a vector). Finally, we sort the data structure in which we have stored all the sequences and return the Kth sequence from it.

```
public class Main {
    static void swap(char s[], int i, int j) {
        char ch = s[i];
        s[i] = s[j];
        s[j] = ch;
    }
    static void solve(char s[], int index, ArrayList < String > res) {
```

```
        if (index == s.length) {
            String str = new String(s);
            res.add(str);
            return;
        }
        for (int i = index; i < s.length; i++) {
            swap(s, i, index);
            solve(s, index + 1, res);
            swap(s, i, index);
        }
    }
    static String getPermutation(int n, int k) {
        String s = "";
        ArrayList < String > res = new ArrayList < > ();
        for (int i = 1; i <= n; i++) {
            s += i;
        }
        solve(s.toCharArray(), 0, res);
        Collections.sort(res);
        return res.get(k); }}
```

Solution 2: backtracking

```
class Solution {
    List<List<Integer>> list = new ArrayList();
    public String getPermutation(int n, int k) {
        int[] arr = new int[n];
        for (int i = 1; i <= n; i++){  arr[i - 1] = i; }
        backtrack(arr, new ArrayList());
        StringBuilder ans = new StringBuilder();
        for (int N: list.get(k - 1)){
            ans.append(Integer.toString(N));
        }
        return ans.toString();
    }
    private void backtrack(int[] arr, List<Integer> lis){
        if (lis.size() == arr.length){
            list.add(new ArrayList(lis));
            return ;
        }
        for (int i = 0; i < arr.length; i++){
            if (lis.contains(arr[i])) continue;
            lis.add(arr[i]);
            backtrack(arr, lis);
            lis.remove(lis.size() - 1);
        }
   }}
```

Solution 3:(Optimal Approach)

Say we have N = 4 and K = 17. Hence the number sequence is {1,2,3,4}.

Intuition: Since this is a permutation we can assume that there are four positions that need to be filled using the four numbers of the sequence. First, we need to decide which number is to be placed at the first index. Once the number at the first index is decided we have three more positions and three more numbers. Now the problem is shorter. We can repeat the technique that was used previously until all the positions are filled. The technique is explained below.

Approach:

STEP 1: Mathematically speaking there can be 4 variations while generating the permutation. We can have our permutation starting with either 1 or 2 or 3 or 4. If the first position is already occupied by one number there are three more positions left. The remaining three numbers can be permuted among themselves while filling the 3 positions and will generate 3! = 6 sequences. Hence each block will have 6 permutations adding up to a total of 6*4 = 24 permutations. If we consider the sequences as 0-based and in the sorted form we observe:-

- The **0th – 5th permutation** will start with 1
- The **6th – 11th permutation** will start with 2
- The **12th – 17th permutation** will start with 3
- The **18th – 23rd permutation** will start with 4.

(For better understanding refer to the picture below.)

BLOCK NO.	NUMBER AT 1ST POSITION	REMAINING NUMBERS
0TH BLOCK	1	{2,3,4}
1ST BLOCK	2	{1,3,4}
2ND BLOCK	3	{1,2,4}
3RD BLOCK	4	{1,2,3}

We make **K = 17-1 considering 0-based indexing**. Since each of the four blocks illustrated above comprises 6 permutations, therefore, the 16th permutation will lie in (16 / 6) = 2nd block, and our answer is the (16 % 6) = 4th sequence from the 2nd block. Therefore 3 occupies the first position of the sequence and **K = 4.**

STEP 2: Our new search space comprises three elements {1,2,4} where K = 4 . Using the previous technique we can consider the second position to be occupied can be any one of these 3 numbers. Again one block can start with 1, another can start with 2 and the last one can start with 4 . Since one position is fixed, the remaining two numbers of each block can form **2! = 2 sequences.** In sorted order :

- The **0th – 1st sequence** starts with 1
- The **2nd – 3rd sequence** starts with 2
- The **4th – 5th sequence** starts with 4

BLOCK NO.	NUMBER AT 1ST POSITION	REMAINING NUMBERS
0TH BLOCK	1	{2,4}
1ST BLOCK	2	{1,4}
2ND BLOCK	4	{1,2}

3 ___ ___ ___

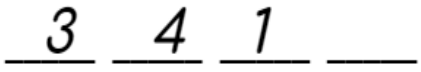
The 4th permutation will lie in **(4/2) = 2nd block** and our answer is the **4%2 = 0th sequence from the 2nd block**. Therefore **4** occupies the second position and **K = 0.**

STEP 3: The new search space will have two elements **{1 ,2}** and **K = 0**. One block starts with 1 and the other block starts with 2. The other remaining number can form only one **1! = 1 sequence**. In sorted form

- The **0th sequence** starts with 1
- The **1st sequence**. starts with 2

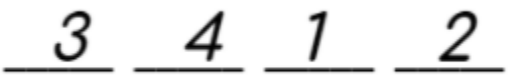
3 4 ___ ___

BLOCK NO.	NUMBER AT 1ST POSITION	REMAINING NUMBERS
0TH BLOCK	1	{2}
1ST BLOCK	2	{1}



The 0th permutation will lie in the **(0/1) = 0th block** and our answer is the 0%1 = **0th sequence from the 0th block**. Therefore **1** occupies the 3rd position and **K = 0.**

BLOCK NO.	NUMBER AT 1ST POSITION	REMAINING NUMBERS
0Th BLOCK	2	{}



STEP 4: Now the only block has 2 in the first position and no **remaining number is present**. This is the point where we place 2 in the last position and stop.
The final answer is 3412.

```
public class Main {
    static String getPermutation(int n, int k) {
        int[] factorial = new int[n];
        List<Integer> list = new ArrayList();
        factorial[0] = 1;
        for (int i = 1; i <= n; i++){
            list.add(i);
            if (i < n)factorial[i] = i*factorial[i - 1];
        }

        StringBuilder sb = new StringBuilder();
        k--; // w.r.t 0th index
        for (int i = n - 1; i >= 0; i--){
            int ind = k/factorial[i];
            sb.append(list.get(ind));
            list.remove(ind);
            k = k % factorial[i];
        }
        return sb.toString();}
```

7. PRINT ALL PERMUTATIONS OF STRING

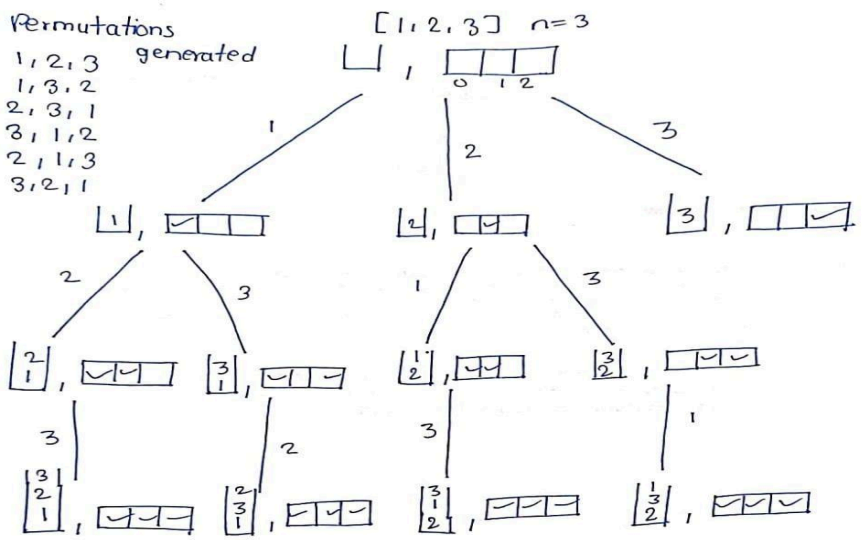
Given an array of distinct integers, print all permutations of String/Array.

Example 1: Input: arr = [1, 2, 3]
Output: [[1, 2, 3],[1, 3, 2],[2, 1, 3],[2, 3, 1],[3, 1, 2],[3, 2, 1]]
Example 2: Input: arr = [0, 1] Output: [[0, 1], [1, 0]]

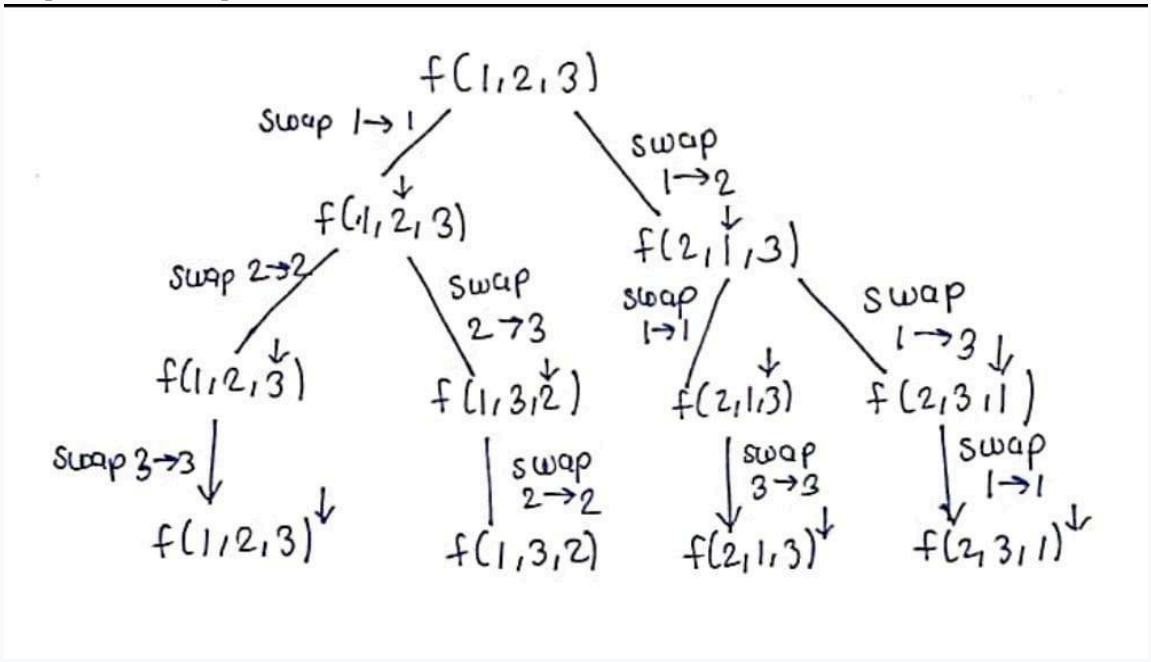
Solution 1: Recursive

Approach: We have given the nums array, so we will declare an ans vector of vector that will store all the permutations and also declare a data structure.
Declare a map and initialize it to zero and call the recursive function
Base condition: When the data structure's size is equal to n(size of nums array) then it is a permutation and stores that permutation in our ans, then returns it.
Recursion: Run a for loop starting from 0 to nums.size() – 1. Check if the frequency of i is unmarked, if it is unmarked then it means it has not been picked and then we pick. And make sure it is marked as picked.
Call the recursion with the parameters to pick the other elements when we come back from the recursion make sure you throw that element out. And unmark that element in the map.
Recursive Tree:

```
class Solution {
    private void recurPermute(int index, int[] nums, List < List < Integer >> ans) {
        if (index == nums.length) {
            // copy the ds to ans
            List < Integer > ds = new ArrayList < > ();
            for (int i = 0; i < nums.length; i++) {
                ds.add(nums[i]);
            }
            ans.add(new ArrayList < > (ds));
            return;
        }
        for (int i = index; i < nums.length; i++) {
            swap(i, index, nums);
            recurPermute(index + 1, nums, ans);
            swap(i, index, nums);
        }
    }
    private void swap(int i, int j, int[] nums) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
    public List < List < Integer >> permute(int[] nums) {
        List < List < Integer >> ans = new ArrayList < > ();
        recurPermute(0, nums, ans);
        return ans;
    }
};
```



Solution 2: With Backtracking.
Approach: Using backtracking to solve this.
We have given the nums array, so we will declare an ans vector of vector that will store all the permutations.Call a recursive function that starts with zero, nums array, and ans vector.
Declare a map and initialize it to zero and call the recursive function
Base condition: Whenever the index reaches the end take the nums array and put it in ans vector and return.
Recursion: Go from index to n – 1 and swap. Once the swap has been done call recursion for the next state.After coming back from the recursion make sure you re-swap it because for the next element the swap will not take place.




```
class Solution {
    private void recurPermute(int index, int[] nums, List < List < Integer >> ans) {
```

```
        if (index == nums.length) {
            // copy the ds to ans
            List < Integer > ds = new ArrayList < > ();
            for (int i = 0; i < nums.length; i++) {
                ds.add(nums[i]);
            }
            ans.add(new ArrayList < > (ds));
            return;
        }
        for (int i = index; i < nums.length; i++) {
            swap(i, index, nums);
            recurPermute(index + 1, nums, ans);
            swap(i, index, nums);
        }
    }
    private void swap(int i, int j, int[] nums) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
    public List < List < Integer >> permute(int[] nums) {
        List < List < Integer >> ans = new ArrayList < > ();
        recurPermute(0, nums, ans);
        return ans;
    }
};
```

8. M COLORING PROBLEM

Problem Statement: Given an undirected graph and a number m, determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

Example 1:

Input: N = 4 M = 3 E = 5 Edges[] = { (0, 1), (1, 2), (2, 3), (3, 0), (0, 2) } **Output:** 1

Explanation: It is possible to color the given graph using 3 colours.

Example 2:Input: N = 3 M = 2 E = 3 Edges[] ={(0, 1), (1, 2), (0, 2) }

Output: 0 **Explanation:** It is not possible to color.

Solution 1: Backtracking

Approach: Basically starting from vertex 0 color one by one the different vertices.

Base condition: If I have colored all the N nodes return true.

Recursion: Trying every color from 1 to m with the help of a for a loop.

Is safe function returns true if it is possible to color it with that color i.e none of the adjacent nodes have the same color.

In case this is an algorithm and follows a certain intuition, please mention it.

Color it with color i then call the recursive function for the next node if it returns true we will return true.

And If it is false then take off the color.

Now if we have tried out every color from 1 to m and it was not possible to color it with any of the m colors then return false.

```
    public static boolean graphColoring(List < Integer > [] G, int[] color, int i,
int C) {
        // Your code here
        int n = G.length;
        if (solve(i, G, color, n, C) == true) return true;
        return false;
    }
    private static boolean isSafe(int node, List<Integer>[] G, int[] color,
                                  int n, int col) {
        for (int it: G[node]) {
            if (color[it] == col) return false;
        }
        return true;}
    private static boolean solve(int node, List<Integer>[] G,
                           int[] color, int n, int m) {
        if (node == n) return true;
        for (int i = 1; i <= m; i++) {
            if (isSafe(node, G, color, n, i)) {
                color[node] = i;
                if (solve(node + 1, G, color, n, m) == true) return true;
                color[node] = 0;
            }
        }
```

9. K-th Symbol in Grammar

We build a table of n rows (1-indexed). We start by writing 0 in the 1st row. Now in every subsequent row, we look at the previous row and replace each occurrence of 0 with 01, and each occurrence of 1 with 10. For example, for n = 3, the 1st row is 0, the 2nd row is 01, and the 3rd row is 0110. Given two integer n and k, return the kth (1-indexed) symbol in the nth row of a table of n rows.

Example 1: `Input: n = 1, k = 1 Output: 0 Explanation: row 1: 0`

Example 2: `Input: n = 2, k = 1 Output: 0`

`Explanation: row 1: 0 row 2: 01`

Optimized

```
class Solution {
    public int kthGrammar(int N, int K) {
       if (N == 1) return 0;
         if (K % 2 == 0) return (kthGrammar(N - 1, K / 2) == 0) ? 1 : 0;
         else return (kthGrammar(N - 1, (K + 1) / 2) == 0) ? 0 : 1;
    }}
```

```java
class Solution {
    public int kthGrammar(int n, int k) {
        String ans = get(n, 1, "0");

        return ans.charAt(k - 1) - '0';
    }
    private String get(int n, int st, String s){
        if (st == n){
            return s;
        }
        String ans = "";
        for (int i = 0; i < s.length(); i++){
            char cur = s.charAt(i);
            if (cur == '0'){
                ans += "01";
            }else if (cur == '1'){
                ans += "10";
            }
        }
       return get(n, st + 1, ans);
    }
}
```

10. Find the Winner of the Circular Game

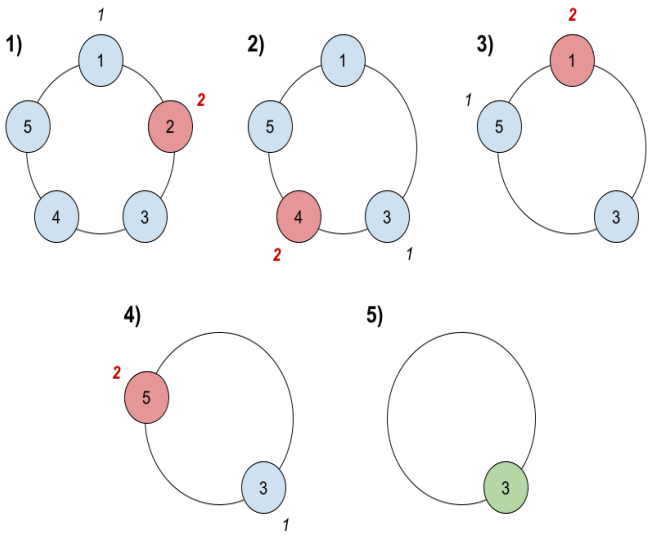
There are `n` friends that are playing a game. The friends are sitting in a circle and are numbered from `1` to `n` in clockwise order. More formally, moving clockwise from the `i`th friend brings you to the `(i+1)`th friend for `1 <= i < n`, and moving clockwise from the `n`th friend brings you to the `1`st friend.

The rules of the game are as follows:

1. Start at the `1`st friend.
2. Count the next `k` friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
3. The last friend you counted leaves the circle and loses the game.
4. If there is still more than one friend in the circle, go back to step `2` starting from the friend immediately clockwise of the friend who just lost and repeat.
5. Else, the last friend in the circle wins the game.

Given the number of friends, `n`, and an integer `k`, return *the winner of the game*.

Example 1:



Input: n = 5, k = 2 **Output:** 3
Explanation: Here are the steps of the game:
1) Start at friend 1.
2) Count 2 friends clockwise, which are friends 1 and 2.
3) Friend 2 leaves the circle. Next start is friend 3.
4) Count 2 friends clockwise, which are friends 3 and 4.
5) Friend 4 leaves the circle. Next start is friend 5.
6) Count 2 friends clockwise, which are friends 5 and 1.
7) Friend 1 leaves the circle. Next start is friend 3.
8) Count 2 friends clockwise, which are friends 3 and 5.
9) Friend 5 leaves the circle. Only friend 3 is left, so they are the winner.

Example 2: **Input:** n = 6, k = 5 **Output:** 1
Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.

Recursive solution

```
class Solution {
    public int findTheWinner(int n, int k) {
        return find(n, k) + 1;
    }
    private int find(int n, int k){
        if (n == 1) return 0;
        return (find(n - 1, k) + k) % n;
    }
}
```

The 3 Easy Approaches are : Circular Linked List, Recursive and Iterative.

1. Brute Force Solution : CIRCULAR LINKED LIST

/* Time Complexity : O(n*k) Space Complexity : O(n) */
If we look at the question, it says that the people are playing the game on a round table. That means, A Circular Linked List can be used to visualize the round table. We store 1,2,3,4,.....,n as the nodes of our List and go on deleting every kth node that we encounter. When we are left with only 1 node i.e. the head node, it is our WINNER. Since we are consuming extra space for Circular Linked List so, our Space Complexity goes to O(n).

```
class Solution {
public
    int helper(ListNode *head,int k){
        ListNode *temp=head;  //For traversing through the List and delete every kth
Node
        while(head->next!=head){   //Till the last Node remains in the list i.e. the
winner
            ListNode *pre=NULL;
            for(int i=1; i<k; i++){
                pre=temp;
                temp=temp->next;
            }
            //Deletion procedure
            if(temp==head)
                head=head->next;
            pre->next=temp->next;
            delete(temp);
            temp=pre->next;
        }
        //The last node remaining becomes the head of the Circular Linked List i.e.
Winner
        return head->val;
    }
    int findTheWinner(int n, int k) {
        if(k == 1)
            return n;
        //Designing Circular Linked List for Node 1,2,3,4,.....,n
        ListNode *head=NULL,*tail=NULL;
        for(int i=1; i<=n; i++){
            ListNode *node=new ListNode(i);
            if(head==NULL)
                head=tail=node;
            else{
                tail->next=node;
                tail=node;
            }
            tail->next=head;
        }
        //Helper function will eliminate the nodes and give us the winner
        return helper(head,k);
    }}
```

Iterative

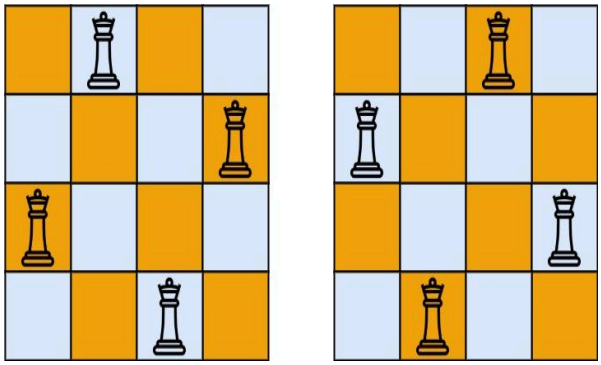
```
public int findTheWinner(int n, int k) {
        int res = 0;
        for (int i = 1; i <= n; ++i)
            res = (res + k) % i;
        return res + 1;
    }
```

II. BackTracking

1. Queens problem

The n-queens puzzle is the problem of placing `n` queens on an `n x n` chessboard such that no two queens attack each other. Given an integer `n`, return *all distinct solutions to the n-queens puzzle*. You may return the answer in any order. Each solution contains a distinct board configuration of the n-queens' placement, where `'Q'` and `'.'` both indicate a queen and an empty space, respectively.

Example 1:



Input: `n = 4`

Output: `[[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]`

`Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above`

Example 2:

`Input: n = 1 Output: [["Q"]]`

BACKTRACKING

```
class Solution {
    int n;
    List<List<String>> ans;
    public List<List<String>> solveNQueens(int n) {
        this.n = n;
```

```
        this.ans = new ArrayList();
        int[][] brd = new int[n][n];
        solve(brd, 0);
        return ans;
    }

    private void solve(int[][] brd, int row){
        if (row == n){
            List<String> list = convert(brd);
            ans.add(list);
            return ;
        }
        for (int c = 0; c < n; c++){
            if (isSafe(brd, row, c)){
                brd[row][c] = 1;
                solve(brd, row + 1);
                brd[row][c] = 0;
            }
        }
    }

    private boolean isSafe(int[][] brd, int r, int c){
        // CHECK R and C
        for (int i = 0; i < n; i++){
            if (brd[r][i] == 1){
                return false;
            }if (brd[i][c] == 1){
                return false;
            }
        }
        // check each diag.
        int i = r + 1, j = c - 1;
        while (i < n && j >= 0){
            if (brd[i][j] == 1){
                return false;
            }
            i++; j--;
        }
        i = r + 1; j = c + 1;
        while (i < n && j < n){
            if (brd[i][j] == 1){
                return false;
            }
            i++; j++;
        }
        i = r - 1; j = c + 1;
        while (i >= 0 && j < n){
            if (brd[i][j] == 1){
                return false;
            }
            i--; j++;
        }
        i = r - 1; j = c - 1;
        while (i >= 0 && j >= 0){
            if (brd[i][j] == 1){
```

```
                return false;
            }
            i--; j--;
        }
        return true;
    }
```

```
    private List<String> convert(int[][] brd){
        List<String> ans = new ArrayList();
        for (int i = 0; i < n; i++){
            String s = "";
            for (int j = 0; j < n; j++) s += (brd[i][j] == 0) ? "." : "Q";
            ans.add(s);
        }
        return ans;
    }}
```

2. Sudoku Solver

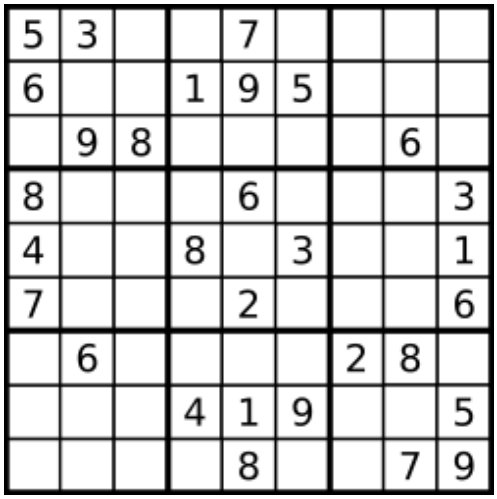
Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

1. Each of the digits `1-9` must occur exactly once in each row.
2. Each of the digits `1-9` must occur exactly once in each column.
3. Each of the digits `1-9` must occur exactly once in each of the 9 `3x3` sub-boxes of the grid.

The `'.'` character indicates empty cells.

Example 1:



Input: `board =`
```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[".","9","8",".",".",".",".","6","."],
["8",".",".",".","6",".",".",".","3"],
["4",".",".","8",".","3",".",".","1"],
["7",".",".",".",".","2",".",".",".","6"],
[".","6",".",".",".",".",".","2","8","."],
[".",".",".","4","1","9",".",".","5"],
[".",".",".",".",".","8",".",".","7","9"]]
```

Output:
```
[["5","3","4","6","7","8","9","1","2"],
["6","7","2","1","9","5","3","4","8"],
["1","9","8","3","4","2","5","6","7"],
["8","5","9","7","6","1","4","2","3"],
["4","2","6","8","5","3","7","9","1"],
["7","1","3","9","2","4","8","5","6"], ["9","6","1","5","3","7","2","8","4"],
["2","8","7","4","1","9","6","3","5"], ["3","4","5","2","8","6","1","7","9"]]
```

Explanation: The input board is shown above and the only valid solution is shown below:



5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

```
class Solution {
    public void solveSudoku(char[][] board) {
        solve(board, 0, 0);
    }
    private boolean solve(char[][] brd, int r, int c){
        if (brd.length == r){
            return true;
        }
        int newR = 0, newC = 0;
        if (c != brd.length - 1){
            newR = r; newC = c + 1;
        }else{
            newR = r + 1; newC = 0;
        }

        if (brd[r][c] != '.'){
            if (solve(brd, newR, newC)){
                return true;
            }
        }else{
            for (int i = 1; i <= 9; i++){
                if (isSafe(brd, r, c, i)){
                    brd[r][c] = (char)(i + '0');
                    if (solve(brd, newR, newC)){
                        return true;
                    }else{
                        brd[r][c] = '.';
                    }
                }
            }
        }
        return false;
    }

    private boolean isSafe(char[][] brd, int r, int c, int num){
```

```
        for (int i = 0; i < 9; i++){
            if (brd[r][i] == (char) (num + '0')){
                return false;
            }else if(brd[i][c] == (char) (num + '0')){
                return false;
            }
        }
        int cellR = (r/3)*3, cellC = (c/3)*3;
        for (int i = cellR; i < cellR + 3; i++){
            for (int j = cellC; j < cellC + 3; j++){
                if (brd[i][j] == (char) (num + '0')){
                    return false;
                }
            }
        }
        return true;
    }}
```

3. Permutations I, II

I. Given an array `nums` of distinct integers, return *all the possible permutations*. You can return the answer in any order.

Example 1: `Input: nums = [1,2,3]`

`Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

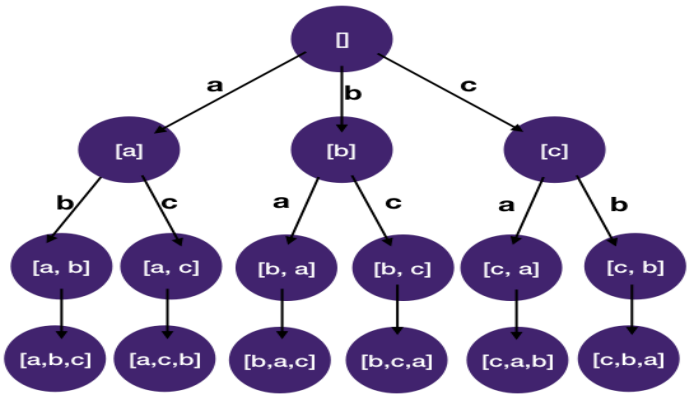
Example 2: `Input: nums = [0,1] Output: [[0,1],[1,0]]`

Classic combinatorial search problem, we can solve it using 3-step system

1. **Identify states:** What state do we need to know whether we have reached a solution (and using it to construct a solution if the problem asks for it). We need a state to keep track of the list of letters we have chosen for the current permutation

What state do we need to decide which child nodes should be visited next and which ones should be pruned? We have to know what are the letters left that we can still use (since each letter can only be used once).

2. Draw the State-space Tree



3. DFS on the State-space tree

   Using the backtracking template as basis, we add the two states we identified in step 1

Backtracking

```
class Solution {
    List<List<Integer>> list = new ArrayList();
    public List<List<Integer>> permute(int[] nums) {
        bt(nums, new ArrayList());
        return list;
    }
    private void bt(int[] nums, List<Integer> cur){
        if (nums.length == cur.size()) list.add(new ArrayList(cur));
        for (int i = 0; i < nums.length ; i++){
            if (cur.contains(nums[i])) continue;
            cur.add(nums[i]);
            bt(nums, cur);
            cur.remove(cur.size() - 1);
        }}}
```

Iterative approach

BFS Intuition: This problem is again similar to 78. Subsets I, solution for which is here
The difference here is - instead of creating subsets by copying the in-process result of `List<List<Integer>>` and then adding the current value to the copied subsets, we add the new element to different positions to the in-process `List<List<Integer>>` and the created set becomes the new result.

For each List<Integer> in result we add the current element in different positions and store each newly constructed list in currResult. At the end of current iteration we update the result with currResult

```
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        result.add(new ArrayList<>());

        for(int num : nums){
            if (result.isEmpty()) result.add(List.of(num));
            else {
                List<List<Integer>> currResult = new ArrayList<>();
                for (List<Integer> list : result){
                    for(int i=0; i <= list.size(); i++){
                        List<Integer> newList = new ArrayList<>(list);
                        newList.add(i, num);
                        currResult.add(newList);
                    }
                }
                result = currResult;
            }
        }
        return result;
    }
}
```

II. Given a collection of numbers, nums, that might contain duplicates, return *all possible unique permutations in any order.*

Example 1: `Input: nums = [1,1,2]`

`Output:[[1,1,2], [1,2,1], [2,1,1]]`

Example 2: `Input: nums = [1,2,3]`

`Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Using Queue

```
public class Solution {
    public List<List<Integer>> permuteUnique(int[] nums) {
        if (nums.length == 0) return new ArrayList<>();
        Queue<List<Integer>> queue = new LinkedList<>();
        queue.add(Collections.singletonList(nums[0]));
        for (int i = 1; i < nums.length; i++) {
            int queueSize = queue.size();
```

```
            while (queueSize-- > 0) {
                List<Integer> record = queue.poll();
                for (int j = 0; j <= record.size(); j++) {
                    boolean shouldBreak = j == record.size() || nums[i] ==
record.get(j);
                    List<Integer> tmp = new ArrayList<>(record);
                    tmp.add(j, nums[i]);
                    queue.add(tmp);
                    if (shouldBreak) break;
                }
            }
        }
        return new ArrayList<>(queue);
    }
}
```

Using HashMap N backtracking TC: O(N*N!) | SC: O(N) |

```
class Solution {
    List<List<Integer>> ans = new ArrayList();
    HashMap<Integer, Integer> map = new HashMap();
    public List<List<Integer>> permuteUnique(int[] nums) {
        for (int n: nums){
            map.put(n, map.getOrDefault(n, 0) + 1);
        }
        solve(nums.length, new ArrayList());
        return ans;
    }
    private void solve(int len, List<Integer> lis){
        if (lis.size() == len){
            ans.add(new ArrayList(lis));
            return ;
        }

        for (int N : map.keySet()){
            int cnt = map.get(N);
            if (cnt == 0) continue;

            map.put(N, cnt - 1);
            lis.add(N);
            solve(len, lis);
            lis.remove(lis.size() - 1);
            map.put(N, cnt);
        }}}
```

```
public void traverse(List<List<Integer>> resultList, List<Integer> current, int[]
nums, boolean[] used) {
      if( current.size() == nums.length )
```

```
            resultList.add(new ArrayList<Integer>(current));
      else{
            for(int i = 0; i < nums.length; i++) {
                  if( used[i] || ( i > 0 && nums[i] == nums[i - 1]
               && !used[i - 1]) ) // remove duplicate
                        continue;
                  current.add(nums[i]);
                  used[i] = true;
                  traverse(resultList, current, nums, used);
                  used[i] = false;
                  current.remove(current.size() - 1);
            }
      }
}

public List<List<Integer>> permuteUnique(int[] nums) {
      Arrays.sort(nums); // sort is necessary
      List<List<Integer>> resultList = new ArrayList<List<Integer>>();
      traverse(resultList, new ArrayList<Integer>(), nums,
                        new boolean[nums.length]);
      return resultList;
}
```

4. Combinations

Given two integers `n` and `k`, return *all possible combinations of* `k` *numbers out of the range* `[1, n]`. You may return the answer in any order.

Example 1: `Input: n = 4, k = 2`

`Output: [ [2,4], [3,4], [2,3], [1,2], [1,3], [1,4], ]`

Example 2: `Input: n = 1, k = 1 Output: [[1]]`

Iterative solution

```
class Solution {
    public List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> result = new ArrayList<>();
        if (k == 0) {
            result.add(new ArrayList<>());
            return result;
        }
        List<Integer> temp = new ArrayList<>(k + 1);
        for (int i = 1; i <= k; i++) {
```

```
            temp.add(i);
        }
        temp.add(n + 1);

        int i = 0;
        while (i < k) {
            result.add(new ArrayList<>(temp.subList(0, k)));
            i = 0;
            while (i < k && (temp.get(i) + 1 == temp.get(i + 1))) {
                temp.set(i, i + 1);
                i++;
            }
            temp.set(i, temp.get(i) + 1);
        }

        return result;
    }
}
```

Backtracking solution

```
class Solution {
    List<List<Integer>> ans = new ArrayList();
    int n, k;
    public List<List<Integer>> combine(int n, int k) {
        this.n = n; this.k = k;
        solve(1, new ArrayList());
        return ans;
    }

    private void solve(int s, List<Integer> lis){
        if (lis.size() == k){
            ans.add(new ArrayList(lis));
            return ;
        }
        for (int i = s; i <= n; i++){
            lis.add(i);
            solve(i + 1, lis);
            lis.remove(lis.size() - 1);
        } }}
```

5.

III. DYNAMIC PROGRAMMING

A. Basic approaches

1. Stock Buy And Sell

Problem Statement: You are given an array of prices where prices[i] is the price of a given stock on an ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: prices = [7,1,5,3,6,4] **Output:** 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note: That buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1] **Output:** 0

Explanation: In this case, no transactions are done and the max profit = 0.

Method -1 Basic storing

```
class Solution {
   public int maxProfit(int[] arr) {
    int maxPro = 0, minPrice = Integer.MAX_VALUE;
    for (int i = 0; i < arr.length; i++) {
      minPrice = Math.min(minPrice, arr[i]);
      maxPro = Math.max(maxPro, arr[i] - minPrice);
    }
    return maxPro; }}
```

2. Maximum Product Subarray in an Array

Given an array that contains both negative and positive integers, find the maximum product subarray.

Example 1:
Input: Nums = [1,2,3,4,5,0]
Output: 120
Explanation: In the given array, we can see 1×2×3×4×5 gives maximum product value.

Example 2:
Input: Nums = [1,2,-3,0,-4,-5]
Output: 20
Explanation: In the given array, we can see (-4)×(-5) gives maximum product value.

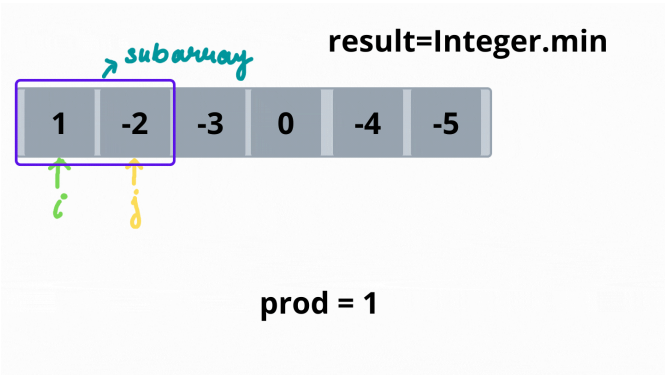
Solution: Brute Force

Approach: Find all possible subarrays of the given array. Find the product of each subarray. Return the maximum of all them.

Following are the steps for the approach:-

- Run a loop on the array to choose the start point for each subarray.
- Run a nested loop to get the end point for each subarray.
- Multiply elements present in the chosen range.

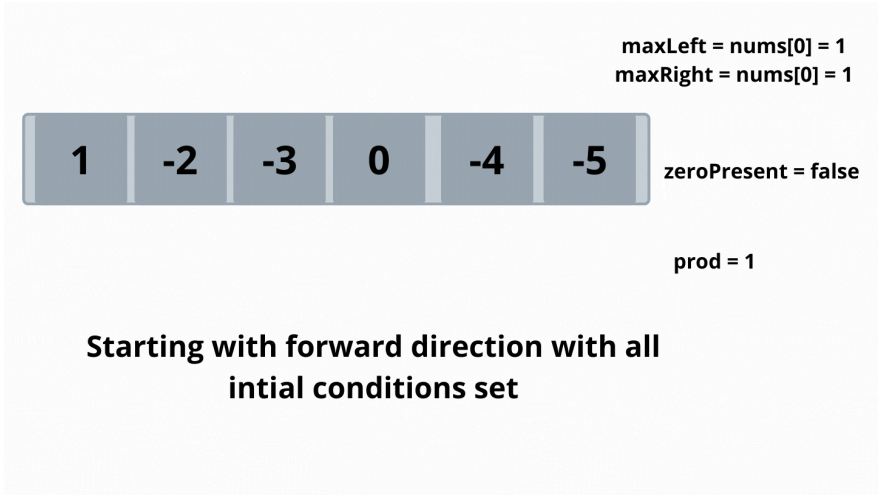
Dry Run:



```
public class Main {
      static int maxProductSubArray(int arr[]) {
          int result = Integer.MIN_VALUE;
          for(int i=0;i<arr.length-1;i++)
              for(int j=i+1;j<arr.length;j++) {
                  int prod = 1;
                  for(int k=i;k<=j;k++)
                      prod *= arr[k];
                  result = Math.max(result,prod);
              }
         return result;
      }
```

Solution: Optimized Brute Force
Approach: We can optimize the brute force by making 3 nested iterations to 2 nested iterations
Following are the steps for the approach:

Run a loop to find the start of the subarrays.
Run another nested loop
Multiply each element and store the maximum value of all the subarray.




```
public class Main {
      static int maxProductSubArray(int arr[]) {
          int result = arr[0];
          for(int i=0;i<arr.length-1;i++) {
              int p = arr[i];
              for(int j=i+1;j<arr.length;j++) {
                  result = Math.max(result,p);
                  p *= arr[j];
              }
              result = Math.max(result,p);
          }
         return result; }
```

Solution: Two Traversals

Approach:

Idea is to find the maximum product from both sides,i.e, once in a forwarding direction and another in the backward direction.
Following are the steps for the approach:

Iteration from left to right once to get maximum product for forward direction.
If zero is encountered, we set all variables again to initial value.

Iteration from right to left once again to get maximum product for backward direction.
If zero is encountered, we set all variables again to zero to find a new subarray with maximum product.

Once both iterations are completed, the overall result for the maximum product subarray of the given array is the maximum product obtained from both the iterations.

```
public class Main{
       static int maxProductSubArray(int arr[]) {
           int maxLeft = arr[0], maxRight = arr[0], prod = 1;
           boolean zeroPresent = false;
           for (int i: arr) {
               prod *= i;
               if (i == 0) {
                   zeroPresent = true;
                   prod = 1;
                   continue; }
               maxLeft = Math.max(maxLeft,prod); }
           prod = 1;
           for(int j=arr.length-1;j>=0;j--) {
               prod *= arr[j];
               if(arr[j] == 0) {
                   zeroPresent = true;
                   prod = 1;
                   continue;  }
               maxRight = Math.max(maxRight,prod); }
           if (zeroPresent) return Math.max(Math.max(maxLeft,maxRight),0);
           return Math.max(maxLeft,maxRight); }}
```

Solution: Kadane's Algorithm

Approach: The following approach is motivated by Kandane's algorithm. To know Kadane's Algorithm follow **Kadane's Algorithm**

The pick point for this problem is that we can get the maximum product from the product of two negative numbers too.
Following are the steps for the approach:
Traverse through the array.
For each element, we will store prod1 and prod2.

Prod1 is maximum of current element, product of current element and prod1 and product of current element and prod2
Prod2 is minimum of current element, product of current element and prod1 and product of current element and prod2
Return maximum of result and prod1

```
public class Main {
       static int maxProductSubArray(int arr[]) {
         int prod1 = arr[0],prod2 = arr[0],result = arr[0];

    for (int i=1; i < arr.length; i++) {
        int temp = Math.max(arr[i],Math.max(prod1*arr[i],prod2*arr[i]));
        prod2 = Math.min(arr[i],Math.min(prod1*arr[i],prod2*arr[i]));
        prod1 = temp;

        result = Math.max(result,prod1);
    }

    return result; }
```

B. 1 - D Array Variations

1. Two City Scheduling

A company is planning to interview 2n people. Given the array costs where costs[i] = [aCosti, bCosti], the cost of flying the ith person to city a is aCosti, and the cost of flying the ith person to city b is bCosti.

Return *the minimum cost to fly every person to a city* such that exactly n people arrive in each city.

Example 1:

```
Input: costs = [[10,20],[30,200],[400,50],[30,20]]
Output: 110
Explanation:
The first person goes to city A for a cost of 10.
The second person goes to city A for a cost of 30.
The third person goes to city B for a cost of 50.
The fourth person goes to city B for a cost of 20.

The total minimum cost is 10 + 30 + 50 + 20 = 110 to have half the people
interviewing in each city.
```

```
class Solution {
    public int twoCitySchedCost(int[][] costs) {
        Arrays.sort(costs, (a, b) -> (a[0] - a[1]) - (b[0] - b[1]) );
        int ans = 0, n = costs.length;
```

```
        for (int i = 0; i < n/2; i++){
            ans += costs[i][0] + costs[i + n/2][1];
        }
        return ans; }}
```

2. Coin change

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money. Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

```
Input: coins = [2], amount = 3
Output: -1
```

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        Arrays.sort(coins);
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;

        for (int coin: coins){
            for (int amt = coin; amt <= amount; amt++){
                if (amt - coin >= 0){
                    dp[amt] = Math.min(1 + dp[amt - coin], dp[amt]);
                } else break;
            }
        }

        return (dp[amount] <= amount) ? dp[amount] : -1; }}
```

3. Longest Increasing Subsequence

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

A subsequence is a sequence that can be derived from an array by deleting some or no elements without changing the order of the remaining elements. For example, `[3,6,2,7]` is a subsequence of the array `[0,3,1,6,2,2,7]`.

Example 1:

```
Input: nums = [10,9,2,5,3,7,101,18]
Output: 4
Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length is 4.
```

Example 2:

```
Input: nums = [0,1,0,3,2,3]
Output: 4
```

```
class Solution {
    Integer mem[];

    public int lengthOfLIS(int[] nums) {
        if (nums.length == 0) return 0;
        mem = new Integer[nums.length];
        int temp = 0;
        for(int i = 0; i < nums.length; i++) {
            temp = Math.max(temp, checkSeq(nums, i));
        }
        return temp;
    }

    private int checkSeq(int[] nums, int start) {
        if (start == nums.length) return 0;
        if (mem[start] != null) return mem[start];
        mem[start] = 1;
        for (int i = start; i < nums.length; i++) {
            if (nums[i] > nums[start]) {
                mem[start] = Math.max(mem[start], 1 + checkSeq(nums, i));
            }
        }
        return mem[start]; }}
```

```
class Solution {
    public int lengthOfLIS(int[] nums) {

        int ans = 1, n = nums.length;
        if (n <= 1) return n;
        int[] dp = new int[n + 1];
        Arrays.fill(dp, 1);

        for (int i = 1; i < n; i++){
```

```
            for (int j = 0; j < i; j++){
                if (nums[i] > nums[j]){
                    dp[i] = Math.max(1 + dp[j], dp[i]);
                    ans =  Math.max(dp[i], ans);
                }
            }
        }
        return ans;}}
```

```
METHOD - 2 USING BINARY SEARCH
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length];
    int size = 0;
    for (int x : nums) {
        int i = 0, j = size;
        while (i != j) {
            int m = (i + j) / 2;
            if (tails[m] < x)
                i = m + 1;
            else
                j = m;
        }
        tails[i] = x;
        if (i == size) ++size;
    }
    return size; }
```

4. **Number of Longest Increasing Subsequence**

Given an integer array `nums`, return *the number of longest increasing subsequences.*

Notice that the sequence has to be strictly increasing.

Example 1:

```
Input: nums = [1,3,5,4,7]

Output: 2

Explanation: The two longest increasing subsequences are [1, 3, 4, 7] and [1, 3, 5,
7].
```

Example 2:

```
Input: nums = [2,2,2,2,2]

Output: 5

Explanation: The length of longest continuous increasing subsequence is 1, and there
are 5 subsequences' length is 1, so output 5.
```

5. **Find the maximum sum in an array, such that you cannot add alternate elements(A)**

```
//Brute force approach.
class Program {
      public static int maxSubsetSumNoAdjacent(int[] array) {
          return  findMaxSum(array,0);
      }
       static int findMaxSum(int[] array,int index){
              if(index >= array.length){
                     return 0;
              }
              int bestWith = array[index]+findMaxSum(array,index+2);
              int bestWithout = findMaxSum(array,index+1);

              return Math.max(bestWith,bestWithout);
       }
}
//Optimized approach: using Memoization
class Program {
      public static int maxSubsetSumNoAdjacent(int[] array) {
              int[] memo = new int[array.length];
         return findMaxSum(array,0,memo);
      }
       static int findMaxSum(int[] array,int index,int[] memo){
              if(index >= array.length){
                     return 0;
              }
              if(memo[index] ==0){
                     int bestWith = array[index]+findMaxSum(array,index+2,memo);
                int bestWithout = findMaxSum(array,index + 1,memo);
                     memo[index] = Math.max(bestWith,bestWithout);
              }
              return memo[index];
       }
}
```

Variation of House robber problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array nums representing the amount of money of each house, return *the maximum amount of money you can rob tonight without alerting the police*.

Example 1: `Input: nums = [1,2,3,1] Output: 4`

```
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.
```

Example 2: `Input: nums = [2,7,9,3,1] Output: 12`

```
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money
= 1).
Total amount you can rob = 2 + 9 + 1 = 12.
```

Solution:

DP made easy - for DP problems please dont write top-down directly

1. Write recursive solution
2. Convert that into Top-Down or Bottom-Up(Memoization)

```
// Recursion Time - O(n^2) Space - O(n)
class Solution {
    public int rob(int[] nums) {
        if(nums == null || nums.length == 0) return 0;
        return getMaxAmount(nums, nums.length - 1);
    }
    public int getMaxAmount(int[] nums, int n) {
       if (n < 0) return 0;
       if (n == 0) return nums[0];
       if (n == 1) return Integer.max(nums[1], nums[0]);
       return Integer.max(nums[n] + getMaxAmount(nums, n-2),
                          getMaxAmount(nums, n-1));
    } }
```

```
// DP - Memoization Time - O(n^2) Space - O(n)
class Solution {
    public int rob(int[] nums) {
        if(nums == null || nums.length == 0) return 0;
        int dp[] = new int[nums.length];
        Arrays.fill(dp, -1);
        return getMaxAmount(nums, nums.length - 1, dp);
    }
    public int getMaxAmount(int[] nums, int n, int[] dp) {
       if(n < 0) return 0;
       if(n == 0) return nums[0];
       if(n == 1) return Integer.max(nums[1], nums[0]);
       if(dp[n] != -1) return dp[n];
```

```
        dp[n] = Integer.max(nums[n] + getMaxAmount(nums, n-2, dp), getMaxAmount(nums,
n-1, dp));
        return dp[n];
    }}
// DP - Top Down Time - O(n) Space - O(n)
class Solution {
    public int rob(int[] nums) {
        if(nums == null || nums.length == 0) return 0;
        if(nums.length == 1) return nums[0];
        return getMaxAmount(nums);
    }

    public int getMaxAmount(int[] nums) {
       int dp[] = new int[nums.length];
       dp[0] = nums[0];
       dp[1] = Integer.max(nums[1], nums[0]);
       for(int i = 2; i < nums.length; i++) {
           if(dp[i] > 0) return dp[i];
            dp[i] = Integer.max(nums[i] + dp[i-2], dp[i-1]);
       }
       return dp[nums.length-1];
    }
}

// Optimised DP
// Time O(n)
// Space O(1)
class Solution {
    public int rob(int[] nums) {
        if(nums == null || nums.length == 0) return 0;
        if(nums.length == 1) return nums[0];

        nums[1] = Math.max(nums[1], nums[0]);
        for(int i = 2; i < nums.length; i++) {
            nums[i] = Math.max(nums[i] + nums[i-2], nums[i-1]);
        }
         return nums[nums.length-1];
    }
}

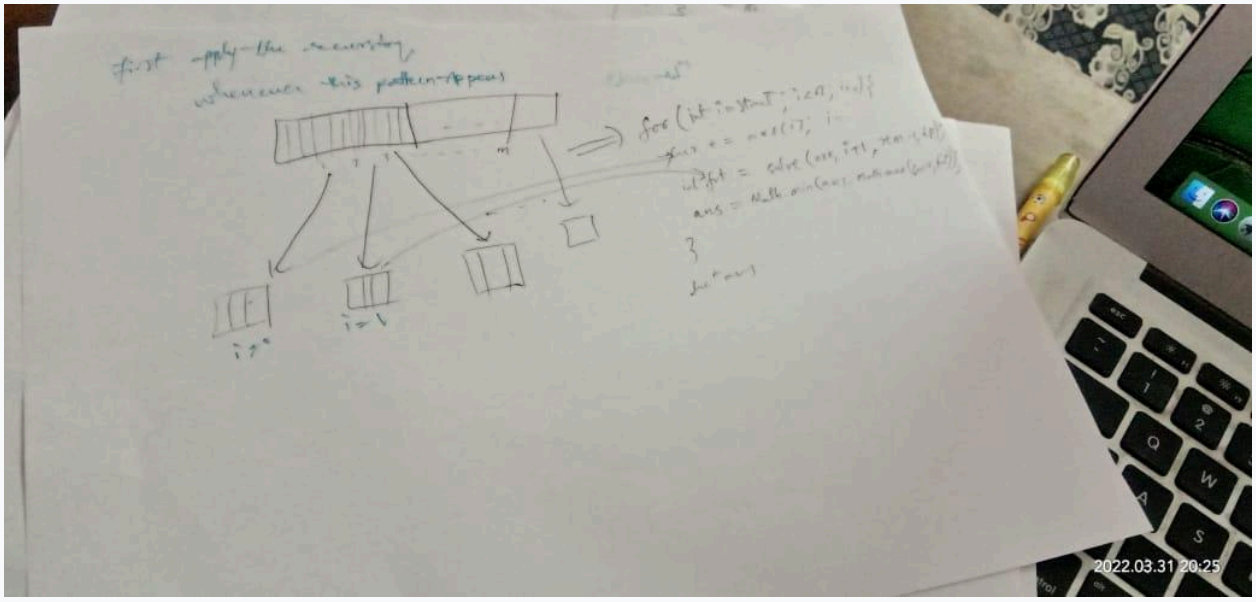
// Method 4 -
//     Optimized Tabulation
//     TC - O(n)
//     SC -O(1)
class Solution {
    public int rob(int[] nums) {
        int prev=nums[0], secPrev=0;
        for(int i = 1; i < nums.length; i++) {
            int pick = nums[i] + secPrev, notPick = prev;
            int curr = Math.max(pick, notPick);
            secPrev = prev;
            prev = curr;
        }
        return prev;
```

```
    }}
```

C. 2-D Arrays Variations

1. Split Array Largest Sum

Given an array `nums` which consists of non-negative integers and an integer `m`, you can split the array into `m` non-empty continuous subarrays. Write an algorithm to minimize the largest sum among these `m` subarrays.



Example 1:

```
Input: nums = [7,2,5,10,8], m = 2
Output: 18
Explanation:
There are four ways to split nums into two subarrays.
```

```
The best way is to split it into [7,2,5] and [10,8],
where the largest sum among the two subarrays is only 18.
```

Also see → **Allocate minimum number of pages on gfg**

Method - 1 Dynamic programming

Intuition

Try all ways of partitioning the input array into m blocks.While doing so, keep track of the minimum of largest sums among all blocks. More concretely, answer is:

min(current partition sum, maximum partition sum for subarray to the right of the current partition)

Key point: Memoize the recursion result so that we don't solve the same subproblem again.

```
recursive
class Solution {
    public int splitArray(int[] nums, int m) {
        return solve(nums, m, 0);
    }
    private int solve(int[] nums, int rem, int start){
        if (rem == 0 && start == nums.length) return 0;
        if (rem == 0 || start == nums.length) return Integer.MAX_VALUE;

        int cur = 0, ans = Integer.MAX_VALUE;
        for (int i = start; i < nums.length; i++){
            cur += nums[i];
            int future = solve(nums, rem - 1, i + 1);
            ans = Math.min(ans, Math.max(cur, future));
        }
        return ans;}}
```

```
class Solution {
    public int splitArray(int[] nums, int m) {
        int[][] memo = new int[nums.length][m+1];
        for (int i = 0; i < memo.length; i++) {Arrays.fill(memo[i], -1); }
        return walk(nums, memo, 0, m);
    }
    private int walk(int[] nums, int[][] memo, int start, int rem) {
          // base case
        if (rem == 0 && start == nums.length) {  return 0; }
        if (rem == 0 || start == nums.length) {
                // if we reach the end and have not used up all partitions
            // or have used up all partitions and have not reached the end,
                  // we do not want to count the current way of partitioning.
            // Return MAX_VALUE so that we don't contribute to the return value.
            return Integer.MAX_VALUE;
        }
        int ans = Integer.MAX_VALUE, curSum = 0;
        if (memo[start][rem] != -1) { return memo[start][rem]; }
```

```
            // try all positions to end the current partition.
        for (int i = start; i < nums.length; i++) {
            curSum += nums[i];
// answer for partitioning the subarray to the right of the current partition,
// with one less partition number allowance, because we already used one
// for the current partition. i.e. (rem - 1).
            int futureSum = walk(nums, memo, i + 1, rem - 1);
// we want to minimum of the largest sum of the partitions.
            ans = Math.min(ans, Math.max(curSum, futureSum));
        }
        memo[start][rem] = ans;
        return ans;}}
```

Analysis Let n = |nums|, and given m, We solve nm number of problems, each time we solve a problem, we access at most n array elements.

Time complexity: ~O(n^2m) **Space complexity**: ~O(nm) for memoization array + call stack of depth n.

Type - 2 dp

```
class Solution {
    public int splitArray(int[] nums, int m) {
        int n = nums.length;

        int[] pfxSum = new int[n + 1];
        for (int i = 0; i < n; i++) {
            pfxSum[i+1] = pfxSum[i] + nums[i];
        }

        // dp[i][j] : result for partition nums[0:i] into j part (for i in [0, n-1])
        // notice that partition into 0 part is not defined, namely dp[i][0] is never
used for all i in [0,n-1]
        int[][] dp = new int[n][m+1];

        for (int i = 0; i < n; i++) Arrays.fill(dp[i], Integer.MAX_VALUE);

        for (int i = 0; i < n; i++) { dp[i][1] = pfxSum[i+1]; }

        for (int i = 1; i < n; i++) {
            for (int j = 2; j <= Math.min(m, i+1); j++) {
                // [0,i] into j part  <==> [0,k] into j-1 part  &  [k+1, i] into 1
part  for k in [0,i-1]
                for (int k = 0; k < i; k++) {
                    dp[i][j] = Math.min(dp[i][j], Math.max(dp[k][j-1], pfxSum[i+1] -
pfxSum[k+1]));
                }
            }
```

```
        }

        return dp[n-1][m];}}
```

Method - 2 Binary search

We can first restrict the search space to [max(nums), sum(nums)], the lower bound max(nums) means the biggest one is grouped solely and the upper bound sum(nums) means there is only one group. To search in a consecutive range, we can easily think of Binary Search to decrease search space. The valid condition in this problem is almost the same as LC1011, which is whether we can separate this array into m groups and each group's sum is no larger than current value.

Since the subarray should be continuous, it is much easier. We can simply calculate how much groups we need and compare it to m. Time Complexity: O(n log(sum(nums))), where n is the length of nums[]. Space Complexity: O(1)

Similar to: LC1011 Capacity To Ship Packages Within D Days

```
class Solution {
    public int splitArray(int[] nums, int m) {
        //range[max(nums), sum(nums)]
        int lo = 0, hi = 0;
        for(int num: nums){
            if(num > lo){
                lo = num;
            }
            hi += num;
        }
        while (lo <= hi){
            int mid = lo + (hi - lo) / 2;
            if(mIsValid(nums, m, mid)){
                hi = mid - 1;
            }else{
                lo = mid + 1;
            }
        }
        return lo; }
    private boolean mIsValid(int[] nums, int m, int val){
        //if group into m groups, each group's sum <= val
        int groupNeed = 0, curSum = 0;
        for(int num: nums){
            curSum += num;
            if(curSum > val){
                ++groupNeed;
                curSum = num;
            }
        }
        if(curSum > 0){ ++groupNeed; }
```

```
        return groupNeed <= m;
    }}
```

2. Longest Common Subsequence

Given two strings `text1` and `text2`, return *the length of their longest common subsequence.* If there is no common subsequence, return `0`. A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, `"ace"` is a subsequence of `"abcde"`.

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

```
Input: text1 = "abcde", text2 = "ace"      Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.
```

Example 2:

```
Input: text1 = "abc", text2 = "abc"     Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.
```

Find LCS;

Let X be “XMJYAUZ” and Y be “MZJAWXU”. The longest common subsequence between X and Y is “MJAU”. The following table shows the lengths of the longest common subsequences between prefixes of X and Y. The ith row and jth column shows the length of the LCS between $X_{1..i}$ and $Y_{1..j}$.

		0	1	2	3	4	5	6	7
		Ø	M	Z	J	A	W	X	U
0	Ø	0	0	0	0	0	0	0	0
1	X	0	0	0	0	0	0	1	1
2	M	0	1	1	1	1	1	1	1
3	J	0	1	1	2	2	2	2	2
4	Y	0	1	1	2	2	2	2	2
5	A	0	1	1	2	3	3	3	3
6	U	0	1	1	2	3	3	3	4
7	Z	0	1	2	2	3	3	3	4

you can refer to here for more details.

Method 1:

```java
class Solution {//Top-Down Memoized
    public int longestCommonSubsequence(String text1, String text2) {
        return dfs(text1, text2, 0, 0, new Integer[text1.length()][text2.length()]);
    }
    private int dfs(String text1, String text2, int i, int j, Integer[][] memo){
        if(i == text1.length() || j == text2.length()) return 0;
        if(memo[i][j] != null) return memo[i][j];

        if(text1.charAt(i) == text2.charAt(j))
            return 1 + dfs(text1, text2, i + 1, j + 1, memo);
        return memo[i][j] = Math.max(dfs(text1, text2, i + 1, j, memo), dfs(text1,
text2, i, j + 1, memo));
    }
}
class Solution {//Bottom-up DP
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length(), dp[][] = new int[m + 1][n + 1];
        for(int i = 1; i <= m; i++)
            for(int j = 1; j <= n; j++)
                if(text1.charAt(i - 1) == text2.charAt(j - 1))
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);

        return dp[m][n];
    }}
```

for understand method 2

text 1 : babcde, tex2: ace

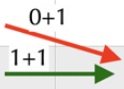
k\str2	"	a	c	e
0 (a idx 1	0	1	1	1
1 (b idx 0	0	0	0	0

k\str2	"	a	c	e
0 (c idx 3	0	1	2	2
1 (b idx 2	0	1	1	1

k\str2	"	a	c	e
0 (e idx 5	0	1	2	3
1 (d idx 4	0	1	2	2

if(text1.charAt(i) == text2.charAt(j)), why not **dp[k][j+1] = 1 + dp[k][j]?**
think of bsb & bms

k\str2	"	b	s	b
0	0	0	0	0
1 (b	0	1	1	1



Method 2 - optimized O(n)

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int[] dp = new int[text2.length()+1];
        for(int i = 0; i< text1.length(); i++){
            int prev = dp[0];
            for(int j = 1; j < dp.length; j++){
                int temp = dp[j];
                if(text1.charAt(i) != text2.charAt(j-1)){
                    dp[j] = Math.max(dp[j-1], dp[j]);
                }else{
                    dp[j] = prev +1;
                }
                prev = temp;
            }
        }
        return dp[dp.length-1];    } }
```

3. Longest Palindromic Subsequence

Given a string `s`, find *the longest palindromic subsequence's length in* `s`.

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

```
Input: s = "bbbab" Output: 4
```

```
Explanation: One possible longest palindromic subsequence is "bbbb".
```

Example 2:

```
Input: s = "cbbd" Output: 2
```

```
Explanation: One possible longest palindromic subsequence is "bb".
```

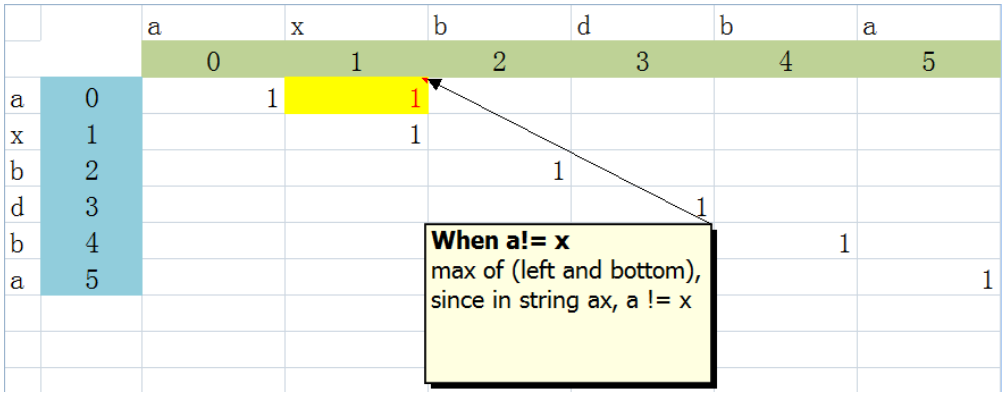
SOLUTION:

I draw a table for better illustrating. The logic is same with @tankztc Example: input string s : axbdba

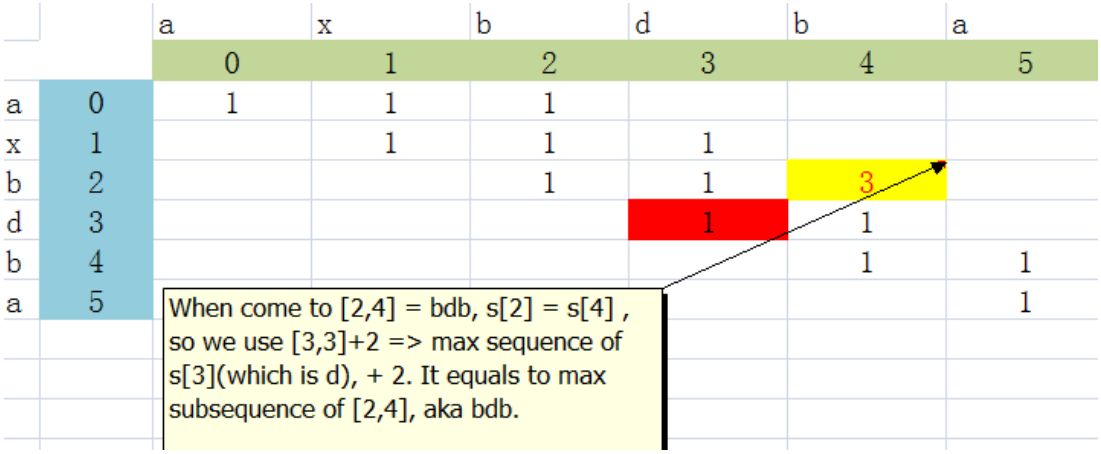
step1: fill the table with start case, dp[i,i] == 1, sicne all single char in a string is a palindrome, which length is 1.

		a	x	b	d	b	a
		0	1	2	3	4	5
a	0	1					
x	1		1				
b	2			1			
d	3				1		
b	4					1	
a	5						1

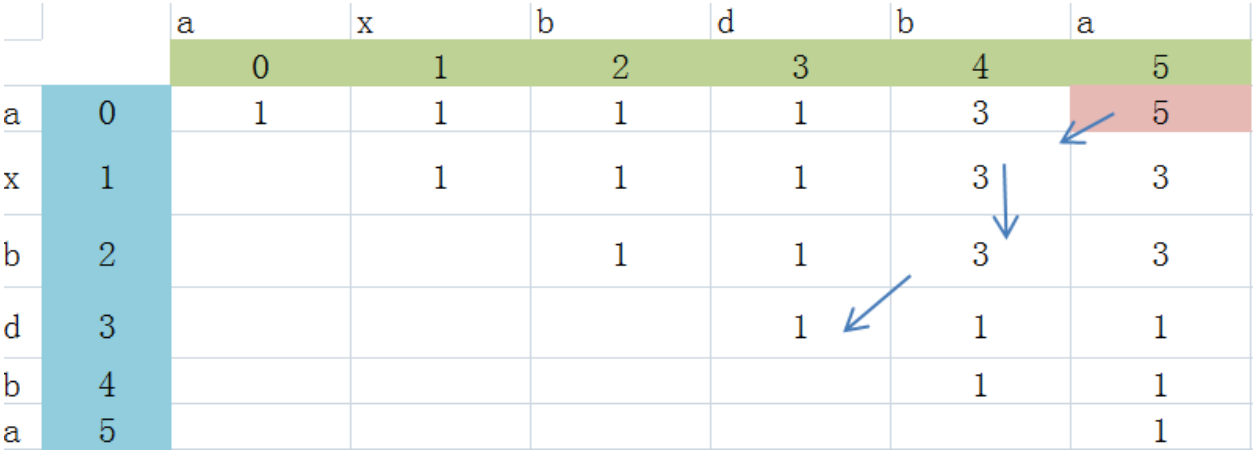
step 2:When s[i] != s[j] case



Step 3: When s[i] == s[j] case



Step 4: fill out all the table, the dp[s.Length,0] is what we are looking for. And as the arrow point, the subsequence is ##dba, which is abdba



```
class Solution {
    int[][] dp;
    public int longestPalindromeSubseq(String a) {
        String b = reverse(a);
        dp = new int[a.length() + 1][a.length() + 1];
        for (int i = 0; i <= a.length(); i++) Arrays.fill(dp[i], -1);
        return lcs(a, b, a.length(), a.length());
    }
    private int lcs(String a, String b, int n, int m){
        if (n == 0 || m == 0) return 0;
        if (dp[n][m] != -1) return dp[n][m];

        if (a.charAt(n - 1) == b.charAt(m - 1)){
            dp[n][m] = 1 + lcs(a, b, n - 1, m - 1);
        }else{
            dp[n][m] = Math.max(lcs(a, b, n, m - 1), lcs(a, b, n - 1, m));
        }
        return dp[n][m]; }

    private String reverse(String a){
        String c = "";
        while (a.length() != c.length()){
            c += a.charAt(a.length() - 1 - c.length());
        }
        return c;
    }}
```

```
public class Solution {
    public int longestPalindromeSubseq(String s) {
        return helper(s, 0, s.length() - 1, new Integer[s.length()][s.length()]);
    }

    private int helper(String s, int i, int j, Integer[][] memo) {
        if (memo[i][j] != null) {
            return memo[i][j];
        }
        if (i > j)      return 0;
        if (i == j)     return 1;

        if (s.charAt(i) == s.charAt(j)) {
            memo[i][j] = helper(s, i + 1, j - 1, memo) + 2;
        } else {
            memo[i][j] = Math.max(helper(s, i + 1, j, memo), helper(s, i, j - 1,
memo));
        }
        return memo[i][j];
    }
}
```

4. Maximum sum increasing subsequence

Given an array of n positive integers. Find the sum of the maximum sum subsequence of the given array such that the integers in the subsequence are sorted in increasing order i.e. increasing subsequence.

Example 1:

Input: N = 5, arr[] = {1, 101, 2, 3, 100}

Output: 106

Explanation:The maximum sum of a

increasing sequence is obtained from

{1, 2, 3, 100}

However, the post only covered code related to finding maximum sum of increasing subsequence, but not to the construction of subsequence. In this post, we will discuss how to construct Maximum Sum Increasing Subsequence itself.

Let arr[0..n-1] be the input array. We define vector L such that L[i] is itself is a vector that stores Maximum Sum Increasing Subsequence of arr[0..i] that ends with arr[i]. Therefore for index i, L[i] can be recursively written as

L[0] = {arr[0]}

L[i] = {MaxSum(L[j])} + arr[i] where j < i and arr[j] < arr[i]

= arr[i], if there is no j such that arr[j] < arr[i]

For example, for array [3, 2, 6, 4, 5, 1],

L[0]: 3

L[1]: 2

L[2]: 3 6

L[3]: 3 4

L[4]: 3 4 5

L[5]: 1

```java
class GFG {
    static int findSum(Vector<Integer> arr) {
        int sum = 0;
        for (int i : arr) sum += i;
        return sum; }
    // Function to construct Maximum Sum Increasing Subsequence
    static void printMaxSumIs(int[] arr, int n) {
        // L[i] - The Maximum Sum Increasing
        // Subsequence that ends with arr[i]
        @SuppressWarnings("unchecked")
        Vector<Integer>[] L = new Vector[n];

        for (int i = 0; i < n; i++)
            L[i] = new Vector<>();
        // L[0] is equal to arr[0]
        L[0].add(arr[0]);
        // start from index 1
        for (int i = 1; i < n; i++) {
            // for every j less than i
            for (int j = 0; j < i; j++) {
                /*
                * L[i] = {MaxSum(L[j])} + arr[i]
                  where j < i and arr[j] < arr[i]
                */
                if ((arr[i] > arr[j])
                    && (findSum(L[i]) < findSum(L[j]))) {
                    for (int k : L[j])
                        if (!L[i].contains(k))
                            L[i].add(k);
                } }
            // L[i] ends with arr[i]
            L[i].add(arr[i]);
            // L[i] now stores Maximum Sum Increasing
            // Subsequence of arr[0..i] that ends with
            // arr[i]
        }
        Vector<Integer> res = new Vector<>(L[0]);
        // res = L[0];
        // find max
        for (Vector<Integer> x : L)
            if (findSum(x) > findSum(res))
                res = x;

        // max will contain result
        for (int i : res) System.out.print(i + " ");
        System.out.println();
    }
    // Driver Code
    public static void main(String[] args) {
        int[] arr = { 3, 2, 6, 4, 5, 1 };
        int n = arr.length;
        // construct and print Max Sum IS of arr
        printMaxSumIs(arr, n); }}
```

5. **Matrix Chain Multiplication**
6. **Palindrome Partitioning (MCM Variation N normal)**

Given a string `s`, partition `s` such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of `s`.

A palindrome string is a string that reads the same backward as forward.

Example 1: `Input: s = "aab" Output: [["a","a","b"],["aa","b"]]`

Example 2: `Input: s = "a" Output: [["a"]]`

`METHOD - 1 USING DP`

```
class Solution {
    public void backtrack (String s, int start, int length, boolean[][] dp,
List<String> list, List<List<String>> answer) {
        if (start == length) {   answer.add (new ArrayList<> (list));  }
        for (int end = start; end < length; end++) {
            if (dp[start][end]) {
                list.add (s.substring (start, end + 1));
                backtrack (s, end + 1, length, dp, list, answer);
                list.remove (list.size () - 1);
            }
        }
    }
    public List<List<String>> partition(String s) {
        int length = s.length ();
        boolean[][] dp = new boolean[length][length];
        List<List<String>> answer = new ArrayList<> ();
        if (s == null || length == 0) {  return answer;   }

        for (int end = 0; end < length; end++) {
            for (int start = 0; start <= end; start++) {
                if (s.charAt (start) == s.charAt (end)) {
                    if (start == end) {
                        dp[start][end] = true;
                    }
                    else if (end - start == 1) {
                        dp[start][end] = true;
                    }
                    else if (dp[start + 1][end - 1]) {
                        dp[start][end] = true;
                    }
                }
            }}
        backtrack (s, 0, length, dp, new ArrayList<> (), answer);
        return answer;
    }}
```

MCM VARIATION

```java
class Solution {
    List<List<String>> list;
    public List<List<String>> partition(String s) {
        list = new ArrayList();
        if (s.equals("")){
            list.add(new ArrayList());
            return list;
        }
        find(s, new ArrayList());
        if(isPalin(s)){
            List<String> temp = new ArrayList();
            temp.add(s);
            list.add(temp);
        }
       return list;
     }
    void find(String s, ArrayList<String> temp){
        int n = s.length();
        for(int k = 1; k<n; k++){
            String left = s.substring(0, k);
            if( isP(left) ){
                temp.add(left);
                String right = s.substring(k, n);
                if( isP(right)){
                    temp.add(right);
                    list.add(new ArrayList(temp));
                    temp.remove(temp.size()-1);
                }
                find(right, temp);
                temp.remove(temp.size()-1);
            } } }
    boolean isPalin(String s){
        int i = 0,  j = s.length()-1;
        while(i < j){
            if (s.charAt(i) != s.charAt(j))  return false;
            i++;  j--;
        }
        return true;
    }}
```

```java
class Solution {
    List<List<String>> list = new ArrayList();// using backtrack only
    public List<List<String>> partition(String s) {
        bt(s, 0, new ArrayList());
        return list;
    }
    private void bt(String s, int start, List<String> cur){
        if (start == s.length())  list.add(new ArrayList(cur));
        for (int i = start ; i < s.length() ; i++){
            if (isPalin(s.substring(start, i + 1))) { // isPalin is same as prev. q
            cur.add(s.substring(start, i + 1));
            bt(s, i + 1, cur);
            cur.remove(cur.size() - 1);}
        } }}
```

7. Maximum sum path in the matrix

Given a sequence of matrices, find the most efficient way to multiply these matrices together. The efficient way is the one that involves the least number of multiplications.
The dimensions of the matrices are given in an array **arr[]** of size **N** (such that N = number of matrices + 1) where the **ith** matrix has the dimensions **(arr[i-1] x arr[i])**.

Example 1:

Input: N = 5 arr = {40, 20, 30, 10, 30} **Output:** 26000

Explanation: There are 4 matrices of dimension 40x20, 20x30, 30x10, 10x30. Say the matrices are named as A, B, C, D. Out of all possible combinations, the most efficient way is (A*(B*C))*D.
The number of operations are - 20*30*10 + 40*20*10 + 40*10*30 = 26000.

Using recursion

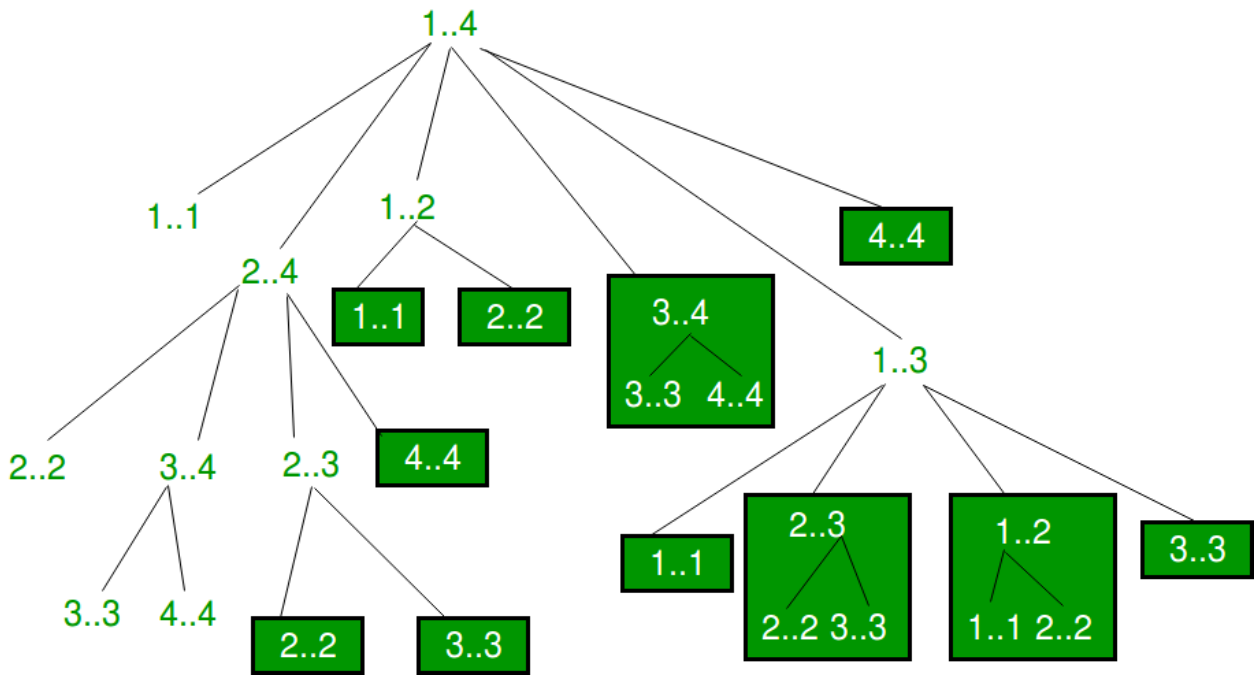
```
static int MatrixChainOrder(int p[], int i, int j) {
        if (i == j return 0;
        int min = Integer.MAX_VALUE;

        // place parenthesis at different places between
        // first and last matrix, recursively calculate
        // count of multiplications for each parenthesis
        // placement and return the minimum count
        for (int k = i; k < j; k++) {
            int count = MatrixChainOrder(p, i, k)
                        + MatrixChainOrder(p, k + 1, j)
                        + p[i - 1] * p[k] * p[j];

            if (count < min) min = count;
        }
        return min;  }         // Return minimum count
```

Output

```
Minimum number of multiplications is 30
```



The time complexity of the above naive recursive approach is exponential. It should be noted that the above function computes the same subproblems again and again. See the following recursion tree for a matrix chain of size 4. The function MatrixChainOrder(p, 3, 4) is called two times. We can see that there are many subproblems being called more than once.

Since the same subproblems are called again, this problem has the Overlapping Subproblems property. So the Matrix Chain Multiplication problem has both properties (see this and this) of a dynamic programming problem. Like other typical Dynamic Programming(DP) problems, recomputations of same subproblems can be avoided by constructing a temporary array m[][] in bottom up manner.

Dynamic Programming Solution

Following is the implementation of the Matrix Chain Multiplication problem using Dynamic Programming

```
class GFG{
  static int[][] dp = new int[100][100];
  // Function for matrix chain multiplication
  static int matrixChainMemoised(int[] p, int i, int j) {
    if (i == j)  return 0;

    if (dp[i][j] != -1)  return dp[i][j];
    dp[i][j] = Integer.MAX_VALUE;

    for (int k = i; k < j; k++){
      dp[i][j] = Math.min(
        dp[i][j], matrixChainMemoised(p, i, k)
        + matrixChainMemoised(p, k + 1, j)
        + p[i - 1] * p[k] * p[j]);
    }
    return dp[i][j];
  }

  static int MatrixChainOrder(int[] p, int n)  {
    int i = 1, j = n - 1;
    return matrixChainMemoised(p, i, j);
  }
  // Driver Code
  public static void main (String[] args){
    int arr[] = { 1, 2, 3, 4 };
    int n= arr.length;

    for (int[] row : dp)
      Arrays.fill(row, -1);

    System.out.println("Minimum number of multiplications is " +
MatrixChainOrder(arr, n));
  }
}
```

8. Shortest Common Supersequence

Given two strings `str1` and `str2`, return *the shortest string that has both* `str1` *and* `str2` *as subsequences*. If there are multiple valid strings, return any of them. A string `s` is a subsequence of string `t` if deleting some number of characters from `t` (possibly `0`) results in the string `s`.

Example 1: `Input: str1 = "abac", str2 = "cab" Output: "cabac"`
`Explanation: str1 = "abac" is a subsequence of "cabac" because we can delete the first "c". str2 = "cab" is a subsequence of "cabac" because we can delete the last "ac". The answer provided is the shortest such string that satisfies these properties.`
Example 2: `Input: str1 = "aaaaaaaa", str2 = "aaaaaaaa" Output: "aaaaaaaa"`

```
MY SOLN.
class Solution {
    int[][] dp;
    public String shortestCommonSupersequence(String str1, String str2) {
         if (str1.length() > str2.length()){
             return shortestCommonSupersequence(str2, str1); }
        dp = new int[str1.length() + 1][str2.length() + 1];
        for (int i = 0; i < str1.length() + 1; i++)
            for (int j = 0; j < str2.length() + 1; j++)  dp[i][j] = -1;

        solveLCS(str1, str2, str1.length(), str2.length());
        String s = "";
        int n = str1.length(), m = str2.length();
        while(n > 0 && m > 0){
            if (str1.charAt(n - 1) == str2.charAt(m - 1)){
                s += str2.charAt(m - 1);
                n--;m--;
            }else{
                if (dp[n][m - 1] > dp[n - 1][m]){
                    s += str2.charAt(m - 1);
                    m--;
                }else{
                    s += str1.charAt(n - 1);
                    n--;
                }
            }
        }
        while(n > 0) {
            s += str1.charAt(n - 1);  n--;   }
        while(m > 0) {
           s += str2.charAt(m - 1);  m--; }
        return reverse(s);
    }
    private int solveLCS(String s1, String s2, int n, int m){
        if (n == 0 || m == 0) return 0;
        if (dp[n][m] != -1) return dp[n][m];
        if (s1.charAt(n - 1) == s2.charAt(m - 1)){
            dp[n][m] = 1 + solve(s1, s2, n - 1, m - 1);
        }else{
            dp[n][m] = Math.max(solve(s1, s2, n, m - 1), solve(s1, s2, n - 1, m));
```

```
        }
        return dp[n][m];
    }

    private String reverse(String s){
        int j = s.length() - 1;
        String str = "";
        while (j >= 0){
            str += s.charAt(j);
            j--;
        }
        return str;}}
```

MODULAR SOLN. N FASTER DUE TO STRING BUILDER

```
class Solution {
    public String shortestCommonSupersequence(String str1, String str2) {
        int length1 = str1.length();
        int length2 = str2.length();
        int [][] dp = new int [length1 + 1][length2 + 1];
        longestCommonSubsequence(str1, str2, dp, length1, length2);
        return buildTheResultString(str1, str2, length1, length2, dp);
    }

    // the code for the longest common subsequence
    public void longestCommonSubsequence(String s1, String s2, int [][] dp,
       int length1, int length2) {
        for (int i=1; i<=length1; i++) {
            for (int j=1; j<=length2; j++) {
                if (s1.charAt(i - 1) == s2.charAt(j - 1))
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    // building the final result
    public String buildTheResultString(String s1, String s2, int length1, int
length2, int [][] dp) {
        StringBuilder sb = new StringBuilder();
        int i = length1 - 1, j = length2 - 1;
        while (i >= 0 || j >= 0) {
            if (i < 0 || j < 0) {
                char c = i < 0 ? s2.charAt(j--) : s1.charAt(i--);
                sb.append(c);
            }
            else if (s1.charAt(i) == s2.charAt(j)) {
                sb.append(s1.charAt(i));
                i -= 1;
                j -= 1;
            }
            else {
                char c = dp[i][j+1] > dp[i+1][j] ? s1.charAt(i--) : s2.charAt(j--);
                sb.append(c);
            }
        }
        return sb.reverse().toString(); }}
```

9. Edit Distance

Given two strings word1 and word2, return *the minimum number of operations required to convert word1 to word2*. You have the following three operations permitted on a word:

i. Insert a character ii. Delete a character iii. Replace a character

Example 1: **Input:** word1 = "horse", word2 = "ros" **Output:** 3

```
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2: **Input:** word1 = "intention", word2 = "execution" **Output:** 5

Explanation:

```
intention -> inention (remove 't')      inention -> enention (replace 'i' with 'e')

enention -> exention (replace 'n' with 'x')

exention -> exection (replace 'n' with 'c')   exection -> execution (insert 'u')
```

```
class Solution {
    int [][] dp;
    public int minDistance(String word1, String word2) {
        dp = new int[word1.length() + 1][word2.length() + 1];
        return lcs(word1, word2, word1.length(), word2.length()) ;
    }

    private int lcs(String s1, String s2, int n, int m){
        if (n == 0 && m == 0) return 0;
        if (n == 0) return m;
        if (m == 0) return n;

        if (dp[n][m] != 0) return dp[n][m];

        if (s1.charAt(n - 1) == s2.charAt(m - 1))
            dp[n][m] =  lcs(s1, s2, n - 1, m - 1);
        else
            dp[n][m] = 1 + Math.min(Math.min(lcs(s1, s2, n, m - 1),
                                    lcs(s1, s2, n - 1, m) ),
                                lcs(s1, s2, n - 1, m - 1));

        return dp[n][m]; }}
```

optimization from ON^2) to O(N)

initial version

```
   public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) dp[i][0] = i;
        for (int j = 1; j <= n; j++) dp[0][j] = j;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                    Math.min(dp[i - 1][j], dp[i][j - 1]));
                }
            }
        }

        return dp[m][n];
    }
```

optimization of space:

```
   public int minDistance1(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[] dp = new int[n + 1];
        for (int j = 1; j <= n; j++) dp[j] = j;
        for (int i = 1; i <= m; i++) {
            int up  = dp[0];
// the value in last loop, in 2d solution, it means dp[i-1][j-1]
            dp[0] = i; // initial state
            for (int j = 1; j <= n; j++) {
                int tmp = dp[j];
 // dp[j] will be replaced, which is the state of dp[i-1][j-1] in next loop
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[j] = up;
                } else {
                    dp[j] = 1 + Math.min(dp[j], Math.min(up, dp[j - 1]));
                }
                up = tmp;
            }
        }
        return dp[n];
    }
```

OTHER METHOD

Let following be the function definition :-

f(i, j) := minimum cost (or steps) required to convert first i characters of word1 to first j characters of word2

Case 1: word1[i] == word2[j], i.e. the ith the jth character matches. f(i, j) = f(i - 1, j - 1)
Case 2: word1[i] != word2[j], then we must either insert, delete or replace, whichever is cheaper
f(i, j) = 1 + min { f(i, j - 1), f(i - 1, j), f(i - 1, j - 1) }

f(i, j - 1) represents insert operation
f(i - 1, j) represents delete operation
f(i - 1, j - 1) represents replace operation

Here, we consider any operation from word1 to word2. It means, when we say insert operation, we insert a new character after word1 that matches the jth character of word2. So, now have to match i characters of word1 to j - 1 characters of word2. Same goes for other 2 operations as well.
Note that the problem is symmetric. The insert operation in one direction (i.e. from word1 to word2) is same as delete operation in other. So, we could choose any direction.
Above equations become the recursive definitions for DP.
Base Case: f(0, k) = f(k, 0) = k
Below is the direct bottom-up translation of this recurrence relation. It is only important to take care of 0-based index with actual code :-

```
public class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();

        int[][] cost = new int[m + 1][n + 1];
        for(int i = 0; i <= m; i++)
            cost[i][0] = i;
        for(int i = 1; i <= n; i++)
            cost[0][i] = i;

        for(int i = 0; i < m; i++) {
            for(int j = 0; j < n; j++) {
                if(word1.charAt(i) == word2.charAt(j))
                    cost[i + 1][j + 1] = cost[i][j];
                else {
                    int a = cost[i][j];
                    int b = cost[i][j + 1];
                    int c = cost[i + 1][j];
                    cost[i + 1][j + 1] = a < b ? (a < c ? a : c) : (b < c ? b : c);
                    cost[i + 1][j + 1]++;
                }
            }
        }
        return cost[m][n];
    }
}
```

10. Subset Sum

Gen a non-empty array `nums` containing only positive integers, find if the array can be partitioned into two subsets such that the sum of elements in both subsets is equal.

Example 1:

```
Input: nums = [1,5,11,5] Output: true
Explanation: The array can be partitioned as [1, 5, 5] and [11].
```

Example 2:

```
Input: nums = [1,2,3,5] Output: false
Explanation: The array cannot be partitioned into equal sum subsets.
```

```
MY SOLN.
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0;
        for (int n : nums){  sum += n; }
        if (sum%2 != 0) return false;
        return isSubsetSum(nums, sum/2, nums.length);
    }
    private boolean isSubsetSum(int[] nums,int sum,int n){
        boolean[][] dp = new boolean[n+1][sum+1];
        for (int i = 0; i < dp.length; i++)  Arrays.fill(dp[i], false);
        dp[0][0] = true;
        for (int i = 1; i < n+1; i++) dp[i][0] = true;
        for (int j = 1; j < sum+1; j++)   dp[0][j] = false;
        for (int i = 1; i < n+1; i++)
            for (int j = 1; j < sum+1; j++) {
                dp[i][j] = dp[i-1][j];
                if (j >= nums[i-1]) dp[i][j] = (dp[i][j] || dp[i-1][j-nums[i-1]]);
            }
        return dp[n][sum]; } }
```

Intuition: So Lets start step by step and concentrate on the process:

1. Since we have to make two subset both having equal sum then our first condition is to check whether the sum of a given array can be divided in two equal parts which is if total sum is odd then partition is not possible at all and if sum is even then there is chance.

   For example:

```
1.) arr1-> [1,5,11,5]   and     2.) arr2 -> [1,5,3,11]
```

```
      Both arr1 and arr2 has even sum but 1st can be partitioned into ([1,5,5] &
[11]) and 2nd can not.
```

2.) Now let's try to visualize it as 0/1 Knapsack Problem:

- Since in 0/1 knapsack we have 2 choices for each object having value v whether to keep it or not in your knapsack having a certain weight W.
- Same as in this case we have n elements in array and we have two choices to make whether to keep it in subset1 or subset2 (inclusion in one is direct exclusion in other) and weight of knapsack/subset will be sum/2.

3.) So now what our target remains is we have to take care about only one subset because if one subset with weight sum/2 is possible then other subset will surely have the weight sum/2.

4.) So now using subset sum problem code we have to just check if its possible to have a subset having sum = totalSum/2;

Approach1: Memoization (TLE)

```
class Solution {
    boolean mem[][];
    public boolean canPartition(int[] nums) {
        int sum = 0, n = nums.length;
        for(int i : nums) sum+=i;
        if(sum%2!=0) return false;
        sum /= 2;
        mem = new boolean[n+1][sum+1];
        return subsetSum(nums,0,sum);
    }
    boolean subsetSum(int[] nums, int pos, int sum){
        if(sum==0) return true;
        else if(pos>=nums.length || sum<0) return false;

        if(mem[pos][sum]) return true;
        return mem[pos][sum] = subsetSum(nums,pos+1,sum-nums[pos]) ||
                               subsetSum(nums,pos+1,sum);
    }}
```

**A little update: After reading comments on my post I analyze my memoization code that why its giving TLE so I figured that in subset function if (mem[pos][sum]==true) then I am returning true but if its false then it will keep on computing but the problem was that boolean array automatically initialized array with false..so i can't differentiate... So for that we need to take an object boolean array so that it initializes with null or assign null value manually to the boolean array.

Updated Memoization Code: (100% Faster)

```
class Solution {
```

```
    Boolean mem[][];
    public boolean canPartition(int[] nums) {
        int sum = 0, n = nums.length;
        for (int i : nums) sum+=i;
        if (sum%2!=0) return false;
        sum /= 2;
        mem = new Boolean[n+1][sum+1];
        return subsetSum(nums,0,sum);
        }
        boolean subsetSum(int[] nums, int pos, int sum){
        if(sum==0) return true;
        else if(pos>=nums.length || sum<0) return false;
        if(mem[pos][sum]!=null) return mem[pos][sum];
        return mem[pos][sum] = subsetSum(nums,pos+1,sum-nums[pos]) ||
                                subsetSum(nums,pos+1,sum);
 }}
```

Approach2: Dynamic Programming

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0, n = nums.length;
        for(int i : nums) sum+=i;
        if(sum%2!=0) return false;

        sum /= 2;
        boolean dp[][] = new boolean[n+1][sum+1];

        for(int i=0;i<=n;i++){
            for(int j=0;j<=sum;j++){
                if(i==0 || j==0)
                    dp[i][j] = false;
                else if(nums[i-1] > j)
// if curr sum value is greater than the current element value then just skip(take previous value)
                    dp[i][j] = dp[i-1][j];
                else if(nums[i-1]==j)  // we got required sum
                    dp[i][j] = true;
                else
                    dp[i][j] = dp[i-1][j] || dp[i-1][j-nums[i-1]];
            }
        }
        return dp[n][sum];    }}
```

Approach3: Space Optimized DP

Explanation: Idea behind reverse loop for sum

- If you observe the 2d DP then dp[i][j] is the value either from dp[i-1][j] or dp[i-1][j-nums[i-1]] and when we converted 2d DP into 1d DP then the thinking was that, dp[i] will have values

for one iteration and then again for next iteration we will use that stored values which will act as dp[i-1] for this iteration.

- So now we need value of dp[i-num] that is value from previous index so here is the problem in left to right loop that each time when we enter into new iteration we need value from previous iteration but the value will already be updated in this iteration and we will loss the previous value.
- That's why using a reverse loop.

Example: Lets in first iteration dp array is filled as [2,6,1,8,5]
So now we started from left and updated 2 as 4 and 6 as 7 and 1 as 5([4,7,5,8,5]) and now we reach to 8 and we need dp[1] value but we need the value from previous iteration which we have lost and hence will get the wrong answer.
So move from right to left and use the previous iteration value.(these are pure random values used in example, so don't think about them just get the example)

```
class Solution {
    public boolean canPartition(int[] nums) {
        int sum = 0, n = nums.length;
        for(int i : nums) sum+=i;
        if(sum%2!=0) return false;
        sum /= 2;
        boolean[] dp = new boolean[sum+1];
        dp[0] = true;
        for (int j : nums) {
            for (int i = sum; i > 0; i--) {
                if (i >= j) {
                    dp[i] = dp[i] || dp[i-j];
                }
            }
        }
        return dp[sum];}}
```

11. Target Sum

You are given an integer array `nums` and an integer `target`.
You want to build an expression out of nums by adding one of the symbols `'+'` and `'-'` before each integer in nums and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a `'+'` before `2` and a `'-'` before `1` and concatenate them to build the expression `"+2-1"`.

Return the number of different expressions that you can build, which evaluates to `target`.

Example 1: `Input: nums = [1,1,1,1,1], target = 3  Output: 5`

Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3

Example 2: **Input**: nums = [1], target = 1 **Output**: 1

Method - 1

```
class Solution {

    public int findTargetSumWays(int[] nums, int target) {
        return target(nums, target, 0, 0);
    }

    private int target(int[] nums, int target, int i, int sum){
        if (i == nums.length){
            if (target == sum) return 1;
            return 0;
        }

         int pos = target(nums, target, i + 1, sum + nums[i]);
         int neg = target(nums, target, i + 1, sum - nums[i]);

        return pos + neg; }}
```

Method - 2 adi way

```
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        //Solution 1
        int sum = 0;
        for(int x : nums) sum += x;
        if(((sum - target) % 2 == 1) || (target > sum))
            return 0;

        int n = nums.length, s2 = (sum - target)/2;
        int[][] t = new int[n + 1][s2 + 1];
        t[0][0] = 1;

        for(int i = 1; i < n + 1; i++) {
            for(int j = 0; j < s2 + 1; j++) {
                if(nums[i - 1] <= j)
                    t[i][j] = t[i-1][j] + t[i - 1][j - nums[i - 1]];
                else
                    t[i][j] = t[i - 1][j];
            }
        }
        return t[n][s2];
}
```

Method - 3 hashmap

```
class Solution {
    Map<String, Integer> map = new HashMap<>();
    public int findTargetSumWays(int[] nums, int S) {
        return helper(nums, S, 0, 0);
    }
    private int helper(int []nums, int targetSum, int index, int sum){
        //Base condition when sum == targetSum. we return 1
        //as we need to find the number of ways we can reach targetSum
        if(index == nums.length && sum == targetSum){
            return 1;
        }
        //Base condition when sum != targetSum . we return 0 from here
        //as we do not want this case in final answer
        if(index == nums.length && sum != targetSum){
            return 0;
        }
        //memoization using map.
        //used immutability property of String for unique key in map.
        String key = sum + "()" + index;
        if(map.containsKey(key)){
            return map.get(key);
        }

        int count = 0;
        //first case we take negative value of current index
        count += helper(nums, targetSum, index + 1, sum - nums[index]);
        //second case we take positive value of current index
        count += helper(nums, targetSum , index + 1, sum + nums[index]);

        //add to map for future use.
        map.put(key, count);

        return count;
    } }
Using dfs


class Solution {
    int count=0;
    public int findTargetSumWays(int[] nums, int target) {
        if(nums==null || nums.length==0) return 0;
        dfs(nums,target,0,0);
        return count;
    }

    public void dfs(int nums[],int target,int index,int currSum){
        if(index==nums.length){
            if(currSum==target) count++;
            return;
        }
        dfs(nums,target,index+1,currSum+nums[index]);
        dfs(nums,target,index+1,currSum-nums[index]); }}
```

12. Minimum Swaps To Make Sequences Increasing

You are given two integer arrays of the same length nums1 and nums2. In one operation, you are allowed to swap nums1[i] with nums2[i].

- For example, if nums1 = [1,2,3,8], and nums2 = [5,6,7,4], you can swap the element at i = 3 to obtain nums1 = [1,2,3,4] and nums2 = [5,6,7,8].

Return *the minimum number of needed operations to make* nums1 *and* nums2 *strictly increasing*.
The test cases are generated so that the given input always makes it possible.
An array arr is strictly increasing if and only if arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1].

Example 1:

```
Input: nums1 = [1,3,5,4], nums2 = [1,2,3,7]  Output: 1
Explanation:
Swap nums1[3] and nums2[3]. Then the sequences are:
nums1 = [1, 3, 5, 7] and nums2 = [1, 2, 3, 4]
which are both strictly increasing.
```

Example 2:

```
Input: nums1 = [0,3,5,8,9], nums2 = [2,1,4,6,9] Output: 1
```

Approach

There are two states possible,

1. Don't swap elements at the current index
2. Swap elements at the current index

We just have to find out which one gives the minimum number of swaps for the rest of the array. That is, we will compute answer for both the states. The answer for the current state is dependent on the relation between the element at the current index and the previous index.
If they are already in increasing order, then the state for the current index is applied to the previous index (that is, no swap remains no swap, swap remains swap). Else, the state for the current index is reversed for the previous index. But, what if swap and no swap both achieve the increasing order? In this case, we take the minimum of both states from the previous index.
Implementations

N = size of nums1/nums2 array

1. Recursion

Time complexity: O(2^N) **Space complexity:** O(N)

```
class Solution {
    public int minSwap(int[] nums1, int[] nums2) {
        return Math.min(recurse(nums1, nums2, nums1.length - 1, 0),
                        recurse(nums1, nums2, nums1.length - 1, 1));
    }
    private int recurse(int[] nums1, int[] nums2, int i, int swap) {
        // base case
        if (i == 0) return swap;
        // default is set as max
        int res = Integer.MAX_VALUE;
        // if array is increasing without swapping
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1])
                res = recurse(nums1, nums2, i - 1, swap);
        // if array is increasing with swapping
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1])
                res = Math.min(res,
                                recurse(nums1, nums2, i - 1, 1 - swap));
        return swap == 0 ? res : res + 1;
    }
}
```

2. Memoization

Time complexity: O(N) **Space complexity:** O(N)

```
class Solution {
    public int minSwap(int[] nums1, int[] nums2) {
        int n = nums1.length;
        // initialize dp table
        int[][] memo = new int[2][n];
        Arrays.fill(memo[0], -1); Arrays.fill(memo[1], -1);
        memo[0][0] = 0; memo[1][0] = 1;
        return Math.min(recurse(nums1, nums2, n - 1, 0, memo),
                        recurse(nums1, nums2, n - 1, 1, memo));
    }
    private int recurse(int[] nums1, int[] nums2, int i, int swap, int[][] memo) {
        //check dp table
        if (memo[swap][i] != -1)  return memo[swap][i];
        // initial value is set as max
        int res = Integer.MAX_VALUE;
        // if array is increasing without swapping
        if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1])
                res = recurse(nums1, nums2, i - 1, swap, memo);

        // if array is increasing with swapping
        if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1])
                res = Math.min(res,
                                recurse(nums1, nums2, i - 1, 1 - swap, memo));

        memo[swap][i] = swap == 0 ? res : res + 1;
        return memo[swap][i];
```

```
    }
}
```

3. Tabulation

Time complexity: O(N) **Space complexity:** O(N)

```
class Solution {
    public int minSwap(int[] nums1, int[] nums2) {
        int n = nums1.length;
        // initialize dp table
        int[][] table = new int[2][n];
        table[0][0] = 0; table[1][0] = 1;
        for (int i = 1; i < n; i++) {
            // initial value
            table[0][i] = Integer.MAX_VALUE;
            table[1][i] = Integer.MAX_VALUE;
            // if array is increasing without swapping
            if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
                table[0][i] = table[0][i - 1];
                table[1][i] = 1 + table[1][i - 1];
            }
             // if array is increasing with swapping
            if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
                table[0][i] = Math.min(table[0][i], table[1][i - 1]);
                table[1][i] = Math.min(table[1][i], 1 + table[0][i - 1]);
            }
        }
        return Math.min(table[0][n - 1], table[1][n - 1]);
    } }
```

4. Tabulation with Constant Space → **Time complexity:** O(N) **Space complexity:** O(1)

```
class Solution {
    public int minSwap(int[] nums1, int[] nums2) {
        // initialize current res
         int curNoSwap = 0, curSwap = 1, n = nums1.length;
         for (int i = 1; i < n; i++) {
            // update previous values
            int prevNoSwap = curNoSwap, prevSwap = curSwap;
            // reset cur values
            curNoSwap = Integer.MAX_VALUE;
            curSwap = Integer.MAX_VALUE;
           // if array is increasing without swapping
            if (nums1[i] > nums1[i - 1] && nums2[i] > nums2[i - 1]) {
                curNoSwap = prevNoSwap;
                curSwap = 1 + prevSwap;
            }
            // if array is increasing with swapping
            if (nums1[i] > nums2[i - 1] && nums2[i] > nums1[i - 1]) {
                curNoSwap = Math.min(curNoSwap, prevSwap);
                curSwap = Math.min(curSwap, 1 + prevNoSwap);
            }
        }

        return Math.min(curNoSwap, curSwap);
```

```
    }}
```

13. Best Team With No Conflicts

You are the manager of a basketball team. For the upcoming tournament, you want to choose the team with the highest overall score. The score of the team is the sum of scores of all the players in the team.

However, the basketball team is not allowed to have conflicts. A conflict exists if a younger player has a strictly higher score than an older player. A conflict does not occur between players of the same age.

Given two lists, `scores` and `ages`, where each `scores[i]` and `ages[i]` represents the score and age of the `ith` player, respectively, return *the highest overall score of all possible basketball teams*.

Example 1:

```
Input: scores = [1,3,5,10,15], ages = [1,2,3,4,5] Output: 34
Explanation: You can choose all the players.
```

Example 2:

```
Input: scores = [4,5,6,5], ages = [2,1,2,1] Output: 16
Explanation: It is best to choose the last 3 players. Notice that you are allowed to
choose multiple people of the same age.
```

Example 3:

```
Input: scores = [1,2,3,5], ages = [8,9,10,1] Output: 6
Explanation: It is best to choose the first 3 players.
```

Brute force approach

```
class Solution {
   class player {
        int score, age;
        player(int score, int age){
            this.score = score;
            this.age = age;
        } }
    public int bestTeamScore(int[] scores, int[] ages) {
        int n = ages.length;
        player[] p = new player[n];
        for (int i = 0; i < n; i++) {
            p[i] = new player(scores[i], ages[i]);
        }
         Arrays.sort(p, (a, b) -> (a.score != b.score) ? (a.score - b.score) :
                                  (a.age - b.age));
        return find(p, 0, 0, n);
    }

    public int find(player[] p, int ind, int prev, int n) {
        if (ind >= n) return 0;

        if (p[ind].age >= prev) {
            int x1 = p[ind].score + find(p, ind + 1, p[ind].age, n);
```

```
            int x2 = find(p, ind + 1, prev, n);
            return Math.max(x1, x2);
        } else {
            return find(p, ind + 1, prev, n);
        }
    }}
```

Dynamic programming knapsack approach.

```
class Solution {
   class player {
        int score, age;
        player(int score, int age){
            this.score = score;
            this.age = age;
        }
    }
    public int bestTeamScore(int[] scores, int[] ages) {
        int n = ages.length;
        player[] p = new player[n];
        int[][] dp = new int[1001][1001];
        for (int i = 0; i < n; i++) {
            p[i] = new player(scores[i], ages[i]);
        }
        for (int i = 0; i <= 1000; i++){
            Arrays.fill(dp[i], -1);
        }

        Arrays.sort(p, (a, b) -> (a.score != b.score) ? (a.score - b.score) :
                                  (a.age - b.age));
        return solveKNP(p, 0, 0, dp);
    }

    public int solveKNP(player[] p, int ind, int prev, int[][] dp) {
        if (ind >= p.length) return 0;
        if (dp[ind][prev] != -1){
            return dp[ind][prev];
        }
        if (p[ind].age >= prev) {
            int x1 = p[ind].score + solveKNP(p, ind + 1, p[ind].age, dp);
            int x2 = solveKNP(p, ind + 1, prev, dp);
            return dp[ind][prev] = Math.max(x1, x2);
        } else {
            return dp[ind][prev] = solveKNP(p, ind + 1, prev, dp);
        }
    } }
```

14. Jump Game I, II, III, IV, V

Jump game I:

You are given an integer array nums. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return true *if you can reach the last index, or* false *otherwise*.

Example 1: Input: nums = [2,3,1,1,4] **Output**: true
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
Example 2: Input: nums = [3,2,1,0,4] **Output**: false
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

INTUITION: What does the question ask for? → The question asks us to determine whether the end index is reachable from index 0.
How may we approach this problem? → The basic idea for this problem is to check the reachability of intermediate indexes to determine the reachability of some index
Approach #1 : Recursion + Visited
Write another method wherein we check whether the end index is reachable from the current index
If we can reach to the current index from 0 and then from current index to end index,
then we can surely reach from 0 to end index (0-> current index -> end index)
What is the purpose of visited?
While calculating the reach. we may end up at the same index multiple times. If that index has been processed earlier -> Either it would have returned true, in which case, no more calculations would have been done, but since we encountered this index again, means that its result was false.

```
class Solution {
    boolean[] visited;
    public boolean canJump(int[] nums) {
        int len = nums.length;
        visited = new boolean[len];
        return canJump(0,len, nums, visited);
    }
    private boolean canJump(int ind, int len, int[] nums, boolean[] visited){
        if(ind>=len-1){
            return true;
        }
        if(visited[ind]){
            return false;
        }
        visited[ind]=true;
        int maxJumps = nums[ind];
        if(maxJumps==0){
            return false;
```

```
        }
        for(int i=maxJumps;i>=1;i--){
            if(canJump(ind+i, len, nums, visited)){
                return true;
            }
        }
        return false;
    }}
```

Approach #2 : Bottom-up DP/Tabulation

How else can we solve the problem → If the problem is solvable recursively, there will be an iterative solution as well.

The idea for this approach is → Maintain an array to check whether the current index is reachable from zero

How will we update the reachable array? → We will use the idea of intermediate indexes:

Say we want to update index i. We will consider some intermediate index j, s.t., 0<=j<i

Then: **a)** We will check that j is reachable from zero (using our reachable array)

b) We will check that i is reachable from j (Maximum index reachable from j = j+nums[j], if i<=max_index, then i is reachable otherwise not)

```
class Solution {
    public boolean canJump(int[] nums) {
        int len = nums.length;
        if(len==1) return true;

        boolean[] isReachableFromZero = new boolean[len];
        isReachableFromZero[0] = true;
        for (int i = 1; i < len; i++){
            for (int j = i - 1; j >= 0; j--){
                // If j is reachable from zero
                // and from j, we can reach i
                // Means we can reach i from Zero
                if (isReachableFromZero[j]&&j+nums[j]>=i){
                    isReachableFromZero[i]=true;
                    break;
                }
            }
        }
        return isReachableFromZero[len-1];
    } }
```

Approach #3: No DP

What can we improve in the previous approach?

What if instead of maintaining an array to check the reachability, we maintain a single variable that determines the maximum reachable index from 0 at a given iteration? In that case, we can simply check if the current index is within reach or out of reach, all the while updating max reach if current index is within reach.

```
class Solution {
    public boolean canJump(int[] nums) {
        int len = nums.length;
        if(len==1) return true;
        int maxReachFromZero = 0;
        for(int i = 0; i < len; i++){
            if(i<=maxReachFromZero){
                maxReachFromZero = Math.max(maxReachFromZero,i+nums[i]);
```

```
            } else{
                return false;
            }
            if(maxReachFromZero>=len-1){
                break;
            }
        }
        return true;
    }}
```

Method - 4 OTHER Ladder method

```
class Solution {
    public boolean canJump(int[] nums) {
        if (nums.length == 1) return true;

        int ladder = nums[0], step = nums[0];
        if (ladder == 0) return false;
        for (int i = 1; i < nums.length - 1; i++){
            ladder = Math.max(ladder, i + nums[i]);
            step--;
            if (step <= 0) {
                if ( i >= ladder) return false;
                step = ladder - i;
            }
        }
        return true;
    }}
```

Jump game II: (segment tree also could be used it seems (rachit jain))

Given an array of non-negative integers nums, you are initially positioned at the first index of the array.
Each element in the array represents your maximum jump length at that position.
Your goal is to reach the last index in the minimum number of jumps.
You can assume that you can always reach the last index.

Example 1: `Input: nums = [2,3,1,1,4] Output: 2`

`Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.`

Example 2: `Input: nums = [2,3,0,1,4]Output: 2`

INTUITION:

In the question it is mentioned that the answer will always exist.the last element of array does not matter also as even if its value is 0 we don't need to hop further.

Brute Force-start from i=0 to n-2 and foreach element i check at how many positions we can go and for each valid position again check how many locations we can go. keep on returning min for each path. Once we reach the last index we have the desired answer.

```
public int bruteForce(int[] nums) {
		return bruteForceRec(nums, 0);
	}

	private int bruteForceRec(int[] nums, int start) {
		if (start >= nums.length - 1) {
			return 0;
		}
		int min = 10001;//max value can be 1000
		for (int i = start + 1; i <= nums[start] + start; i++) {
			min = Math.min(min, 1 + bruteForceRec(nums, i));
		}
		return min;
	}
```

Top Down -In above approach we are visiting every branch and at most of the times the same branch is re-calculated. to save that take output array and before calling the method check if that index output array already has some data. if yes return directly else call method.

```
	public int topDownDp(int[] nums) {
		return topDownRec(nums, 0, new int[nums.length]);
	}
	private int topDownRec(int[] nums, int start, int[] output) {
		if (start >= nums.length - 1) {
			return 0;
		}
		if (output[start] != 0) {
			return output[start];
		}
		int min = 10001; // max possible jump is 10^4
		for (int i = start + 1; i <= nums[start] + start; i++) {
			min = Math.min(min, 1 + topDownRec(nums, i, output));
		}
		output[start] = min;
		return min;
	}
```

Bottom Up → we can just reverse the outer loop and can make a recursive solution iterative.

```
	public int bottomUpDp(int[] nums) {
		int n = nums.length;
		int[] output = new int[n];
```

```
            for (int start = n - 2; start >= 0; start--) {
                  int min = 10001;
                  for (int i = start + 1; i <= start + nums[start]; i++) {
                        min = Math.min(min, 1 + (i>=n ? 0 : output[i]));
                  }
                  output[start] = min;
            }
            return output[0];
      }
```

Greedy Approach → we can make use of greedy idea + sliding window concept also and can reduce the complexity from o(n^2) to o(n)

for that keep maintaining left and right pointers to hold the window size on which we can go from each step.
jump will store the total jump needed to reach from the first window till the last window.
initially set l=r=jump=0 now loop till right< n-1
check for item starting from left till right what is max value i.e.
max=Math.max(max,i+nums[i]);
Once the inner loop finishes we have max hops that we can go. now, increment jump, and left and right boundary also. Once the outer loop finishes, jump has the desired answer.

```
public int greedy(int[] nums) {
            int jump = 0, left = 0,right = 0;
            //till we reach end of the array
            while (right < nums.length - 1) {
                  int max = 0;
                  //find max reach (in range of indexes left and right)
                  for (int i = left; i <= right; i++) {
                        max = Math.max(nums[i] + i, max);
                  }
                  //update new boundaries
                  left = right + 1;
                  right = max;
                  jump++;
            }
            return jump;
      }
```

Method - 4 OTHER Ladder method

```
class Solution {
    public int jump(int[] nums) {
        if (nums.length <= 1) return 0;
        int step = nums[0], ladder = nums[0], jump = 1;

        for (int i = 1; i < nums.length - 1; i++){
```

```
            ladder = Math.max(ladder, i + nums[i]);
            step--;
            if (step == 0){
                jump++;
                step = ladder - i;
            }
        }
        return jump;
    }
}
```

A. Others

1. Egg Dropping

IV. GRIDS

1. Grid Unique Paths

There is a robot on an `m x n` grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time. Given the two integers `m` and `n`, return *the number of possible unique paths that the robot can take to reach the bottom-right corner*. The test cases are generated so that the answer will be less than or equal to `2 * 109`.

Example 1:

```
Input: m = 3, n = 2
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach
the bottom-right corner:
```

```
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down
```

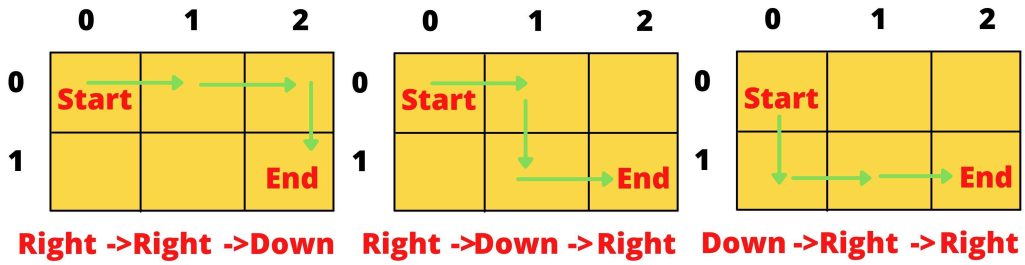
Method - 1 Dynamic Prog. approach Bottom UP

```
class Solution {
    public int uniquePaths(int m, int n) {
        if (m == 1|| n == 1) return 1;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) Arrays.fill(dp[i], 1);
        for (int i = 1; i < m; i++){
            for (int j = 1; j < n; j++){
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        return dp[m - 1][n - 1]; }}
```

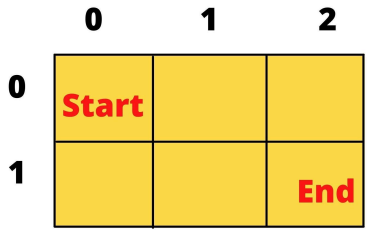
Method - 2 **Combinatorics**

Intuition: If we observe examples there is a similarity in paths from start to end. Each time we are taking an exactly **m+n-2** number of steps to reach the end.



From start to reach the end we need a certain number of rightward directions and a certain number of downward directions. So we can figure out we need **n-1** no. of rightward direction and **m-1** no. of downward direction to reach the endpoint. Since we need an **m+n-2** number of steps to reach the end among those steps if we choose **n-1** rightward direction or **m-1** downward direction and calculate the combinations **(ie: m+n-2Cn-1** or **m+n-2Cm-1**) we'll get the total number of paths.

Approach: The approach of this solution is very simple just use a for loop to calculate the **m+n-2Cn-1** or **m+n-2 Cm-1** **Dry Run**: We'll take the input m = 2 and n = 3



According to our observation, we need an **m+n-2** number of steps to reach the end. So we need **3** steps and in every step, we need an **n-1** number of rightward direction and **m-1** number of downward direction. Among **3** steps if we choose **2** rightward directions then the result will be **3 (3C2)** or among **3** steps if we choose **1** downward direction then the result will also be **3 (3C1)**.

```
class Solution {
    public int uniquePaths(int m, int n) {
        int N = n + m - 2, r = m - 1;
        double res = 1;
        for (int i = 1; i <= r; i++){
            res = res * (N - r + i) / i;
        }

        return (int) res; }}
```

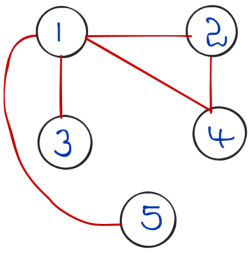
V. GRAPHS

A. DFS & BFS

1. DFS & BFS Implementations

Given a graph, traverse through all the nodes in the graph using **Depth First Search**.

Example:

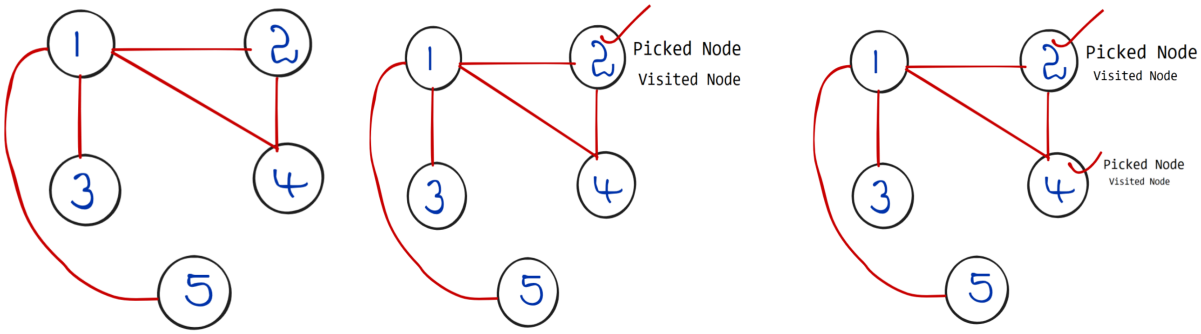


Input: **Output:** 2 4 1 3 5

Explanation: Note: For above I/P we started DFS from Node 2,We can start from any node.

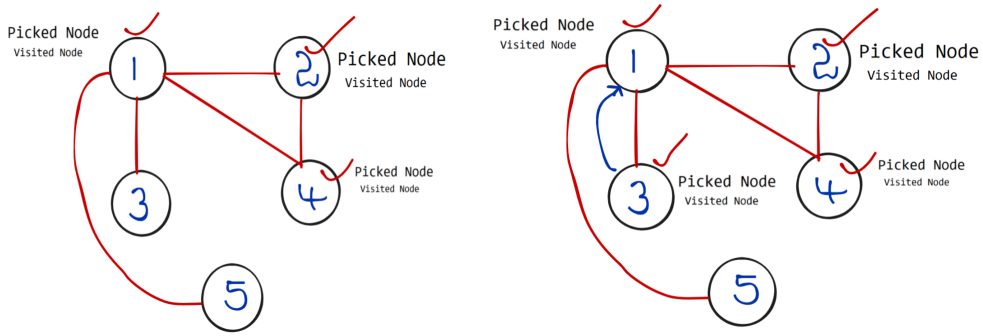
Solution N Intuition:

Depth First Traversal is a traversal technique/algorithm, used to traverse through all the nodes in the given graph.
It starts traversal through any one of its neighbor nodes and explores the farthest possible node in each path/branch and then starts Back-tracking.
Backtracking happens when the DFS algorithm reaches a particular node that has no neighbors to visit further, once it reaches that node with no neighbors to be visited it'll backtrack to its previous node (say N) and from this node N algorithm will start searching for unvisited neighbor nodes and starts visiting them. Let's see how it works pictorially;



Let's pick a random node to start the traversal, let it be 2 (You can choose any node).

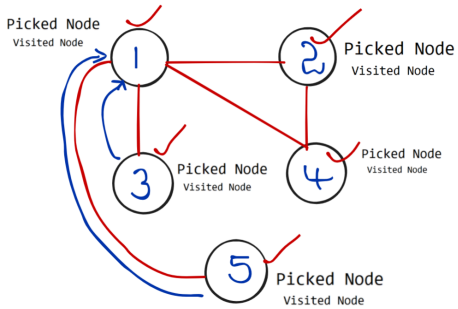
- Now, visit any of its neighbor nodes (neighbors of 2 are 1,4) and pick any of its neighborhood nodes. *Note: Now the Output console has 2 as the answer.*
- Now, let's say the algorithm picks 4.
- *Note : Now the Output console has 2,4 as the answer.*
- Now the algorithm checks for the unvisited neighbors of node 4, node 4 has only one unvisited neighbor node i.e 1 , So 1 will be visited.

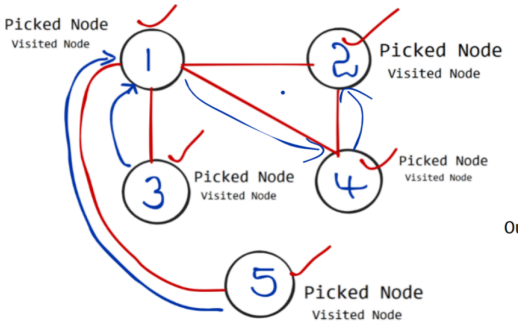


Now, neighbors of 1(3,4) that are not visited will be visited, which means **repetition of the same process which we did for previous nodes.**

- *Note : Now the Output console has 2,4,1 as the answer.*
- Now let's assume the algorithm picks 3, so 3 will be visited.

Once 3 has been visited, there are no neighbor nodes for 3 which are to be visited so it will backtrack to 1.
Note : Now the Output console has 2,4,1,3 as the answer.
Now the algorithm is at node 1, when it checks for unvisited neighbours it will only find one node i.e 5, so it will visit 5 and backtrack to 1 as shown below.



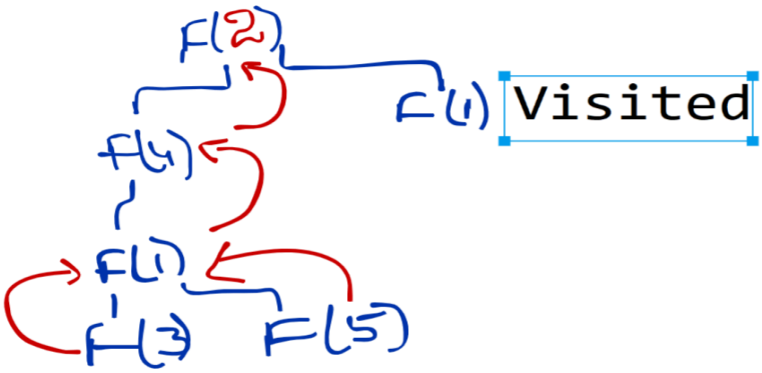



```
Output: 2 4 1 3 5
```

Note: *Now the Output console has 2,4,1,3,5 as the answer, Which is our expected answer!*
When it backtracks to 1, now 1 has to backtrack to its previous node and this backtracking will happen till it reaches the start node.
Approach:*Once you have gone through the above pictorial representation, you might have got an idea of how to approach this problem, if not let's see the approach.*

1. Start with a random node from graph
2. Make an array to keep track of visited nodes, once visited make that node as visited
3. Print this current node
4. Get neighbor nodes and perform above steps recursively for each node deeply/depthly if node is unvisited.

Recursion Tree for more understandability :



Code:

```
class Solution{
    public static void dfs(int node, boolean vis[],
             ArrayList<ArrayList<Integer>> adj, ArrayList<Integer> storeDfs) {
        storeDfs.add(node);
        //marking current node as visited
        vis[node] = true;
        //getting neighbor nodes
        for(Integer it: adj.get(node)) {
            if(vis[it] == false) {
                dfs(it, vis, adj, storeDfs);
```

```
            }}}

    public static ArrayList<Integer> dfsOfGraph(int V,
                                ArrayList<ArrayList<Integer>> adj) {
        ArrayList<Integer> storeDfs = new ArrayList<>();
        //boolean array to keep track of visited vertices
        boolean vis[] = new boolean[V+1];

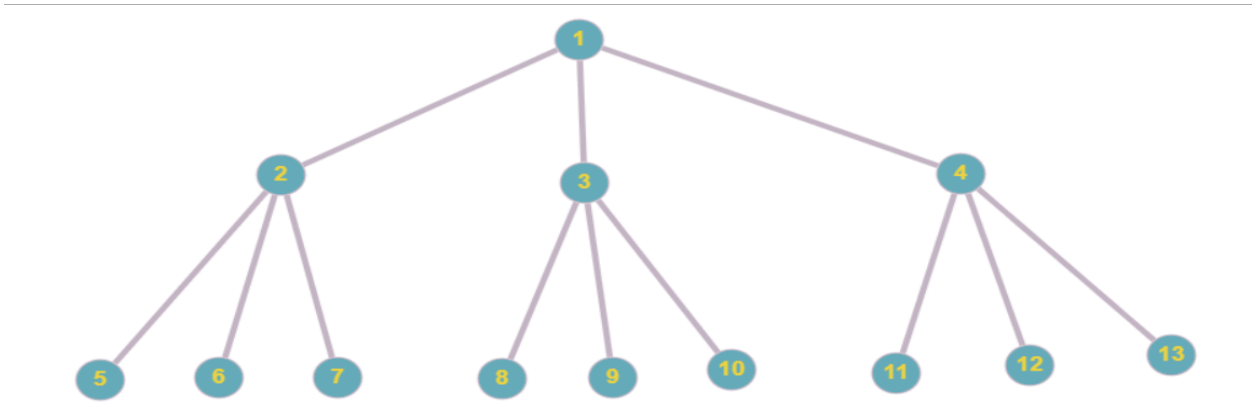
        //If you are starting from node 2, then i should start from 2.
        for(int i = 1;i <= V;i++) {
            if(!vis[i]) dfs(i, vis, adj, storeDfs);
        }

        return storeDfs; }}
```

BFS IMPLEMENTATION:

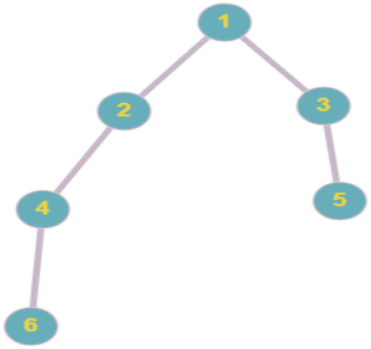
An Undirected Graph will be given. Return a vector of all the nodes of the Graph by Breadth-First Search (BFS) Technique. **Pre-req.** : Queue STL, Introduction to graphs

Example 1: Input:



Output: 1 , 2 , 3 , 4 , 5 , 6 , 7, 8 , 9 , 10, 11, 12, 13

Example 2: Input:



Output: 1 , 2 , 3 ,4 , 5 , 6

Solution N Intuition :

-> We will capture a node. After capturing this node, we will put this node's value into our answer vector and mark its visited value as **true**. Now, this node will be having some adjacent nodes connected to it.

-> We will push all the adjacent nodes which are **NOT VISITED** into the queue data structure.

If we encounter any **VISITED** adjacent nodes will not be considering them because we don't want to consider them twice.

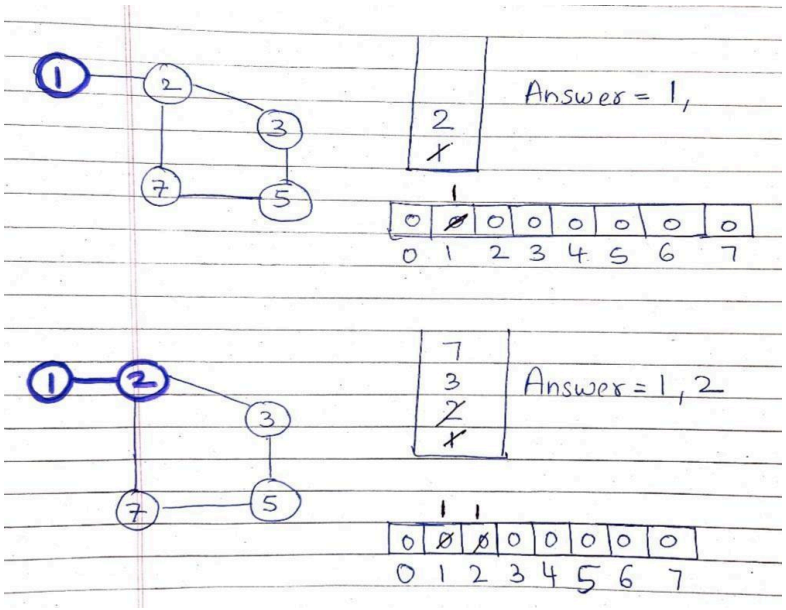
-> By this way, we are covering all the nodes of the component breadth-wise.

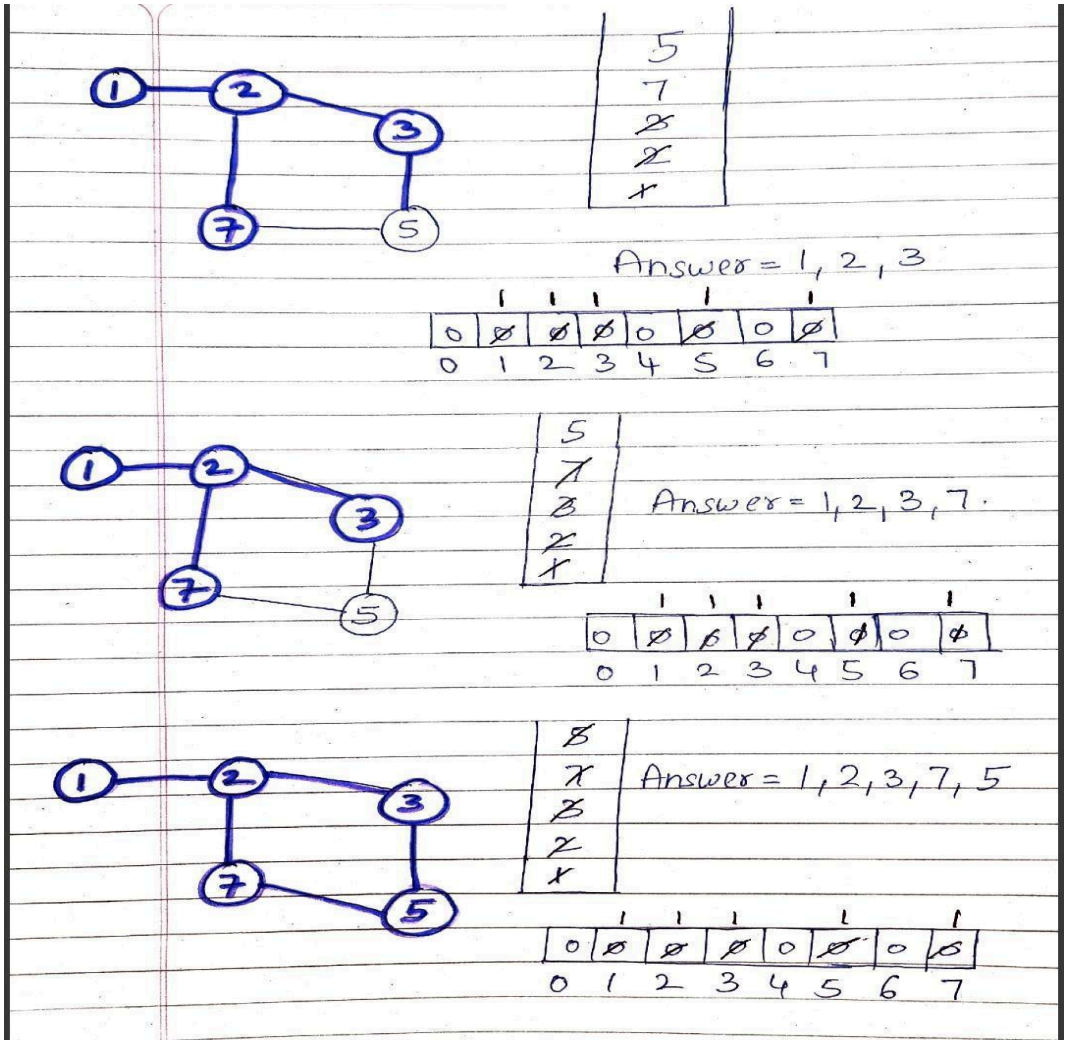
Approach : We need to have this before proceeding :

1. Queue Data structure
2. Visited Array – An array with all values initialized with 0**.**

-> We will push the 1st node into the queue data structure and mark it as visited. After this, we will find its adjacent nodes. If we get some unvisited node, we will simply push this adjacent node into the queue data structure

-> Now since the work of the 1st node is done, we will pop out this node from the queue. Now, this process goes on until the queue is not empty.





Code:

```
class Solution {
    public static ArrayList<Integer> bfsOfGraph(int V,
            ArrayList<ArrayList<Integer>> adj) {

        ArrayList<Integer> bfs = new ArrayList();
        boolean vis[] = new boolean[V];
        Queue<Integer> q = new LinkedList();

        q.add(0);
        vis[0] = true;
        while (!q.isEmpty()) {
            Integer node = q.poll();
            bfs.add(node);

            // Get all adjacent vertices of the dequeued vertex s
            // If a adjacent has not been visited, then mark it
            // visited and enqueue it
```

```
            for (Integer it: adj.get(node)) {
                if (vis[it] == false) {
                    vis[it] = true;
                    q.add(it);
                }
            } }
        return bfs;  } }}
```

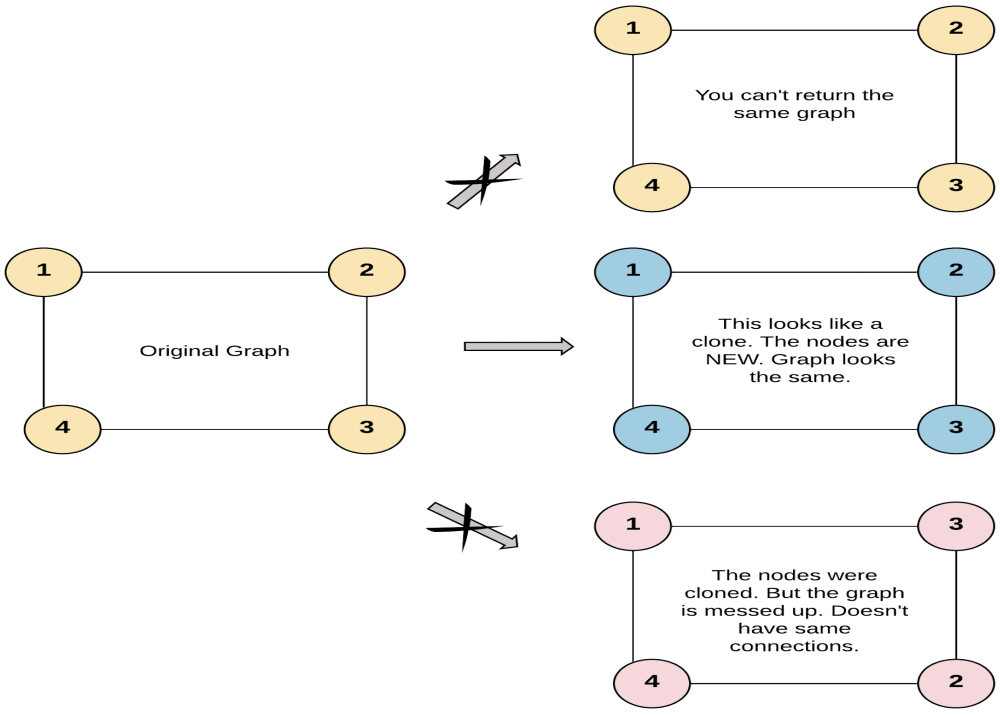
2. Clone a graph

Given a reference of a node in a connected undirected graph. Return a deep copy (clone) of the graph.Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

Test case format: For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with val == 1, the second node with val == 2, and so on. The graph is represented in the test case using an adjacency list. An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with val = 1. You must return the copy of the given node as a reference to the cloned graph. Example:




```
Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
Output: [[2,4],[1,3],[2,4],[1,3]]
Explanation: There are 4 nodes in the graph.
```

```
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
```

METHOD - 1 USING BFS

```
class Solution1 {
    public Node cloneGraph(Node node) {
        if (node == null)  return null;
        HashMap<Node, Node> visited = new HashMap<>();
        Node newNode = new Node(node.val);
        visited.put(node, newNode);
        Queue<Node> queue = new LinkedList<>();
        queue.offer(node);
        while (!queue.isEmpty()) {
            Node cur = queue.poll();
            List<Node> newNeighbors = visited.get(cur).neighbors;
            for (Node n : cur.neighbors) {
                if (!visited.containsKey(n)) {
                    visited.put(n, new Node(n.val));
                    queue.offer(n);
                }
                newNeighbors.add(visited.get(n));
            }
        }

        return newNode; }}
```

METHOD - 2 USING DFS

```
class Solution {
    public void dfs(Node node , Node copy , Node[] visited){
        visited[copy.val] = copy;
                for(Node n : node.neighbors){
            if(visited[n.val] == null){
                Node newNode = new Node(n.val);
                copy.neighbors.add(newNode);
                dfs(n , newNode , visited);
            }else{
                copy.neighbors.add(visited[n.val]);
            }
        }
    }
    public Node cloneGraph(Node node) {
        if(node == null) return null;
        Node copy = new Node(node.val);
        Node[] visited = new Node[101];
// in this question we will create an array of Node(not boolean) why ? , because i
have to add all the adjacent nodes of particular vertex, whether it's visited or not,
so in the Node[] initially null is stored, if that node is visited, we will store the
respective node at the index, and can retrieve that easily.
        Arrays.fill(visited , null);
        dfs(node , copy , visited);
```

```
        return copy;     }}
```

METHOD - 3 RECURSIVE SOLN. USING MAP

```
class Solution {
    HashMap<Integer, Node> map = new HashMap();
    public Node cloneGraph(Node node) {
        return clone(node);
    }
    private Node clone(Node node){
        if (node == null) return null;
        if (map.containsKey(node.val)) return map.get(node.val);

        Node newNode = new Node(node.val, new ArrayList());
        map.put(newNode.val, newNode);
        for (Node child: node.neighbors){
            newNode.neighbors.add(clone(child));
        }
        return newNode;
    }}
```

3. Detect a cycle of Undirected using Bfs & Dfs

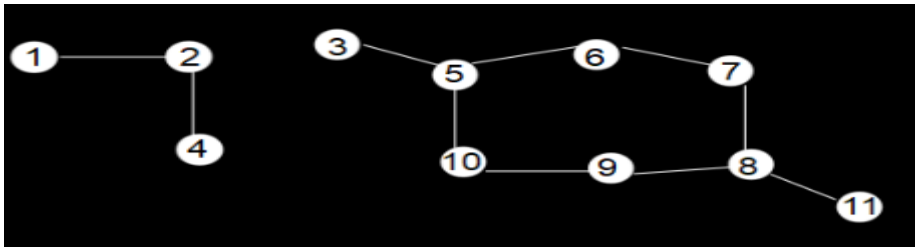
Given an undirected Graph, check for a cycle using BFS (Breadth-First Search) Traversal.

Example:

Input:



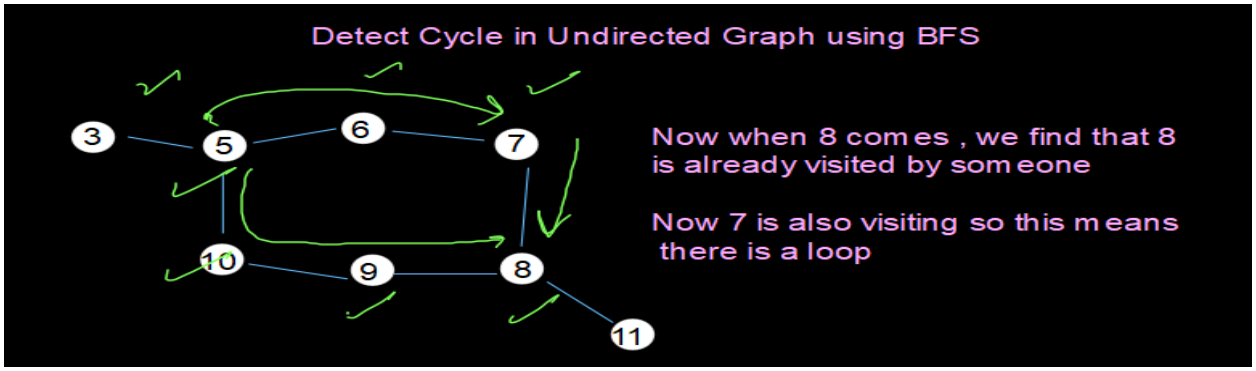
Output: Yes

Explanation: 

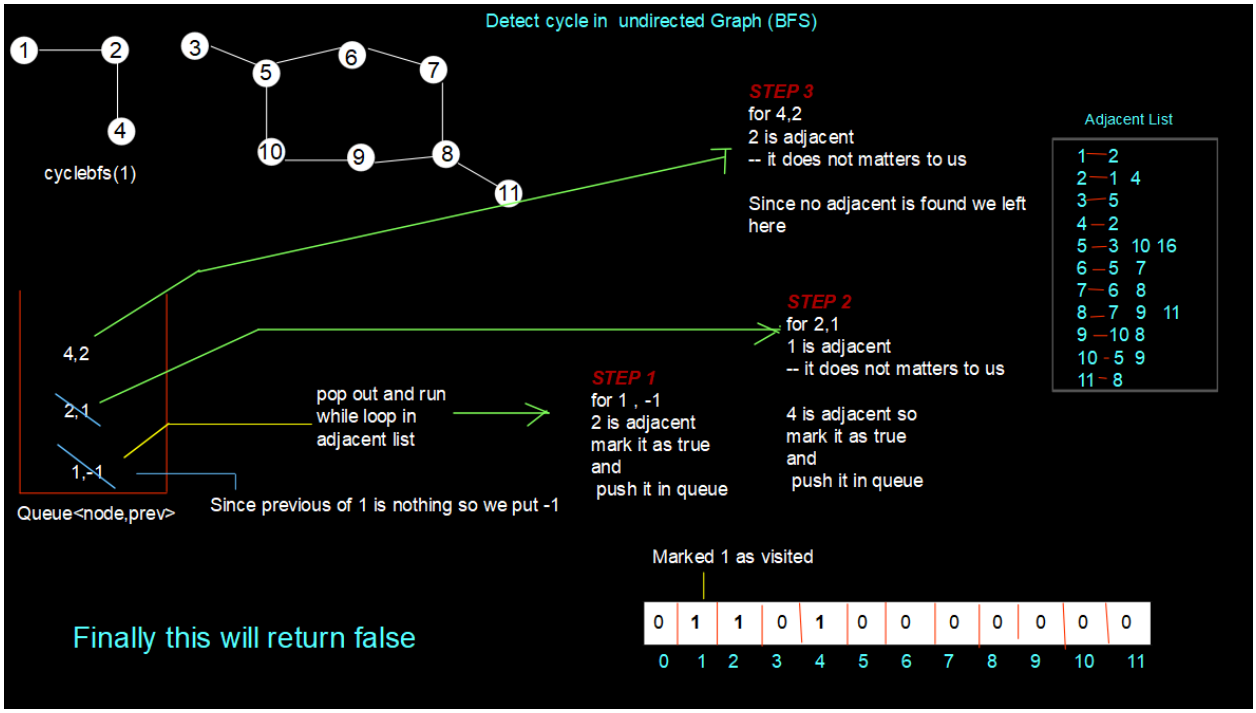
Since 8 is a point where loop is formed

Solution 1: Intuition: The intuition behind this is to check for the visited element if it is found again, this means the cycle is present in the given undirected graph.

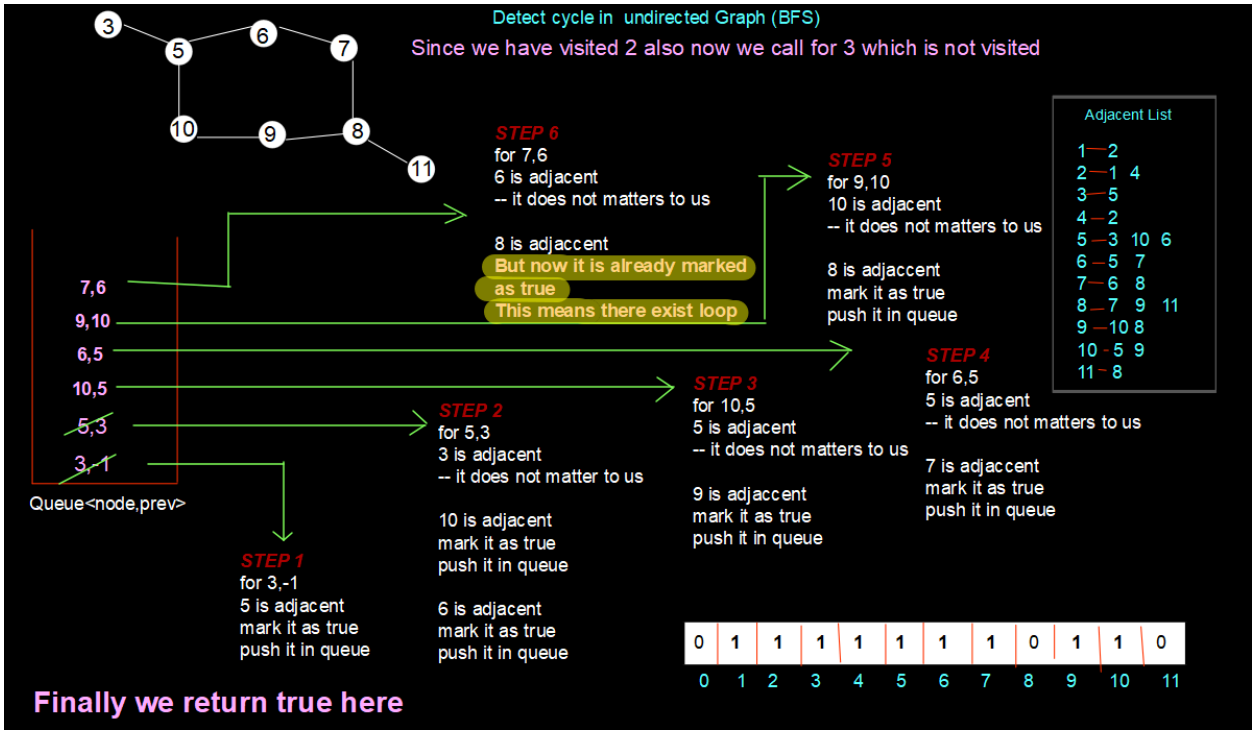
```
for (i= 1->N) {
if (!vis[i])
  if (cyclebfs(i)) // if cycle is found , we return true
            Return True;
}
```



Approach: For the first component, firstly we insert the first element of the graph by marking it as visited in the Queue having prev of the first element as -1. Now we pop out that element and traverse the adjacent element list for that and then insert the elements in the Queue and continue the process until no adjacent element is found for the present element. Here, we can see that no loop is formed. So, we return false for the first component.



For the second component, a similar process will be done as we have done for the first component. Here we will be able to see that when we come to 7, 8 is one of the next adjacent ones which matters to us, and also, it is already marked as positive. This means someone is linked already with this element. So, it forms a loop and we return true finally.



Code:

```
boolean checkForCycle(ArrayList<ArrayList<Integer>> adj, int s,
           boolean vis[]){
       Queue<Node> q =  new LinkedList<>(); //BFS
       q.add(new Node(s, -1));
       vis[s] =true;

       while(!q.isEmpty()){
            Node nodeInfo = q.poll();
           int node = nodeInfo.curr;
           int parent = nodeInfo.parent;

           for (Integer nbr: adj.get(node)){
               if (vis[nbr] == false) {
                   q.add(new Node(it, node));
                   vis[nbr] = true;
               }
               else if (par != nbr) return true;
           } }
       return false;
   }
   public boolean isCycle(int V, ArrayList<ArrayList<Integer>> adj){
        boolean vis[] = new boolean[V];
        Arrays.fill(vis,false);

        for(int i = 0; i < V; i++)
            if (vis[i] == false)
                if (checkForCycle(adj, i, vis))
                    return true;
```

```
        return false;
    }

public static void main(String[] args) throws IOException{

            int V = 5;
            ArrayList<ArrayList<Integer>>adj = new ArrayList<>();
            for(int i = 0; i < V; i++)
                adj.add(i, new ArrayList<Integer>());
                adj.get(0).add(1);
                adj.get(0).add(2);
                adj.get(2).add(3);
                adj.get(1).add(3);
                adj.get(2).add(4);

            Solution obj = new Solution();
            boolean ans = obj.isCycle(V, adj);

            if(ans)  System.out.println("Yes");
            else System.out.println("No");   } }
```

USING DFS APPROACH

Approach: Run a for loop from the first node to the last node and check if the node is visited. If it is not visited then call the function recursively which goes into the depth as known as DFS search and if you find a cycle you can say that there is a cycle in the graph.

- Basically calling the isCyclic function with number of nodes and passing the graph Traversing from 1 to number of nodes and checking for every node if it is unvisited
- If the node is unvisited then call a function checkForCycle, that checks if there is a cycle and returns true if there is a cycle.
- Now the function checkForCycle has the node and previous of the parent node. It will also have the visited array and the graph that has adjacency list
- Mark it as visited and then traverse for its adjacency list using a for loop.
- Calling DFS traversal if that node is unvisited call recursive function that checks if its a cycle and returns true
- If the previously visited node and it is not equal to the parent we can say there is cycle again and will return true
- Now if you have traveled for all adjacent nodes and all the DSF have been called and it never returned true that means we have done the DSF call entirely and now we can return false, that mean there is no DSF cycle

Code:

```
class Solution {
    public boolean checkForCycle(int node, int parent, boolean vis[],
ArrayList<ArrayList<Integer>> adj) {
        vis[node] = true;
        for (Integer it: adj.get(node)) {
            if (vis[it] == false) {
                if (checkForCycle(it, node, vis, adj) == true)
```

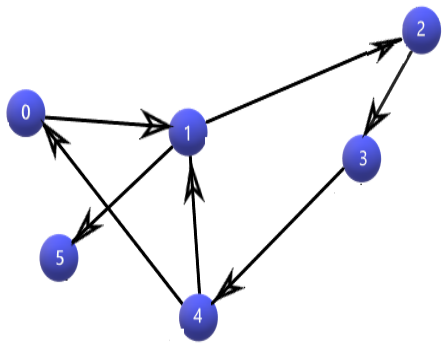
```
                    return true;
                } else if (it != parent)
                    return true;
            }
            return false;
        }
        // 0-based indexing Graph
        public boolean isCycle(int V, ArrayList < ArrayList < Integer >> adj) {
            boolean vis[] = new boolean[V];

            for (int i = 0; i < V; i++) {
                if (vis[i] == false) {
                    if (checkForCycle(i, -1, vis, adj))
                        return true;
                }
            }

            return false;}}
```

1. Detect a cycle of Directed graph using Bfs & Dfs

Given is a 2D adjacency list representation of a directed graph. Check whether the graph has cycles are not.
Example: Input: Graph= { 0 -> 1 1 -> 2, 5 2 -> 3 3 -> 4 4 -> 0, 1 5 -> null }
Output: Cycle detected
Explanation: The input graph is basically represented by an adjacency list. Each index denotes a vertex in the graph and also has a list of vertices, with which the index vertex is connected. Refer to the picture below.



Using dfs approach Solution 1: Intuition: A cycle involves at least 2 nodes. The basic intuition for cycle detection is to check whether a node is reachable when we are processing its neighbors and also its neighbors' neighbors, and so on. Let's take the cycle in Example 2 : 0 -> 1 -> 2 -> 3 – > 4 -> 0.
We can see that 0 is reachable through 1, 2, 3, and 4. We can use the normal DFS traversal with some modifications to check for cycles.
Approach: Usually, we need a separate array for DFS traversal, which has information on whether a particular node has been visited or not. For this cycle detection, we need one more array which has information on whether we are exploring one's neighbor or not. That is, for example in Example 2, in DFS, we go from Nodes 0 to 1 to 2. Only when we are finished visiting node 2's neighbors, we can visit node 1's other neighbors. Also, only when we are finished visiting node 1's neighbors, we can go to other neighbors of 0. Within that time if any neighbors of nodes 1 and 2, has an edge to 0, then we can say that there is a cycle. So, we need a second array to tell whether, at any point in time, we are processing a node's neighbors. Once we finished processing the node's neighbors, we can update the array.

So, in our algorithm, we are starting with node 0. This is an important step. Before going through its neighbors, we have to first set 1 to its index in the visited array. And then we have set 1 to the index in the special array(in this case: dfsVis[]), which just says that we going to process its neighbors. As we do for DFS traversal, we check whether we have already visited a node (which is currently a neighbor of another node). If it is already visited (this is the heart of this algorithm), then we check whether we are processing its neighbors (by checking dfsVis []). If so, then yes, we have found a cycle. Because, we haven't finished processing the node's neighbor (variable "**node's**" neighbor, variable "**neighbor**"), but we found an edge from the variable **node** to a variable neighbor. There is also another important thing to do. After traversing a node's neighbors, we should inform the dfsVis array, by setting it to 0, which means we have finished processing its neighbors.
That is our whole algorithm. If our function returns true, then we have a cycle, if it returns false, then we don't have any cycle.

```
// Okay so to clarify the (cnt == N) logic. Although it was a great explanation.
// In Kahn's algorithm, a node only enters the queue if its indegree becomes zero
// and then it further decrements the indegree values of its adjacent nodes right.
// But in the case of cyclic graph , there will be a node from where the cycle will
// be starting and then coming to an end. So in order for the elements present in the
// cycle to get pushed into the queue, that cycle starting node's indegree must
become
// zero right. But but but, that node's indegree will never become zero as one of the
node from the cycle (the second last element) will be pointing towards the starting
node.
// Just imagine with a diagram in the video. So if that node's indegree will never
become zero ,then it will restrict its adjacent nodes(at least) to get evaluated and
hence the total number of nodes getting pushed into the queue (cnt) will never become
(=N).
private static boolean checkCycle(int node, ArrayList<ArrayList<Integer>> adj, int
vis[], int dfsVis[]) {
        vis[node] = 1;
        dfsVis[node] = 1;
        for(Integer neighbor: adj.get(node)) {
             if(vis[neighbor] == 0) {
                if(checkCycle(neighbor, adj, vis, dfsVis) == true) {
                    return true;
             }} else if(dfsVis[neighbor] == 1) {
             return true;
             }
        }
        dfsVis[node] = 0;
        return false;
    }
    public static boolean isCyclic(int N, ArrayList<ArrayList<Integer>> adj) {
        int vis[] = new int[N];
        int dfsVis[] = new int[N];
        for(int i = 0;i<N;i++) {
             if(vis[i] == 0) {
                if(checkCycle(i, adj, vis, dfsVis) == true) return true;
             }
        }
        return false; }}
Using bfs approach also called as kahn's algo
class Solution {
    public boolean isCyclic(int N, ArrayList<ArrayList<Integer>> adj) {
        // int topo[] = new int[N];
```

```
        int indegree[] = new int[N];
        for(int i = 0;i<N;i++) {
            for(Integer it: adj.get(i)) {
                indegree[it]++;
            }
        }
        Queue<Integer> q = new LinkedList<Integer>();
        for(int i = 0;i<N;i++) {
            if(indegree[i] == 0) {
                q.add(i);
            }
        }
        int cnt = 0;
        while(!q.isEmpty()) {
            Integer node = q.poll();
            cnt++;
            for(Integer it: adj.get(node)) {
                indegree[it]--;
                if(indegree[it] == 0) {
                    q.add(it);
                }
            }
        }
        if(cnt == N) return false;
        return true;  }}
```

2. Bipartite check using Bfs & Dfs

Given is a 2D adjacency list representation of a graph. Check whether the graph is Bipartite graph. (Note: In a Bipartite graph, one can color all the nodes with exactly 2 colors such that no two adjacent nodes have the same color)

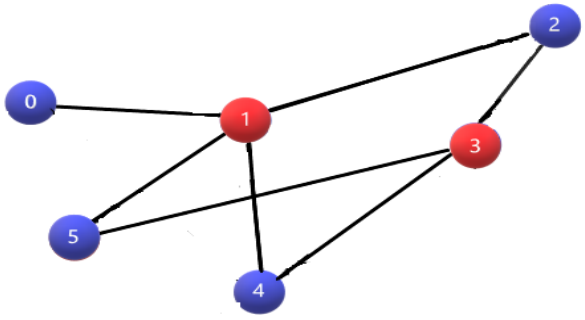
Example 1:

Input:

Graph= { [0 -> 1], [1 -> 0, 2, 4, 5], [2 -> 1, 3], [3 -> 2, 4, 5], [4 -> 1, 3], [5 -> 1, 3] }

Output: It is a Bipartite Graph.

Explanation: The input graph is basically represented by an adjacency list. Each index denotes a vertex in the graph and also has a list of vertices, with which the index vertex is connected. Here we can see that the nodes can be colored such that the Bipartite graph condition holds. For instance, 0 –blue, 1 – red, 2 - blue, 3 – red, 4 – blue, 5 – blue. Refer to the picture.



Solution N Intuition: If we look closely, we can notice that if the graph has no cycle, then such a graph is always bipartite(because you can just change the color of neighbour nodes recursively, the bipartite condition still holds). Also, think of this one, in order to prove that the graph is not a Bipartite one, we need to have adjacent nodes with the same color. Since the graph is not going to have a cycle, we still have the opportunity to change the color. So, the question is how to approach graphs with cycles. Let's see.

Approach:
In the above examples, we can see that both the graphs have cycles, but the 1st graph is bipartite. How is it possible? Let's take a closer look at a cycle. From the 2nd example, **0 -> 1 -> 5 -> 0 is a cycle**. If 0 is colored with red then 1 and 5 have to be colored with blue. But 1 and 5 are connected. Let's take the 1st example, **1 -> 2 -> 3 -> 5 -> 1**. As usual, 1 – red, 2 – blue, 3 – red, 5 – blue.
The key point is that if a cycle has an **even number of nodes**, then there is **no issue**. But on the other hand, if it has an **odd number of nodes**, then the Bipartite condition **fails**. Because we only have 2 colors, if we have an odd number of nodes, definitely the starting and ending nodes will have the same color, but for even it won't be.
Our color array basically has **3 states**. It tells whether the node has been colored, if colored then whether it is the 1st color or 2nd one. In the **checkBipartite** function, we are initializing the color array with -1(not yet colored). We can use the color array to check whether a node is visited or not (we don't need a separate visited array).
Then we are checking whether the node is colored or not. If it is not colored, we are coloring with the opposite color of its neighbor. That is, 1 – color[node], gives 1 if the color[node] is 0 and 0 if the color[node] is 1. This is how we are inverting the color of the adjacencies.
If the current neighbor is already colored, we check its color and the variable, node's color. We know that they are adjacent nodes, if the colors are the same, we can infer that it is not a Bipartite graph, so returning false, else returning true.

Code: Using dfs

```
class Solution {
    static boolean dfsCheck(ArrayList < ArrayList < Integer >> graph, int node, int
color[]) {
        for (Integer it: graph.get(node)) {
            if (color[it] == -1) {
                //Color of variable neighbor is the inverted color of variable node
                color[it] = 1 - color[node];
                if (!dfsCheck(graph, it, color))
                    return false;
            } else if (color[it] == color[node]) {
                return false;
            } }
        return true; }
    static boolean checkBipartite(ArrayList < ArrayList < Integer >> graph, int n) {
        int color[] = new int[n];
        for (int i = 0; i < n; i++) {
            color[i] = -1; }
        for (int i = 0; i < n; i++) {
            if (color[i] == -1) {
                if (!dfsCheck(graph, i, color)) {
                    return false;
                } } }
        return true; }
```

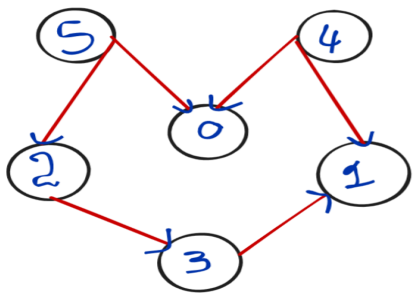
Using bfs

```
class Main{
      boolean bfsCheck(ArrayList<ArrayList<Integer>> adj, int node, int color[]) {
            Queue<Integer> q = new LinkedList<>();
            q.add(node);
```

```
            color[node] = 1;
            while(!q.isEmpty()) {
                  Integer nde = q.poll();
                  for(Integer it: adj.get(nde)) {
                        if(color[it] == -1) {
                              color[it] = 1 - color[nde];
                              q.add(it);
                        }
                        else if(color[it] == color[nde]) {
                              return false;
                        }
                  }}
            return true;  }
    boolean checkBipartite(ArrayList<ArrayList<Integer>> adj, int n) {
        int color[] = new int[n];
        for(int i = 0;i<n;i++) {
             color[i] = -1;
        }
        for(int i = 0;i<n;i++) {
             if(color[i] == -1) {
                   if(!bfsCheck(adj, i, color)) {
                         return false;
                   }
             } }
        return true;  }
```

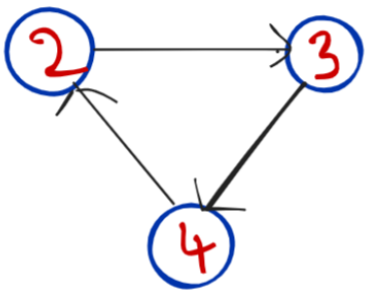
3. TopoSort using Bfs & Dfs

Given a graph, find the topological order for the given graph.
Topological sort: The linear ordering of nodes/vertices such that if there exists an edge between 2 nodes u,v then ‘u’ appears before ‘v’.



Example: Input:
Output: 4 5 2 0 3 1

Explanation:
4 is appearing before its neighbours (1,0)
5 is appearing before its neighbours (0,2)
2 is appearing before its neighbours (3)
3 is appearing before its neighbours (1)
Note: This can be possible only for DAG (Directed acyclic graph) because in an undirected graph we can’t decide which node will come first because there will be no direction, and if there is a cycle topological order will not be possible (See below figure to understand why it is not possible for graphs containing cycle).

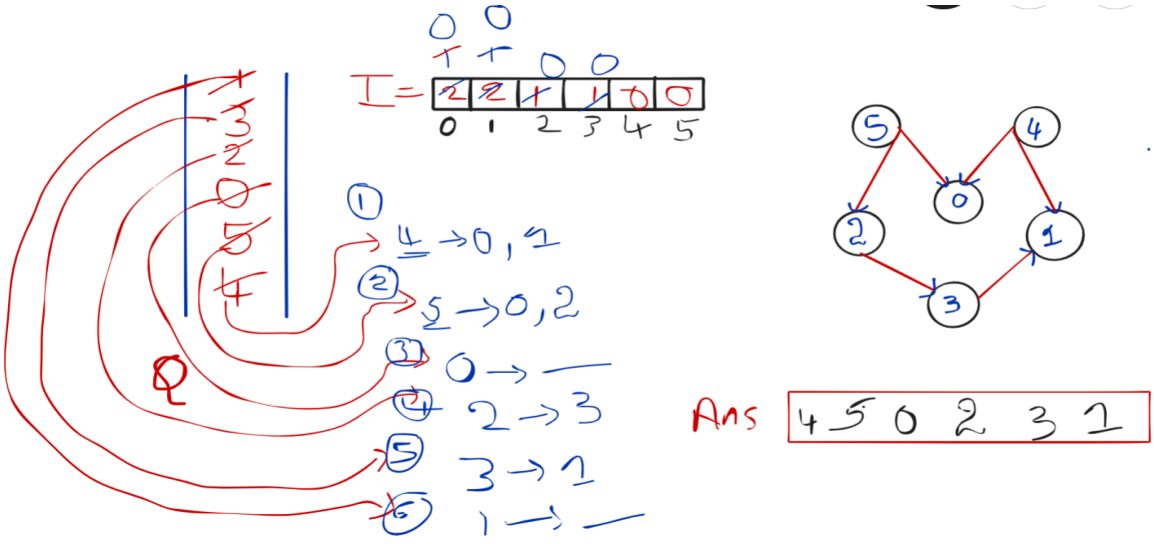


2 ->3 ->4 ->2 here if you can observe
2 is appearing before 3
3 is appearing before 4
4 is appearing before 2

But 2 has already appeared before 4, So topological ordering will not be possible
Intuition: The question states that if there is an edge between u and v then u should appear before v, Which means we have to start this question from a node that doesn't have any previous edges. But how to find that node that has no edge before if? Here, we use the concept of in-degree (Number of edges pointing toward a node). We find an in-degree which has indegree=0 and we start from this. We use the Indegree concept to find topological sorting. Let's see how.
Approach: In order to maintain the In-degree of each and every node, we take an array of size v(where v is the number of nodes).
Find in-degree of all nodes and fill them in an array
Now take Queue data structure and add nodes that have in-degree is 0 (as we discussed in the intuition), and simply apply bfs on queue with some condition.
Take the top/peek node from Queue (let the popped node be x), add this x to our answer. (If you can observe clearly the above step is nothing but Normal BFS traversal).
We'll apply some conditions to the BFS:

1. Now take neighbour nodes of popped nodes and reduce their in-degree by 1.
2. Check if any of the popped element nodes in degree becomes 0, after reducing in-degree by 1 if it happens then add this neighbour element to the queue for which the in-degree became zero. Repeat step 4 till the queue becomes empty.

We'll apply some conditions to the BFS:
a) Now take neighbour nodes of popped nodes and reduce their indegree by 1.
b) Check if any of popped element nodes in degree become 0, after reducing in-degree by 1, if it happens then add this neighbour element which in-degree became 0 to the queue.
Let's see how it works pictorially:



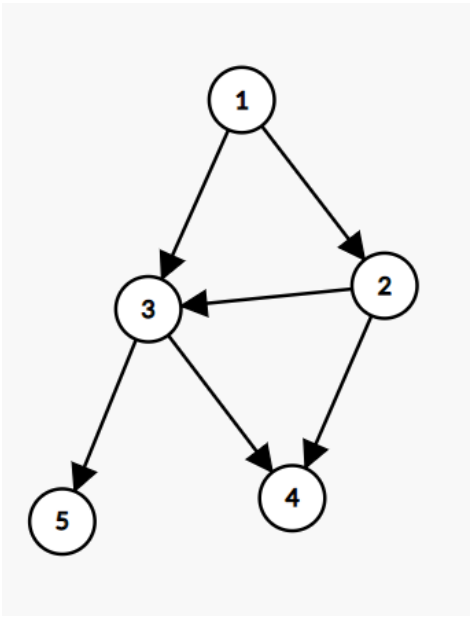
Code: of Bfs

```
public boolean isCyclic(int N, ArrayList<ArrayList<Integer>> adj) {
    int topo[] = new int[N], indegree[] = new int[N];
    //finding indegree
    for(int i = 0;i < N;i++) {
        for(Integer it: adj.get(i)) {
```

```
                indegree[it]++; }}
        Queue<Integer> q = new LinkedList<Integer>();
        for(int i = 0;i<N;i++) {
            //adding nodes to queue with indegree = 0
            if(indegree[i] == 0) {
                q.add(i);
            } }
        int cnt = 0, ind = 0;
        while(!q.isEmpty()) {
            Integer node = q.poll();
            topo[ind++] = node;
            cnt++;
            //getting neighbour nodes of popped node and decreasing  their
            indegree by1
            for(Integer it: adj.get(node)) {
                indegree[it]--;
                if(indegree[it] == 0) {
                    q.add(it);
                } } }
         //printing topological ordering of nodes
        for (int i=0;i< topo.length;i++){
            System.out.print(topo[i]+" ");
        }
        if(cnt == N) return false;
        return true;  } }
```

Using Dfs approach:

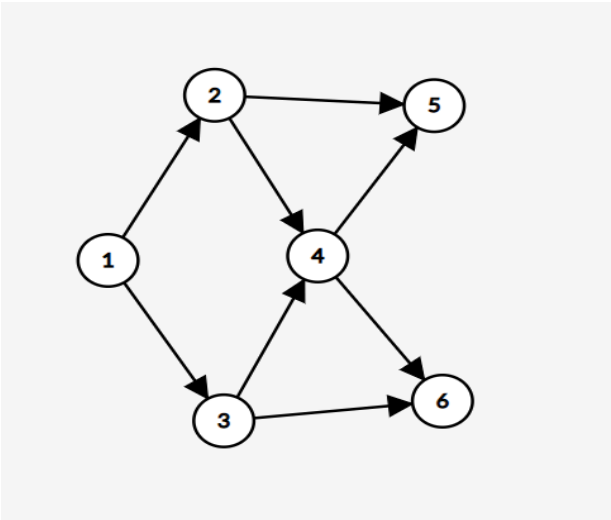
Given a DAG(Directed Acyclic Graph), print all the vertex of the graph in a topologically sorted order. If there are multiple solutions, print any. **Pre-req**: DFS traversal, Graphs, Stack data structure.



Example 1: Input:

Output:
One of the solutions is 1,2,3,5,4

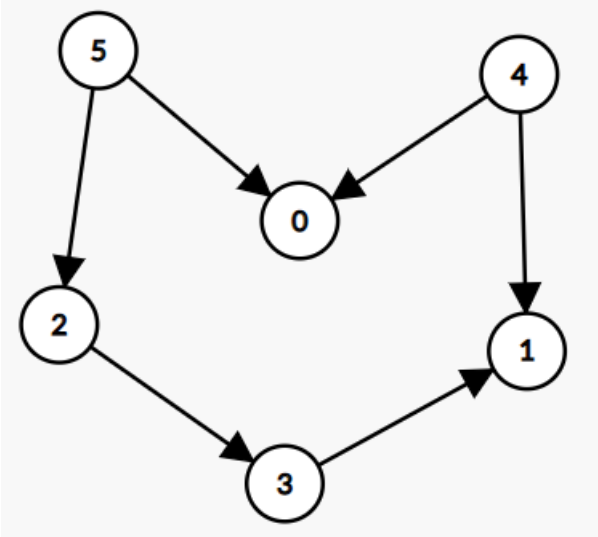
Example 2:
Input:



Output: One of the solution is 1,2,3,4,5,6

Solution -> First of all let's understand Topological Sorting. It means linear ordering of vertices such that there is an edge u—-> v**,** u appears before v in the ordering.

Suppose for a given graph,



Some of the possible Topological orders can be:

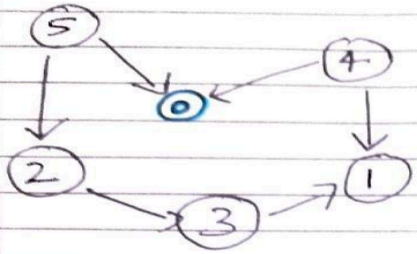
1. 5,4,2,3,1,0
2. 4,5,2,3,1,0

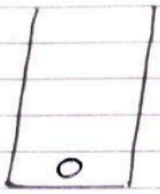
-> In both cases we can see, that
4->0 (4 appears before 0) , 5>3 (5 appears before 3), …
-> Similarly there can be **multiple toposorts order for the given graph** but the condition should be if there is an edge u->v then **u should always appear before v.**
-> Topological Sorting is applicable only for **DAG(Directed Acyclic Graph)**. Why is it so? Because of the following reasons:

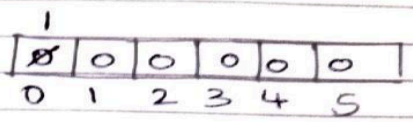
1. For Undirected graphs ,only u->v is not applicable . It cannot be sure whether the edge is between u to v or v to u (u-v) .
2. In a cyclic graph there will always be a dependency factor . You cannot make sure that you can have linear ordering of vertices.

-> Finally, now you have a clear understanding of what Topological Sorting is. We will be using the **DFS(Depth First Search)** method to solve the problem. What we will be doing is for each vertex we will explore its adjacent vertex. After exploring, we will store the current vertex in a data structure to maintain Topo Sort.
Approach: We will be using the following data structure to get Topo sort:

1. Visited Vector – To store visit of each vertex
2. Stack – To maintain the topo sort order.

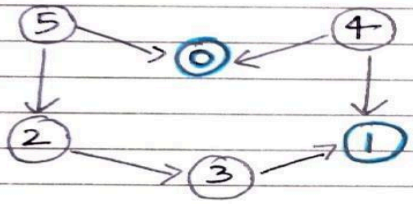


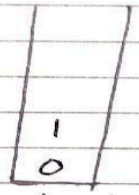


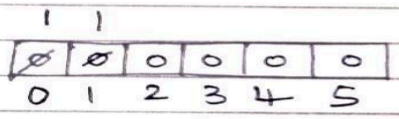


dfs(0)

x x

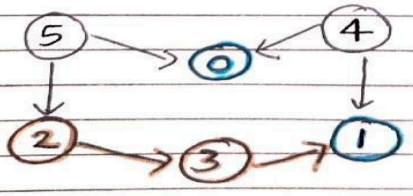


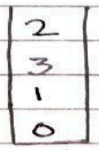


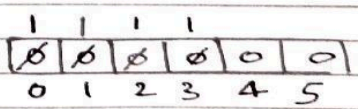


dfs(1)

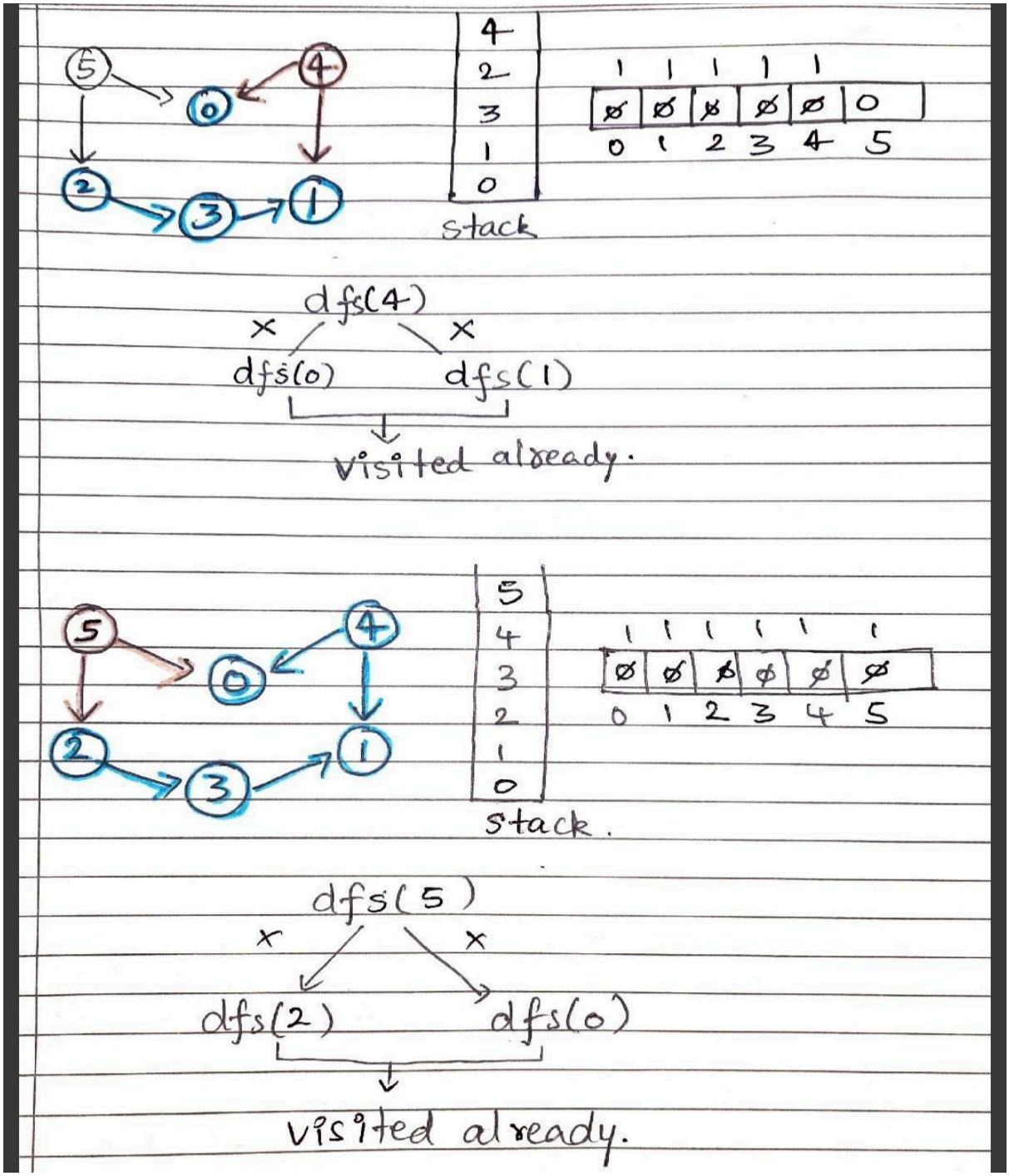
x x







dfs(2) → dfs(3) ~~→~~ dfs(1) [visited]



Did you notice something while using Stack? Just because there was an edge from u to v. Dfs call will go from u to v. The 1st dfs (v) will get over first and then dfs(u). Here we are making sure that if u->v, then we will **first push v into the stack and then u will be pushed**. This is how Topological order is maintained in the Stack.

Code:

```java
class Solution {
    static void findTopoSort(int node, int vis[], ArrayList<ArrayList<Integer>> adj,
Stack<Integer> st) {
        vis[node] = 1;
        for(Integer it: adj.get(node)) {
            if(vis[it] == 0) {
                findTopoSort(it, vis, adj, st);
            }
        }
        st.push(node);
    }
    static int[] topoSort(int N, ArrayList<ArrayList<Integer>> adj) {
        Stack<Integer> st = new Stack<Integer>();
        int vis[] = new int[N];

        for(int i = 0;i<N;i++) {
            if(vis[i] == 0) {
                findTopoSort(i, vis, adj, st);
            }  }

        int topo[] = new int[N];
        int ind = 0;
        while(!st.isEmpty()) {
            topo[ind++] = st.pop();
        }
        // for(int i = 0;i<N;i++) System.out.println(topo[i] + " ");
        return topo;
    }
```

B. SHORTEST PATH ALGOS

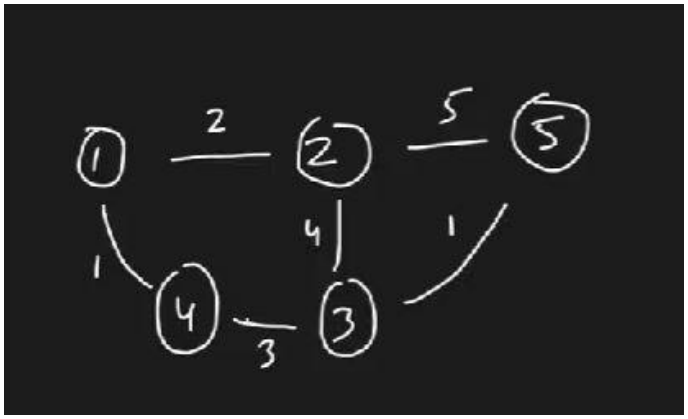
(DIJKSTRA, BELLMAN FORD & FLOYD WARSHELL)

1. Calculate shortest distance using all 3 algos

Given a weighted, undirected, and connected graph of V vertices and E edges, Find the shortest distance of all the vertex's from the source vertex S.**Note:** The Graph doesn't contain any negative weight cycle.

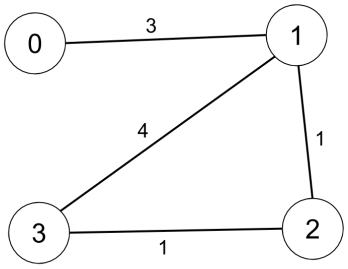
Example 1:

Input: Adj list :
[[1,2,2],[1,4,1],[2,1,2],[2,3,4],[2,5,5],[3,2,4],[3,4,3],[3,5,1],[4,1,1],[4,3,3],[5,2,5],[5,3,1]] **Output:** [0,2,4,1,5]



Explanation: Given the following undirected graph, the shortest distance from node 1(source node) to itself is definitely zero. For nodes 2 and 4, the minimum distance is 2 and 1 respectively.
For node 3, the possible paths are 1->2->3(cost = 6) or 1->4->3(cost = 4) or 1->2->5->3(cost = 8). The minimum of these three paths would be our answer which is 4.
For node 5, the possible paths are 1->2->5(cost = 7) or 1->4->3->5(cost = 5) or 1->4->3->2->5(cost = 13). The minimum of these three paths would be our answer which is 5.

Example 2: Input: Adj list :
[[0,1,3],[1,2,1],[1,3,4],[1,0,3],[2,3,1],[2,1,1],[3,1,4],[3,2,1]]
Output: [0,3,4,5]



Explanation: Given the following undirected graph, the shortest distance from node 0(source node) to itself is definitely zero. For node 1 to shortest distance would be 3(only way to reach node 1).
For node 2 we have paths 0->1->2(cost = 4) or 0->1->3->2(cost = 8). The minimum of these three paths would be our answer which is 4. For node 3 we have paths 0->1->3(cost =7) or 0->1->2->3(cost = 5)

Intuition: The above problem seems familiar to finding the shortest distance in the case of unit edge weights for undirected graphs. Hence, our first guess could be a BFS kind of approach. The only thing that we need to take care of is that of all the paths possible between a pair of nodes, we need to store the minimum distance between them. That is, say we have a node that has been reached by two paths, one with a cost of 5 and another with a cost of 10. It is obvious that the path with a cost of 5 would continue to give optimal distance for its adjacent node as opposed to the path with a cost of 10.

Approach: **We can implement the algorithm using the following steps:**

1. We would be using a min-heap and a distance array of size N initialized with infinity (indicating that at present none of the nodes are reachable from the source node) and initialize the distance to source node as 0.
2. We push the source node to the queue.
3. For every node at the top of the queue we pop that element out and look out for its adjacent nodes. If the current reachable distance is better than the previous distance indicated by the distance array, we update the distance and push it in the queue.
4. A node with a lower distance would be at the top of the priority queue as opposed to a node with a higher distance. By following the steps 3, until our queue becomes empty, we would get the minimum distance from the source node to all other nodes. Here's a quick demonstration of the same.

Dijkstra algo
Using priority Queue and bfs approach

Code:

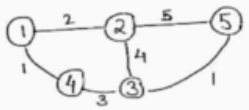
```
void shortestPath(int s, ArrayList<ArrayList<Node>> adj, int N)  {
        int dist[] = new int[N];

        for(int i = 0; i < N; i++) dist[i] = 100000000;
        dist[s] = 0;
        PriorityQueue<Node> pq = new PriorityQueue<Node>(N, new Node());
        pq.add(new Node(s, 0));

        while(pq.size() > 0) {
            Node node = pq.poll();
            for (Node nbr: adj.get(node.getV())) {
                if (dist[node.getV()] + nbr.getWeight() < dist[nbr.getV()]) {
                    dist[it.getV()] = dist[node.getV()] + nbr.getWeight();
                    pq.add(new Node(it.getV(), dist[it.getV()]));
                }
            }
        }

        System.out.println("The distances from source "+s+" are : ");
        for (int i = 0; i < N; i++) System.out.print( dist[i] + " ");
    }
```

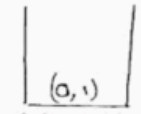
Consider the following graph



Array:

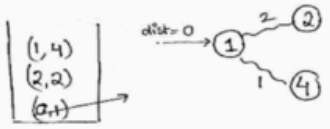
∞	0	∞	∞	∞	∞
0	1	2	3	4	5

Min-Heap



<Dist, Node>

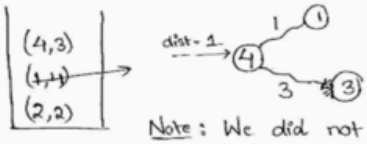
Iteration 1 :



Updated Array :

∞	0	2	∞	1	∞
0	1	2	3	4	5

Iteration 2 :

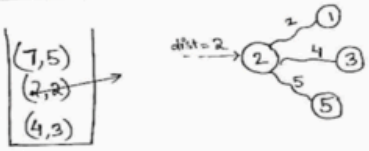


Updated Array :

∞	0	2	4	1	∞
0	1	2	3	4	5

Note: We did not update the dist. of node 1 since the current distance was already better.

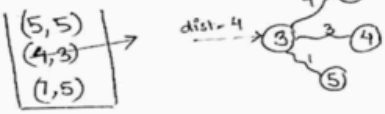
Iteration 3 :



Updated Array :

∞	0	2	4	1	7
0	1	2	3	4	5

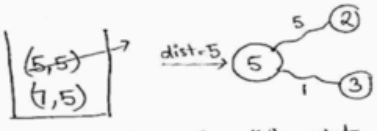
Iteration 4 :



Updated Array :

∞	0	2	4	1	5
0	1	2	3	4	5

Iteration 5 :



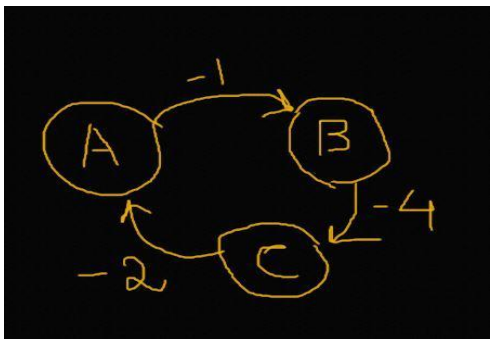
Updated Array :

∞	0	2	4	1	5
0	1	2	3	4	5

Note: Notice how in this state we have two distances for the same node in our min-heap. The distance of 7 would redundant anyways as it won't change our distance array. This why we need a minimum heap!

Using Bellman ford algo

Why Djikstra's doesn't work? The answer is plain and simple. The presence of negative edge weights makes the situation dicey. Djikstra's algorithm might work in some cases and fail in some. Consider the following example:



Recall that in Dijkstra's algorithm, we update the distance array every time we find a better solution which is a lesser distance. With the presence of negative edge weights, our algorithm would continue to update the distance array with lesser and lesser values and we might end up in time limit exceeded or segmentation fault error. Try to dry run the following example and you would understand the issue yourself.

Intuition: We would try to update our distance array in a somewhat same manner as we did in Dijkstra's algorithm. However, we need to handle the negative edge weights as well. Thus, we need to update the array in a controlled fashion a specific number of times. Remember, we don't want to end up in an endless loop.

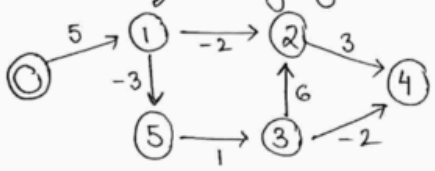
Approach:

1. We would use a technique known as "Relaxing Edges" wherein we would update our distance array if we find a better solution. We would do this N-1 times.
2. Now why N-1? Understand that for a given graph with N nodes, the maximum number of edges we could have between any two nodes is N-1 in a single path. Moreover, our adjacency list might be in such a manner that only one node is updated in a single pass. Thus, to try out all nodes, we would require at least N-1 iterations.The following is the demonstration of the same:

```
void bellmanFord(ArrayList<Node> edges, int N, int src) {
       int dist[] = new int[N];
       for(int i = 0;i<N;i++) dist[i] = 10000000;
       dist[src] = 0;

       for (int i = 1; i <= N - 1; i++) {
              for (Node node : edges) {
                     if (dist[node.getU()] + node.getWeight() < dist[node.getV()]) {
                            dist[node.getV()] = dist[node.getU()] + node.getWeight();
                     }
              }
       }
       int fl = 0;
       for(Node node: edges) {
              if(dist[node.getU()] + node.getWeight() < dist[node.getV()]) {
                     fl = 1;
                     System.out.println("Negative Cycle");
                     break;
              }
        }
       if(fl == 0) {
       System.out.println(Arrays.toString(dist));
              }
}
```

Consider the following graph:



Array:

	5	3		6	-2
0	∞	∞	∞	∞	∞
0	1	2	3	4	5

→ Relax all edges N-1 times.

if (dist[u] + wt < dist[v])

dist[v] = dist[u] + wt

Relaxation 1:

dist[3] + 6 < dist[2] → NO
dist[5] + 1 < dist[3] → NO
dist[0] + 5 < dist[1] → YES → update array
dist[1] − 3 < dist[5] → YES → update array
dist[1] − 2 < dist[2] → YES → update array
dist[3] − 2 < dist[4] → NO
dist[2] + 3 < dist[4] → YES → update array

The updated array is as follows:

0	5	3	∞	6	-2
0	1	2	3	4	5

Relaxation 2:

dist[3] + 6 < dist[2] → NO
dist[5] + 1 < dist[3] → YES → update array
dist[0] + 5 < dist[1] → NO
dist[1] − 3 < dist[5] → NO
dist[3] − 2 < dist[4] → YES → update array
dist[1] − 2 < dist[2] → NO
dist[2] + 3 < dist[4] → NO

Updated array:

0	5	3	-1	-3	-2
0	1	2	3	4	5

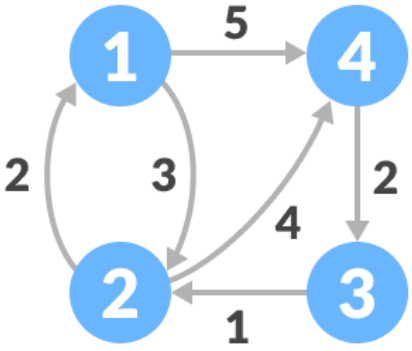
On doing the above step 3 more times, our distance array would be as follows:

0	5	3	-1	-3	-2
0	1	2	3	4	5

Floyd-Warshall Algorithm is an algorithm for finding the shortest path between all the pairs of vertices in a weighted graph. This algorithm works for both the directed and undirected weighted graphs. But, it does not work for the graphs with negative cycles (where the sum of the edges in a cycle is negative). A weighted graph is a graph in which each edge has a numerical value associated with it. Floyd-Warhshall algorithm is also called as Floyd's algorithm, Roy-Floyd algorithm, Roy-Warshall algorithm, or WFI algorithm.

How does the Floyd-Warshall Algorithm Work?

Let the given graph be:



Initial graph

Follow the steps below to find the shortest path between all the pairs of vertices.

1. Create a matrix A0 of dimension n*n where n is the number of vertices. The row and the column are indexed as i and j respectively. i and j are the vertices of the graph. Each cell A[i][j] is filled with the distance from the ith vertex to the jth vertex. If there is no path from ith vertex to jth vertex, the cell is left as infinity.Fill each cell with the distance between ith and jth vertex
2. Now, create a matrix A1 using matrix A0. The elements in the first column and the first row are left as they are. The remaining cells are filled in the following way.

Let k be the intermediate vertex in the shortest path from source to destination. In this step, k is the first vertex. A[i][j] is filled with (A[i][k] + A[k][j]) if (A[i][j] > A[i][k] + A[k][j]).

That is, if the direct distance from the source to the destination is greater than the path through the vertex k, then the cell is filled with A[i][k] + A[k][j]. In this step, k is vertex 1. We calculate the distance from source vertex to destination vertex through this vertex k.

$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{array}$$

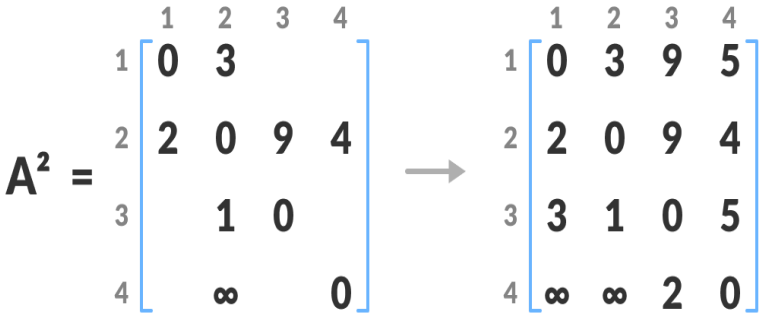
$$A^1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & & \\ \infty & & 0 & \\ \infty & & & 0 \end{bmatrix} \end{array} \rightarrow \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & 9 & 4 \\ \infty & 1 & 0 & 8 \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{array}$$

Calculate the distance from the source vertex to destination vertex through this vertex k

For example: For A1[2, 4], the direct distance from vertex 2 to 4 is 4 and the sum of the distance

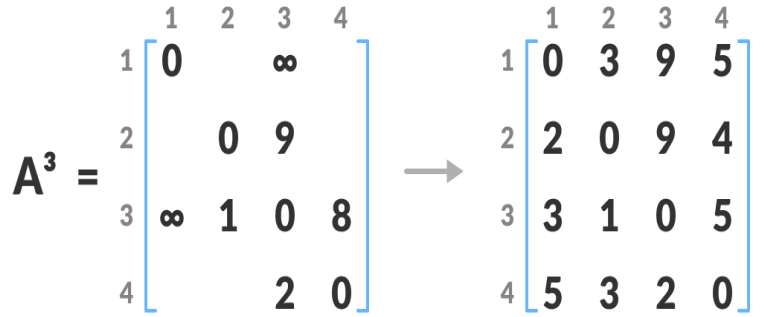
from vertex 2 to 4 through vertex (ie. from vertex 2 to 1 and from vertex 1 to 4) is 7. Since 4 < 7, A0[2, 4] is filled with 4.

3. Similarly, A2 is created using A1. The elements in the second column and the second row are left as they are.

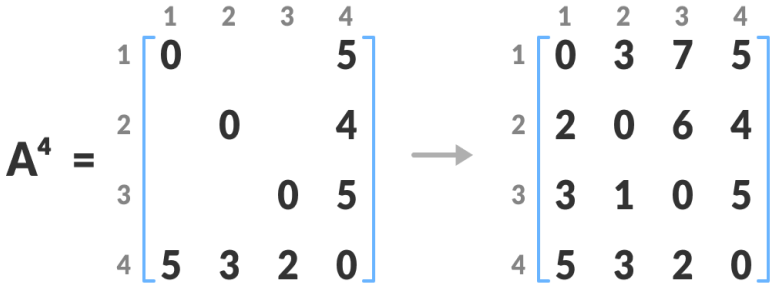


In this step, k is the second vertex (i.e. vertex 2). The remaining steps are the same as in step 2. Calculate the distance from the source vertex to destination vertex through this vertex 2

4. Similarly, A3 and A4 is also created.Calculate the distance from the source vertex to destination vertex through this vertex 3 Calculate the distance from the source vertex to destination vertex through this vertex 4



5. A4 gives the shortest path between each pair of vertices.



```
n = no of vertices
A = matrix of dimension n*n
for k = 1 to n
    for i = 1 to n
        for j = 1 to n
            Ak[i, j] = min (Ak-1[i, j], Ak-1[i, k] + Ak-1[k, j])
return A
```

```java
// Floyd Warshall Algorithm in Java

class FloydWarshall {
  final static int INF = 9999, nV = 4;

  // Implementing floyd warshall algorithm
  void floydWarshall(int graph[][]) {
    int matrix[][] = new int[nV][nV];
    int i, j, k;

    for (i = 0; i < nV; i++)
      for (j = 0; j < nV; j++)
        matrix[i][j] = graph[i][j];

    // Adding vertices individually
    for (k = 0; k < nV; k++) {
      for (i = 0; i < nV; i++) {
        for (j = 0; j < nV; j++) {
          if (matrix[i][k] + matrix[k][j] < matrix[i][j])
            matrix[i][j] = matrix[i][k] + matrix[k][j];
        }
      }
    }
    printMatrix(matrix);
  }

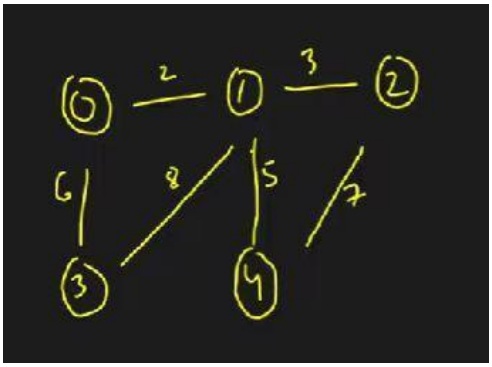
  void printMatrix(int matrix[][]) {
    for (int i = 0; i < nV; ++i) {
      for (int j = 0; j < nV; ++j) {
        if (matrix[i][j] == INF)
          System.out.print("INF ");
        else
          System.out.print(matrix[i][j] + "   ");
      }
      System.out.println();
    }
  }

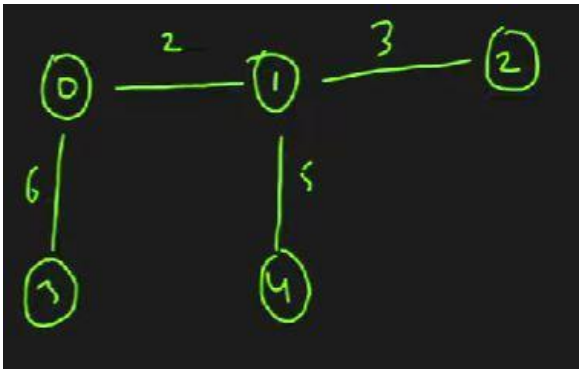
  public static void main(String[] args) {
    int graph[][] =
      { { 0, 3, INF, 5 }, { 2, 0, INF, 4 }, { INF, 1, 0, INF }, { INF, INF, 2, 0 } };
    FloydWarshall a = new FloydWarshall();
    a.floydWarshall(graph);
  }
}
```

C. MIN SPANNING TREE (MST) (KRUSKAL'S & PRIMS)

1. Get MST using prims and kruskals

Prims: Given a weighted, undirected, and connected graph of V vertices and E edges. The task is to find the sum of weights of the edges of the Minimum Spanning Tree.
Definition: A minimum spanning tree consists of N nodes and N-1 edges connecting all the nodes which have the minimum cost(sum of edge weights).
Note: It is known as a tree since a tree doesn't have cycles involved. A graph with N nodes and N-1 edges is equivalent to a tree.
Example: Input: **Output:**





Explanation: Given the following graph, the minimum spanning tree is the one given in the output. It consists of 5 nodes and 4 edges and this configuration gives us the minimum weight(cost = 16)
Solution 1(Brute Force): Intuition: Let's start with any node in our graph. As the first step, we find out all the adjacent edges connected to this node and then pick up the minimum one. Now we have 2 nodes. We further continue this process but now we would consider all the edges connected to these two nodes and pick the minimum one. We then continue this process until all the nodes are covered.
Note: It might happen that while picking up an edge we might end up forming a cycle. In that case, we would pick up the next lowest edge that doesn't form a cycle.
Approach: We would have 3 arrays(key, mst and parent).
Significance of all three arrays:

- Key : This array holds the weight/ cost of the MST(intialized to INT_MAX except the index 0 which is set with value of zero)
- MST : This is a boolean array which indicates whether a node is already a part of MST or not(initialized to false except the index 0 which is true)
- Parent : This indicates the parent of a particular node in the MST(initialized to -1)

Steps:

- Assuming we start with node 0, the index 0 in the key array is initialized to zero(because it is the first node in the MST). We find the index/node in the key array which has the minimum weight. We then find all its adjacent edges and pickup the the one with minimum weight.
- Also at the same time we mark this node as true(indicating that it is now a part of the MST) and also set it's parent as node '0'.
- After this, we would continue to find the one with minimum weight in the key array that is not a part of the MST(Notice that this is where we ensure that we pickup the node with minimum weight and we do not choose an edge that might cause a cycle)
- We continue this process until all nodes become a part of the MST

Code:

```
void primsAlgo(ArrayList<ArrayList<Node>> adj, int N) {
        int key[] = new int[N];
        int parent[] = new int[N];
        boolean mstSet[] = new boolean[N];
        for(int i = 0;i<N;i++) {
             key[i] = 100000000;
             mstSet[i] = false;
        }
        key[0] = 0;
        parent[0] = -1;
        for(int i = 0; i < N - 1; i++) {
             int mini = 100000000, u = 0;
             for(int v = 0; v < N; v++) {
                   if(mstSet[v] == false && key[v] < mini) {
                         mini = key[v];
                         u = v;
                   } }
             mstSet[u] = true;
             for(Node it: adj.get(u)) {
             if(mstSet[it.getV()] == false && it.getWeight() < key[it.getV()]) {
                         parent[it.getV()] = u;
                         key[it.getV()] = it.getWeight();
                   } }  }
        for(int i = 1; i < N; i++) {
             System.out.println(parent[i] + " - " + i);
        } } }

class Pair {
    int node;
    int distance;
    public Pair(int distance, int node) {
        this.node = node;
        this.distance = distance;
    }
}
class Solution {
    //Function to find sum of weights of edges of the Minimum Spanning Tree.
    static int spanningTree(int V,
                            ArrayList<ArrayList<ArrayList<Integer>>> adj) {
        PriorityQueue<Pair> pq =
            new PriorityQueue<Pair>((x, y) -> x.distance - y.distance);

        int[] vis = new int[V];
        // {wt, node}
        pq.add(new Pair(0, 0));
        int sum = 0;
        while (pq.size() > 0) {
            int wt = pq.peek().distance;
```

```
            int node = pq.peek().node;
            pq.remove();

            if (vis[node] == 1) continue;
            // add it to the mst
            vis[node] = 1;
            sum += wt;

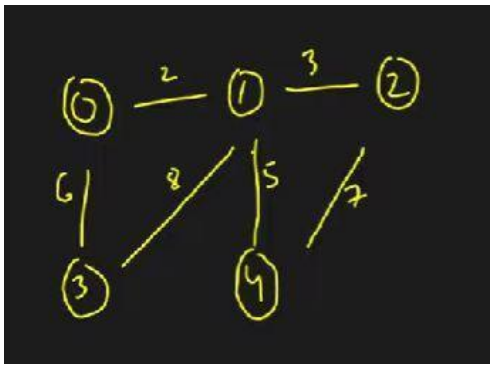
            for (int i = 0; i < adj.get(node).size(); i++) {
                int edW = adj.get(node).get(i).get(1);
                int adjNode = adj.get(node).get(i).get(0);
                if (vis[adjNode] == 0) {
                    pq.add(new Pair(edW, adjNode));
                }
            }
        }
        return sum;
    }
}
```

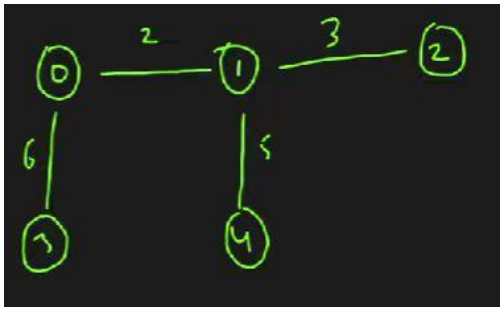
Kruskals: Given a weighted, undirected, and connected graph of V vertices and E edges. The task is to find the sum of weights of the edges of the Minimum Spanning Tree.

Definition: A minimum spanning tree consists of N nodes and N-1 edges connecting all the nodes which have the minimum cost(sum of edge weights).

Note: It is known as a tree since a tree doesn't have cycles involved. A graph with N nodes and N-1 edges is equivalent to a tree.

Example: Input: **& Output**





Explanation: Given the following graph, the minimum spanning tree is the one given in the output. It consists of 5 nodes and 4 edges and this configuration gives us the minimum weight(cost = 16)

Solution N Intuition: In Prim's algorithm our idea was to get hold of the minimum edge weight possible. The idea remains the same. Could we somehow not use our adjacency list and try to incorporate what we learned in the Disjoint Set data structure? Intuitively there are primarily two things we need to keep in mind:

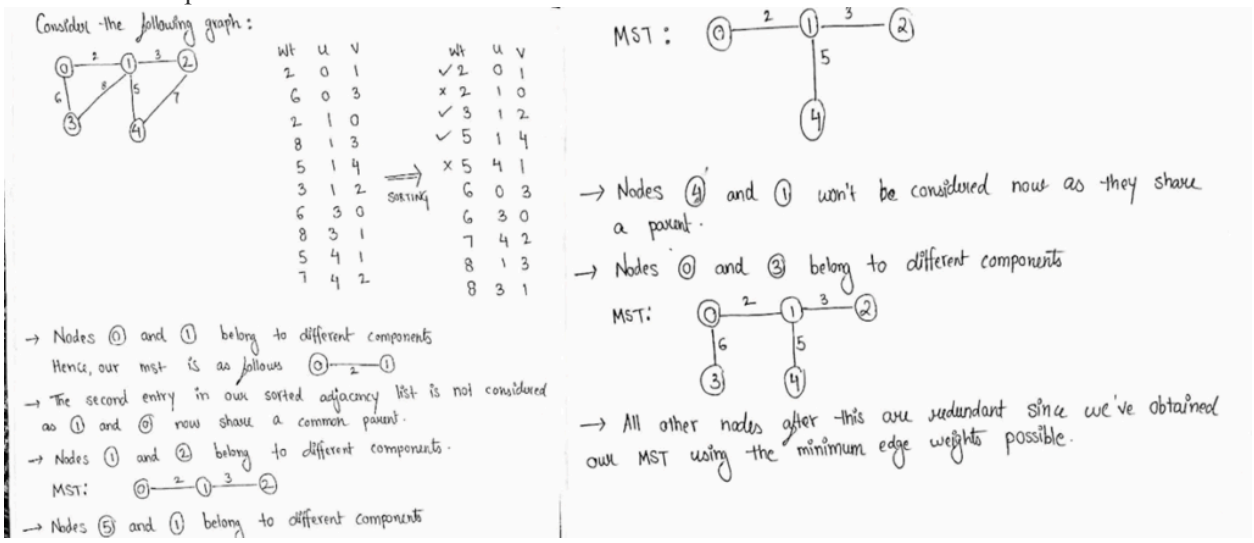
- We must always try to get the minimum edge weights
- The only constraint is that an edge must not be picked up such that it forms a cycle.

Can we address this constraint using the findParent function?

Approach:

- Sort all the edges according to their weight

- Greedily pick the minimum edge and make sure the two nodes belong to different components(using disjoint set data structure findParent operation). Remember, if they belong to the same component it would indicate we would be having cycles which is not possible in a MST.
- Also, once we have a node to be a part of the MST, we must join the two components using the union operation of DSU.



Code:

```
class Node{
private int u;
private int v;
private int weight;

    Node(int _u, int _v, int _w) { u = _u; v = _v; weight = _w; }
    Node() {}
    int getV() { return v; }
    int getU() { return u; }
    int getWeight() { return weight; }
}

class SortComparator implements Comparator<Node> {
    @Override
    public int compare(Node node1, Node node2){
        if (node1.getWeight() < node2.getWeight()) return -1;
        if (node1.getWeight() > node2.getWeight()) return 1;
        return 0; }
}

class Main {
      private int findPar(int u, int parent[]) {
            if(u==parent[u]) return u;
            return parent[u] = findPar(parent[u], parent); // path compress
      }
      private void union(int u, int v, int parent[], int rank[]) {
            u = findPar(u, parent);
            v = findPar(v, parent);
            if(rank[u] < rank[v]) {
                  parent[u] = v;
             }
```

```
            else if(rank[v] < rank[u]) {
                parent[v] = u;
            } else {
                parent[v] = u;
                rank[u]++;
            }
        }

    void KruskalAlgo(ArrayList<Node> adj, int N){
        Collections.sort(adj, new SortComparator());
        int parent[] = new int[N];
        int rank[] = new int[N];

        for(int i = 0;i<N;i++) {
            parent[i] = i;
            rank[i] = 0;
        }

        int costMst = 0;
        ArrayList<Node> mst = new ArrayList<Node>();
        for(Node it: adj) {
            if(findPar(it.getU(), parent) != findPar(it.getV(), parent)) {
                costMst += it.getWeight();
                mst.add(it);
                union(it.getU(), it.getV(), parent, rank);
            }   }
        System.out.println(costMst);
        for(Node it: mst) {
            System.out.println(it.getU() + " - " +it.getV());
        }}
```

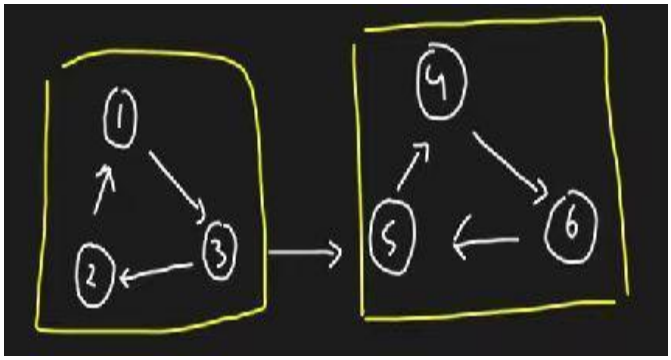
D. TOPOSORT ALGO (KHAN'S ALGO (Bfs) & KOSARAJUS (Dfs))

1. Kosaraju's Algorithm for Strongly Connected Components(SCC)

Given a Directed Graph with V vertices (Numbered from 0 to V-1) and E edges, Find the number of strongly connected components in the graph.

Example: Input:



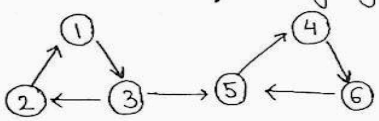


Explanation:
The idea behind SCC is that if we start from any node in a component, we must be able to reach all other nodes in that component. Note that by components here, we mean that we group certain nodes in the graph that meet the condition for every node in that component. The marked components in the following graph are from an SCC.
Solution N Intuition: The idea behind KosaRaju's algorithm is to do a DFS in a controlled fashion such that we won't be able to go from one SCC to another. One DFS call would visit all the nodes in an SCC only.

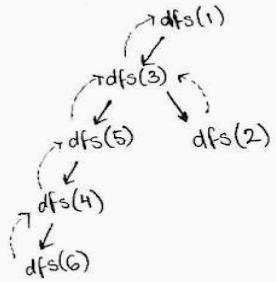
Approach :

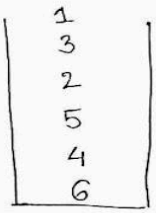
- Sort all the nodes according to their topo sort(loosely based topo sort as we may have cycles here)
- Transpose the graph i.e reverse all the edges of the graph
- Use the topo sort or the increasing order of finishing time to find the strongly connected components using DFS.
- Following is the demonstration of the above algorithm:

Consider the following graph:

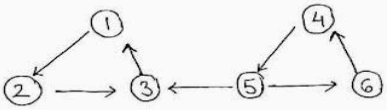


Step 1 : Find the topo sort



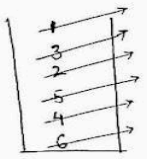


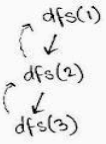
Step 2 : Transpose the graph (Reverse all the edges)

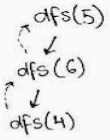


The advantage of this step is that we are able to cluster our scc. Notice how we won't be able to go from node 3 to node 5 now.

Step 3 : Do a DFS on the transposed graph using the toposort.







scc 1 : 3 2 1

scc 2 : 4 6 5

Note : After completing a DFS call for node 1, we do not consider nodes 3 and 2 in the stack as they are already visited.

Similarly after doing a DFS call for node 5, we do not consider nodes 4 and 6 in the stack as they are already visited.

Code:

```
class Main {
      private void dfs(int node, Stack<Integer> st, ArrayList<ArrayList<Integer>>
adj, int vis[]) {
            vis[node] = 1;
            for(Integer nbr : adj.get(node)) {
                  if(vis[nbr] == 0) {
                        dfs(nbr, st, adj, vis);
                  }
            }
            st.push(node);
      }
      private void revDfs(int node, ArrayList<ArrayList<Integer>> transpose, int
vis[]) {
            vis[node] = 1;
            System.out.print(node + " ");
            for(Integer it : transpose.get(node)) {
                  if(vis[it] == 0) {
                        revDfs(it, transpose, vis);
                  }
            }
      }

    void kosaRaju(ArrayList<ArrayList<Integer>> adj, int n) {
        int vis[] = new int[n];
        Stack<Integer> st = new Stack<Integer>();
        for(int i = 0;i < n; i++) {
            if(vis[i] == 0) {
                  dfs(i, st, adj, vis);
            }
        }
     ArrayList<ArrayList<Integer> > transpose = new ArrayList<ArrayList<Integer> >();
            for (int i = 0; i < n; i++)  transpose.add(new ArrayList<Integer>());

            for(int i = 0; i < n; i++) {
                  vis[i] = 0;
                  for(Integer it: adj.get(i)) transpose.get(it).add(i);

            }

            while(st.size() > 0) {
                  int node = st.pop();
                  if(vis[node] == 0) {
                        System.out.print("SCC: ");
                        revDfs(node, transpose, vis);
                        System.out.println();
                  }
            }    }
```

1. **Course schedule**

2.

E. UNION FIND/DISJOINT SET